Aarhus University
Computer Science Dept.
Ny Munkegade, Bldg. 540
8000 Århus C, Denmark

Cooperative Authoring using Open Spatial Hypermedia

Dirk Bucka-Lassen 912181
Claus Aagaard Pedersen 900755
Ólavur H. Reinert 901001

21st December 1998

Advisory Comittee

Kaj Grønbæk
Peter J. Nürnberg

Abstract

Spatial hypermedia systems support the information analysis domain. In the information analysis problem, some information space is given that contains information units; generally, the units are represented visually. The task of the analyst is to arrange the units spatially to reflect relationships between them. These relationships represent an implicit structure that emerges gradually during the process. The task can be supported by a spatial parser that interprets the spatial layout of units to discover its implicit structure. The parser may create an explicit representation of the implicit structure it finds.

Information analysis is of benefit in its own right for intellectual clarification and structuring of information. It also occurs as a subtask of larger tasks, in which case, the structure produced is reused in later stages. System support of seamless reuse of spatial structure in other (hypermedia) contexts would be desirable.

Supporting cooperative structuring of information introduces several benefits. For example, it allows the development of a better common understanding of a structure between coworkers through participation in its creation; and, it allows specialists to cooperate, each contributing with an individual skill set, thereby enhancing the resulting structure.

This thesis proposes three requirements for next-generation spatial hypermedia systems to realize a solution combining the observations mentioned above. Firstly, both the information space, and the explicit structure derived from it, should be stored persistently. Consistency between the two should be defined and enforced by the system. Secondly, the spatial hypermedia service should be integrated into a CB-OHS. It should be open because openness facilitates application integration, thereby enabling ubiquitous use of spatial structure; and, it should be a component of a CB-OHS because this facilitates cross-domain reuse of spatial structure. Thirdly, there should be support for collaborative creation and use of spatial structure.

Based on these requirements, we have implemented the CAOS system, a collaborative and open spatial hypermedia structure service. The CAOS spatial editor is our sample spatial client.

The CAOS server is designed to operate as an open spatial structure service component in a CB-OHS. Specifically, our target framework is the Construct CB-OHS framework. The spatial structure service includes a behaviour component that implements a spatial parser, which automatically updates the explicit structure whenever changes occur to the spatial structure. Such strict synchronisation promotes the explicit structure to a status of accurate and reliable first-class structure; and, it resolves inconsistency issues, which is particularly important in a collaborative context. For efficiency reasons, the parser is incremental, so that only regions of the space actually affected by changes are re-parsed.

Both spatial structure made by users, and explicit structure generated by the spatial parser, are stored persistently in a hyperstore. Both the spatial structure service component, and the spatial parser, store their domain-specific structural abstractions using a generic structure model served by the hyperstore. The spatial service is a mediator between applications and hyperstores; it performs structure processing to transform spatial structure to general structure, and vice versa. Storage of all structure types in a common hyperstore facilitates cross-domain structure exchange by using the generic structure model as an interchange format between structure services.

Multiple clients can edit a space simultaneously, each with instant feedback of all state changes made to the space. CSCW functionality has been adapted and tailored to the spatial area of the information analysis domain so that concurrency of work is

maximised. The support provided for collaboration relies on concurrency control and notification mechanisms in the hyperstore to work.

Many aspects of the functionality of the CAOS spatial editor are inspired by VIKI (e.g., its expressive strength). A special frontend feature has been introduced that takes advantage of the services offered by the spatial structure service component. The explicit structure found by the spatial parser is indicated to users graphically. This permits users to engage in an indirect dialog with the parser, by inspecting its results, to verify that the explicit structure it generates is what was intended.

This thesis demonstrates the feasibility and the utility of implementing a spatial structure service component in a CB-OHS. Having persistent explicit spatial structure facilitates structure sharing and instant visual feedback of parsed structure. The introduction of an automatic and incremental spatial parser is an important enabling component in the final system. A synergy effect is achieved by simultaneously solving the problem of structure inconsistencies (which are particularly aggravating in a collaborative setting) and by removing potential efficiency bottlenecks (due to frequent reparsing).

The thesis starts with a general introduction to the three fields of research presented in this thesis, namely: openness; spatial hypermedia; and, collaboration. A scenario is presented to illustrate our initial ideas. An analysis of the scenario reveals which fields of the literature that have to be studied and treated, to form the basis for a later realisation of the scenario through an implementation.

The literature survey addresses openness; spatial hypermedia; and, collaboration support. Each field is characterised, and different systems belonging to each field are analysed. The current state of the art within the respective fields is highlighted. The thesis chapter refines our focus from the scenario analysis, by relating theory from the literature with our ideas. From that we make a concise outline of the problems we wish to solve in the thesis, and provide motivation for doing so.

Our project work is described in terms of its conceptual design, stressing general aspects of the system features needed to support the required functionality. An outline of how involved technical issues are handled is also given. The details of the prototypical implementation is presented next, with emphasis on the parts that are of most general interest.

The evaluation of our project work compares our prototypical implementation against both the scenario, and the literature survey, respectively. This reveals how well our implementation supports our goals. The future work chapter outlines in what directions we see ways to enhance our implementation. Finally, the conclusion summarises the experience and valuable knowledge that has been gained during the course of this project.

Contents

1 Introduction **1**
1.1 Fields of interest . . . 2
1.1.1 Hypermedia domains . . . 2
1.1.2 System architecture . . . 4
1.1.3 Collaboration support . . . 5
1.2 Choice of focus . . . 6
1.3 Summary . . . 7

2 Scenario **8**
2.1 Scenario . . . 8
2.1.1 Single-user scenario . . . 8
2.1.2 Multi-user scenario . . . 9
2.1.3 Interoperability scenario . . . 10
2.2 Analysis . . . 10
2.2.1 Single-user system . . . 10
2.2.2 Multi-user system . . . 11
2.2.3 Interoperability of systems . . . 11
2.3 A cooperative open spatial hypermedia system . . . 12
2.4 Summary . . . 14

3 Literature Survey **16**
3.1 Hypermedia system architecture . . . 16
3.1.1 Terminology . . . 16
3.1.2 Systems . . . 20
3.1.3 Summary . . . 27
3.2 Spatial hypermedia . . . 28
3.2.1 Terminology . . . 28
3.2.2 Characterisation . . . 31
3.2.3 Systems . . . 33
3.2.4 Summary . . . 44
3.3 Computer supported cooperative work . . . 44
3.3.1 Terminology . . . 45
3.3.2 Systems . . . 51
3.3.3 Summary . . . 60
3.4 Summary . . . 61

4 Thesis **62**
4.1 Analysis . . . 62
4.1.1 System architecture . . . 62
4.1.2 Spatial functionality . . . 66
4.1.3 Collaboration support . . . 68
4.2 Synthesis . . . 72
4.2.1 Motivation . . . 72
4.2.2 Analysis of issues . . . 73
4.3 Our focus . . . 74
4.4 Summary . . . 75

5 Conceptual Design **76**
5.1 Open hypermedia architecture . . . 76
5.1.1 CB-OHS . . . 76
5.1.2 Our CB-OHS . . . 78
5.1.3 Summary . . . 80
5.2 Spatial model . . . 80
5.2.1 Spatial structure abstraction . . . 81
5.2.2 Spatial structure . . . 82
5.3 Spatial server . . . 83
5.3.1 Spatial server architecture . . . 84
5.3.2 Spatial parser . . . 86
5.3.3 Collaboration support . . . 90
5.4 Spatial client . . . 92
5.4.1 Client view of spatial structure . . . 92
5.4.2 Spatial service communication . . . 92
5.4.3 Caching of space units . . . 93
5.4.4 Our spatial editor . . . 94
5.5 Spatial interface . . . 98
5.5.1 Server interface . . . 98
5.5.2 Client interface . . . 100
5.6 Summary . . . 102

6 Prototypic Implementation **103**
6.1 Open hypermedia architecture . . . 103
6.1.1 Construct . . . 103
6.1.2 Our components . . . 106
6.1.3 Summary . . . 106
6.2 Spatial model . . . 106
6.3 Spatial server . . . 107
6.3.1 Spatial server model . . . 107
6.3.2 Spatial server architecture . . . 107
6.3.3 Spatial parser . . . 109
6.3.4 Collaboration support . . . 114
6.3.5 Summary . . . 116
6.4 Example client: a spatial editor . . . 116
6.4.1 Process architecture . . . 116

6.4.2 Spatial model . . . 117
6.4.3 Graphical user interface . . . 118
6.4.4 Collaboration support . . . 119
6.4.5 Summary . . . 120
6.5 Spatial interface . . . 120
6.5.1 Server interface . . . 120
6.5.2 Client interface . . . 121
6.6 Summary . . . 121

7 Evaluation **122**
7.1 Evaluation of functionality . . . 122
7.1.1 Spatiality . . . 122
7.1.2 Openness and interoperability . . . 129
7.1.3 Collaboration support . . . 131
7.1.4 Summary . . . 132
7.2 Evaluation against scenario . . . 133
7.2.1 Supported functionality . . . 133
7.2.2 Unsupported functionality . . . 134
7.2.3 Summary . . . 135
7.3 Summary . . . 136

8 Future Work **137**
8.1 Future work on CAOS . . . 137
8.1.1 Spatial service . . . 137
8.1.2 Spatial editor . . . 138
8.2 Future work on spatial hypermedia . . . 139
8.2.1 Generalised spatial model . . . 139
8.3 Summary . . . 144

9 Conclusion **145**
9.1 Summary of research . . . 145
9.2 Research conclusions . . . 146
9.3 CAOS . . . 147

Chapter 1

Introduction

Hypermedia, in its broadest sense, is a means to organise information. The need for organisation has always existed, but with today's growing supply of information, the demand for organisation is also increasing.

Today, bits of information are taken from many different media and contexts; and, with modern tools, such as the computer and the Internet, it has become easy to retrieve virtually infinite amounts of information. Although these tools stress the problem and the concept of information overload, it has in fact been discussed for a long time [Bus45]. In fact, the hypermedia field originates in the acknowledgement of this problem, and it aims at solving or alleviating it.

The question is no longer whether one has access to information or not, but how we structure this information. This implies that structural abstractions will become more important, since the value of information is not only defined by the data itself, but more and more by the structures it is part of. The overwhelming amount of information we receive today makes filtering and structuring a necessity.

Dealing with large amounts of information spurs the incentive to investigate new structural abstractions. New hypermedia domains emerge, and the diversity of hypermedia systems increases. In general, structure is still represented by means of traditional data types, because so far there is no consensus on a generic way to represent structure.

Within the field of structural computing [Nür97], it has recently been suggested that the level of abstraction used when processing structure should be raised. The concept of structure should be made explicit, thus making structure a first-class abstraction, instead of modelling structure implicitly using traditional data types. The concept of a generalised structure element has been suggested as the basic building block in such systems. Practical experience in a larger scale in this field is still lacking.

The work described in this thesis considers the combination of mainly three areas of hypermedia research: the information analysis domain, the openness of hypermedia systems, and the support for collaboration. Our goal is to show that a collaborative open spatial hypermedia system, being the result of such a combination, is a suitable tool for alleviating the problem of information organisation and analysis addressed above.

This chapter proceeds with a brief introduction of the three hypermedia fields mentioned above, followed by a more concise description of what the focus of our work will be throughout the rest of this report.

1.1 Fields of interest

As hinted above, our main fields of interest are: the information analysis hypermedia domain, the openness of hypermedia systems, and the support for collaboration. These three subjects are presented in the sections below. Each section introduces the fields they are part of, followed by an introduction of the subjects themselves. A motivation for why we assume the chosen subjects can help in alleviating the organisation problem rounds off each introduction.

1.1.1 Hypermedia domains

Hypermedia systems typically focus on some specific problem domain. This has led to a variety of systems that support particular domains, but what are hypermedia domains?

Hypermedia domains are identified by the discovery of a problem in which special structural abstractions with specific properties would be a helpful aid in solving the problem. For example, the classification domain was introduced by biologists wanting support for the task of creating taxonomies of the species they were researching.

The issue in hypermedia upon the introduction of a new domain is *not* to express the domain structure using some general model of structure, and to let users work with that. On the contrary, users demand to be able to work directly with domain specific structure, in order to exploit the specific features of such structure. By maintaining domain specific restrictions on the structure users work with, tools can be developed that utilise the properties of the domain specific structure to perform syntactic and/or semantic checks, and maybe to perform structural computations that are only relevant within the domain.

The discovery of new problem domains has led to the introduction of new hypermedia systems supporting the domain specific structures and functionality—see Table 1.1 for an overview. These systems were often made explicitly to support a particular new domains. Thus, a general characteristic in all of them is that they lack support for such aspects as openness and/or collaboration, because they were outside the scope of intention of the system builders. Also, the systems in this group are rarely based on general hypermedia frameworks—most of them have been built from scratch [NLW98].

Information analysis domain

During the 1990's, much attention has been given to systems enabling people to access information easily. Different ways to present and navigate information structures have been presented, and much research has been devoted to the area of structure navigation. The World Wide Web is a prominent example of such a navigational hypermedia system.

Domain	*System type*	Example implementation(s)
associative	navigational	Chimera [ATW94], HyperDisco [WL96], Microcosm [DKH94], SPn/HBn [LS94], DHM [GT94], WWW
information analysis, analytical	spatial	VIKI [MSC94], WebSquirrel [ES95]
classification	taxonomic, set-based	HyperSet [Par91], TaxMan [Nür97]
argumentation support	issue-based	gIBIS [CB88], PHIDIAS [MB+90], AAA [SS90]
literary		Storyspace [Joy91]
art		

Table 1.1: *A list of hypermedia domains; the names of the corresponding hypermedia system types; and, names of some example implementations.*

However, not much attention had been paid to the problem of structure creation. It was observed [MSC94] that traditional authoring systems were restrictive and impractical to use, because they were based on the construction of explicit structure. There was no opportunity for authors to explore the structure they wanted to express. Thus, authors were often forced to articulate tacit knowledge or to commit to structure prematurely.

System designers today are aware of the fact that users tend to work with structure in an exploratory way. This raises the need for a hypermedia domain with characteristics that are useful for this way of working with structure, and capable of expressing the structure needed.

It was observed that people often apply informal spatial methods when structuring material, and that structuring is a process in which structure emerges gradually during the process [Mar92]. For example, if a person is asked to organise a stack of paper, most people perform the task by sorting papers into stacks on a desk. Stacks may be spatially related (put next to each other) to indicate a certain relation of meaning between the papers in the stacks. Other observations noted the use of Post-It notes attached to a wall to organise complex information. The spatial arrangement of the notes indicated how people structured the information on the notes.

From this, the information analysis domain was made to support construction of hypertext structure through exploration and gradual development. It supports visual/spatial methods of constructing structure that take advantage of the human perceptual system, spatial and geographic memory, and spatial intelligence in general. These structure building abstractions support a smooth transition from informal to formal structure, by allowing the use of a broad set of flexible visual cues that provide a high level of expressiveness to users. Emerging structure can enhance the understanding of a structuring problem and let users get hands on experiences with a problem while they "learn by struc-

turing". The possibility to analyse visually expressed structure by a computer to create explicit structure can support development of common understanding for created visual structure, and thereby for the problem at hand. Automated analysis of visual structure is called spatial parsing. The systems that support this kind of structuring are called spatial hypermedia systems.

Motivation

The main aspects of spatiality that we consider important properties in the task of structuring information are: the ability to handle large structures; the ability to allow structure to emerge while working with it; flexibility with respect to expressiveness; and, the provision of analytical tools that can help the user in understanding the structure at hand.

1.1.2 System architecture

In contrast to the research concentrating on new hypermedia domains, research has also been conducted on the architecture of hypermedia systems and its consequences.

Early hypermedia systems were characterised by their monolithic architecture and design, which forced users to use some bundled user interface in order to access hypermedia functionality [AMY88, EWN73, HMT87]. Those systems were mainly navigational. Even though they were all within the same problem domain, the lack of standards made it almost impossible for the systems to exchange structures [LS94].

This led to the idea of open hypermedia systems that focus on providing an interface for an open hypermedia service and the separation of structure and data. The interface can be used by external applications tailored for use with the hypermedia service, while the separation of structure and data allows structure to be applied to an open set of data formats. These systems are typically characterised by a client/server or three-layer architecture (application, link service, structure store).

The appearance of new domains lead to the desire of being able to reuse the structure produced in one domain in other structural domains. In [NWL98], authoring is mentioned as a task that can consist of different processes/phases, (e.g., pre-writing and writing). In this scenario, an information analysis tool may be used in the pre-writing phase to obtain the overall structure of the items to be discussed. Subsequently, associative structure is used during the writing phase to interconnect relevant issues in the written work. It would be convenient for users to be able to base their writing on the structure produced in the pre-writing phase. This kind of reuse is an aspect of the interoperability issue that is attracting more and more attention.

Openness and interoperability

Recently, the open hypermedia systems working group (OHSWG) has suggested a so called component-based open hypermedia system (CB-OHS) framework that tries to cope with the openness and interoperability issues addressed above.

This framework is a three layer architecture, with hyperstores at the bottom layer; structure services at the middle layer; and, domain tailored applications at the top layer. A hyperstore offers storage of general structure elements. Structure services specialise the general structure elements to domain specific structure elements, thereby defining a domain. Applications work on domain specific structure elements supplied by structure services.

Traditional open hypermedia systems are supported by the two upper layers. New domains are supported by introducing new structure services. Reuse of structure across domains is supported by the generic structure hyperstore that is provided to all structure services.

The first practical experiences with the suggested CB-OHS framework are about to emerge. This means that implementations of structure servers reflecting well-known domains, and tailored applications for these, are still lacking. Actually, this thesis is concerned with the design and implementation of a spatial structure service, thereby providing experiences with CB-OHSs.

Motivation

The properties that we consider important in the task of organising structure with respect to hypermedia system architecture are: the ability to structure any kind of data; the general provision of hypermedia services, applicable by any application; and, the explicit support for other domains to use the structure produced by another domain.

1.1.3 Collaboration support

Support for collaboration, or so called computer supported cooperative work (CSCW), has been a separate field of research for many years. The adaptation of it in the hypermedia field is an extension of the possible applications of hypermedia.

This adaptation reflects another evolutionary direction of hypermedia systems, emerged through the desire to provide support for collaboration (i.e., multiple users using shared hypermedia structure, either asynchronously or synchronously). Systems supporting cooperativity have been closed so far (from an architectural point of view) because they enforce certain client applications on users, despite having a client/server system architecture (e.g., SEPIA [SH$^+$92] and DOLPHIN [SGHH94]). Only recently, there have been attempts at specifying open architectures for collaboration, the framework suggested by the OHSWG being one of them.

Motivation

Supporting cooperative work is a natural requirement when faced with the task of structuring information. Often, the structure is to express a group of people's perception of the information at hand. Collaboration with other users supports the development of a common understanding between the people participating in the organising task.

1.2 Choice of focus

The previous section has argued for the positive properties of each of the considered fields. A system implementing these properties will become a powerful tool with respect to organising information. It will provide spatial hypermedia structuring properties that are available to anyone through the provision of a spatial protocol. Users will be able to work together, structuring information in collaboration. Additionally, the resulting structure will be available to other systems for further use.

This thesis aims at designing and implementing such a collaborative open spatial hypermedia system. The system is called CAOS, which stands for Cooperative Authoring using Open Spatial hypermedia. The following are the main topics of this thesis, briefly motivating why they are of interest to us.

- *Spatial hypermedia:* Information analysis by spatial hypermedia has our interest, because the field in itself presents structure abstractions powerful enough to enable emergent structuring.

  We will investigate how to make a spatial parser incremental, and how to provide feedback from the parser to the users. We would like to investigate how spatial hypermedia works in combination with other hypermedia domains, by allowing explicit structure originating from spatial hypermedia to be reused in other hypermedia domains. We will also look at how spatial hypermedia can be applied in a collaborative setting, since work seems to be missing in this direction.

- *Openness and interoperability:* Hypermedia is a means to structure information. Since the information to be structured is likely to originate from applications external to the hypermedia system, making the system open will allow integration between external applications and hypermedia systems. Spatial hypermedia systems are no exception to this rule. Our focus will include aspects of openness, such as the definition of interfaces between spatial clients and servers.

  The nature of spatial hypermedia suggests itself to be the first step in an information authoring process, because of its support for evolving structure. If the resulting spatial structure is to be used by other domains, we need some way of translating it across domains. This is what we call structure processing. One possible solution, supported by the CB-OHS framework, would be to let all domains use a generic structure element as their fundamental storage unit.

  Our focus will be on providing an open spatial structure service component in a CB-OHS. By using such a framework, our structure service will be open. Also, the structure produced by it will be open for cross-domain structure re-use, i.e., interoperability.

- *Collaboration support:* People often cooperate when authoring documents. Computer supported collaborative work (CSCW) systems aim at providing computerised tools that allow people to cooperate when using com-

puters to do their work. They also introduce opportunities to support collaboration in situations that are difficult or cumbersome to achieve in a non-computerised environment. For example, CSCW systems can support collaboration despite geographic or temporal separation.

Our focus on cooperativity support will be on adapting CSCW functionality to the spatial hypermedia setting; and, to discuss the system infrastructure required to support collaboration. Adaptation consists of applying CSCW concepts to the spatial hypermedia in a suitable and useful way (e.g., finding the relevant set of coupling modes, what group awareness means in spatial hypermedia, determining the appropriate lock granularity in various contexts, etc.). Infrastructure requirements include session handling, notifications, shared data management, etc.

1.3 Summary

This chapter has outlined the motivation behind the hypermedia field, and has given our proposal for a tool that alleviates the problem of organising information. Subsequently, the three fields of research, defining our proposed system, were presented and motivated. A presentation of the information analysis domain, on which spatial hypermedia is based, was followed by an introduction to hypermedia system architecture, which considers the openness and interoperability issues. The properties of collaboration support were briefly introduced. The following section presented the focus of this thesis.

The rest of this thesis proceeds as follows: Chapter 2 begins by presenting some scenarios that outline the functionality that a collaborative open spatial hypermedia system should provide. Subsequently, we derive the key concepts that characterise the functionality requirements described by the scenarios. Chapter 3 surveys the literature in which the key concepts are addressed. Chapter 4 presents our thesis, in which we narrow down our approach to designing a system that meets the key requirements. Chapter 5 presents the conceptual design of a spatial hypermedia system that is able to satisfy the key requirements. Chapter 6 describes our prototypical implementation based on the conceptual design. Chapter 7 evaluates to what degree the prototype satisfies the key requirements from the usage scenarios; the prototype is considered to be the result of one iteration of a cyclic process of evolution. Chapter 8 considers possible future work that could improve the prototype so that it better matches the requirements stated in the scenario, the key requirements, and, the thesis. Also, it points out some directions of development (scenarios) that cross the boundaries established by the scenario in chapter two. Finally, Chapter 9 presents our conclusions gained from working on this thesis.

Chapter 2

Scenario

This chapter presents a scenario that exemplifies the usefulness of a spatial hypermedia system endowed with the features outlined in the introduction (Section 1.2). It demonstrates how the system can be used to solve a concrete task using spatial hypermedia.

The scenario explains how specific system properties (e.g., openness, and cooperativity) benefit users. It is sub-divided into three parts. It is followed by an analysis section in which each sub-scenario is analysed separately, followed by a presentation of a system that supports all requirements.

2.1 Scenario

As described in the introduction (Section 1.2), our focus is to make a collaborative open spatial hypermedia system. To better express our ideas, we present some scenarios that capture and exemplify what we envision might be typical tasks performed using such a system.

2.1.1 Single-user scenario

Allan is interested in 19th century European philosophy; at the moment he is conducting research on a book on Kierkegaard. He uses a spatial hypermedia system during the research phase of the project, to organise his material, and to perform brainstorming. The spatial tool aids him in experimenting with various ways to structure his material and as a convenient way of recording his ideas as they occur.

Allan activates his favourite editor application, extended to support spatial hypermedia facilities. He then opens an information space in which he collects documents related to the subject. Documents are represented to Allan as small paper notes (icons) on a big table (the space).

Allan has collected some papers in advance. He adds some icons that contain references to hard-copy documents; others that contain pointers to documents if they are available in electronic form. There are even icons in the space that are empty—Allan uses them as placeholders and reminders for content he wants to investigate later.

More icons are added to the space as Allan searches his sources for more information. One source of information to him is the Internet. A web browser extended for use with spatial hypermedia allows him to gather web pages into his spatial system easily. Each time Allan finds an interesting page, he marks it, so that a reference to the page is automatically added to his information space. Also, whenever Allan needs a closer look at what a reference is about, in order to organise it properly, the spatial system can display the document using the browser (if it is available electronically).

While Allan is busy collecting information, he also discovers some patterns in the material that he wants to record. He spatially sorts the material tentatively, by moving related icons near each other. In some cases, Allan makes patterns with the icons, in the form of piles and lists, to indicate more detailed structure between the icons. Some documents are marked by Allan in a way that makes them stand out and capture his attention, because they were eye-openers to Allan, or because they point at material that requires further research. Allan devises a system for marking such icons as he goes along, using colours and icon shapes as visual mnemonic tags.

The system helps Allan to structure his material by providing various visual structure indicators whenever the system discovers familiar patterns of icons (e.g., piles, lists, etc.) in the way Allan has organised his space. He never asks the system to look for the structure—it happens automatically in the background while he is working.

2.1.2 Multi-user scenario

Suddenly, the phone rings. It's Brian, one of Allan's colleagues, who calls to tell Allan about his trip to Copenhagen, where he discovered a hitherto unknown diary written by Kierkegaard. It contains many personal reflections and comments on the seminal works of the philosopher. Brian also happens to use a spatial hypermedia tool, and he has recorded some observations found while reading the diary in a space of his own. Brian allows sharing of a part of his space that is of particular interest to Allan.

Brian starts up a spatial client on his own terminal, and, using a URL to Allan's spatial document given by him, opens the space Allan is working on. Brian's view of the space is shown according to his own preferences—he does not fancy the gloomy colours Allan tends to use. Brian adds his references to the common space, and his additions are immediately shown on Allan's terminal.

Subsequently, because they are both passionately interested in the subject, they agree on working together to organise the new additions to Allan's material. So they start a collaborative work session in the tool. Then, Brian moves his new icons about in Allan's space, showing him where he thinks they belong. While talking on the phone and pointing with their telepointers, they talk and move objects around in the shared space, according to the common understanding of the subject that they develop, as they go along.

2.1.3 Interoperability scenario

Eventually, Allan and Brian end up with some material with which they are both very pleased, and decide to publish some of their findings on the World Wide Web. They both start their web browsers, and browse through the same structure that they worked on as spatial structure before, to check its appearance. Corrections and additions are made in the spatial editor, and they spend some time making some additions here, and brushing up something there. Finally, when they are pleased with the result, they both close their spatial editors. The structure they have made is stored persistently in a hyperstore.

The published parts of their spatial structure are still available to anyone who wants to browse them—a World Wide Web gateway connected to the hyperstore offers navigational structure based on the stored spatial structure. Allan and Brian may, at any time, return to the spatial hypermedia system and further modify the structure in the spatial environment, with changes to the published parts being reflected immediately to people browsing the material.

2.2 Analysis

This section analyses the scenarios above, thereby trying to isolate the requirements of systems satisfying the scenarios, and their properties.

2.2.1 Single-user system

To realise a system like the one Allan uses, many requirements have to be satisfied. The notion of Allan using his favourite editor to organise his documents implies that there is some kind of server that serves structure to the editor; in this case, structure belonging to the information analysis domain. This suggests a client/server architecture, known from traditional open hypermedia systems, that allows an open set of client applications.

The fact that visual indicators are immediately and continuously shown requires some kind of automatic, dynamic parsing of the implicit structure created by the user. In other words, a spatial parser is required that is triggered by user actions that change the existing visual structure. One way to determine explicit structure changes quickly would be to use an incremental parser, in the sense that only the area in the vicinity of the modification is reparsed, not the entire space.

The ability to add new icons to the spatial editor via another application (e.g., a browser) requires either communication directly between the applications or indirectly through a structure server.

The possibility of launching an external application by clicking on an icon requires knowledge about what kind of information the icon represents. This introduces the idea of icons being annotated by some characteristics that describe various properties of the icon/document. In addition, an icon should have some visual indication of the document to which it is referring. A short title text would be one way to indicate this.

The means of expressing structure used by Allan (position, alignment, colour, borders, etc.) call for a spatial editor that provides a rich set of mechanisms that give users freedom of expressiveness when structuring information.

2.2.2 Multi-user system

The notion of multiple users working synchronously on the same spatial document gives rise to new requirements for the system. The multi-user scenario describes a distributed storage system in which spatial documents are stored. Thus, some naming scheme is required to locate and identify spatial documents (e.g., by using universal resource locators (URLs)).

Personalised views is also worth considering, since different people have many good reasons to prefer views on the same spatial document based on individual preferences (Section 3.2.2). It would be desirable to support this by letting users define their personal preferences that control the way spaces are visualised to them. One may consider these mechanisms as an enhancement of the expressive capabilities of the system.

The ability of Brian to add new icons to Allan's spatial document calls for some kind of cooperativity support. Specifically, shared data management functionality is needed to guarantee structure coherence at all times; access control mechanisms are needed to restrict the range of distribution of structure; and, some mechanism for management of shared user interfaces is needed to support collaboration efficiently. The last issue includes support of a notion of "working together", so it includes a concept of group awareness.

The properties of the way users work together also needs to be specified. This is normally defined using sessions that operate in certain coupling modes. A coupling mode specifies what kind of user actions are reflected to other collaborators. The telepointers, for instance, are reflections of user actions (mouse movement) and are a mechanism whereby users work together. The life span of sessions is also an issue worth considering—if Allan and Brian decide to resume work on their space later, do they necessarily have to create a new session to do so, or can sessions be persistent?

Another issue worth considering is to what extent other communication channels, apart from (through) the application itself, should be supported. It is desirable to incorporate some kind of teleconferencing support into an application, thereby making, e.g., phone calls or other external communication, unnecessary.

2.2.3 Interoperability of systems

Working on the same structure across different hypermedia domains is one of the issues presented in the last scenario. But what are the implications of doing this?

Firstly, the structure produced by a structure service must be persistent in order to be shareable in a sensible way. If it could not be stored it would be lost when all (collaborating) users close their applications. This persistence also gives the ability to work asynchronously in cooperation with others. Addition-

ally, things like versioning, access control, etc., are required to facilitate and control collaboration, respectively.

The next issue is how we allow other domains to work on the persistent structure. Different domains have different representations of structure with unique characteristics designed to meet the particular needs posed by the domain. This gives rise to some definition of a structure processing policy and devising a method for how to execute it.

2.3 A cooperative open spatial hypermedia system

The first scenario describes the basic functionality of a spatial hypermedia system. The second scenario extends this functionality to support collaboration, requiring only changes orthogonal to the basic structure service functionality. The last scenario tries to extend the usability of a structure by allowing interdomain reusability. This puts new requirements on the structure and the structure service(s) that generate or manipulate it.

To summarise the requirements of the three scenarios, we will now look at components of a system designed to satisfy the requirements posed by all scenarios.

- *Spatial editor:* To support the task of information structuring, the system must provide a broad set of mechanisms that allow the user to express relations among items. Position, proximity and alignment is one way to express relations spatially; also, colour, shapes, borders, fonts, etc., gives users some means by which relations among items can be expressed. A spatial editor is required to visualise spatial structure to users, and to allow manipulation of it. The editor should support a rich set of tools that allow users to express structure using the mechanisms mentioned.

  We want the editor to be a separate entity that communicates with a server of spatial structure. In other words, it acts as a client to a central spatial structure service. This division opens up the possibility of distributed access to spatial structure services, which is a requirement in the last two scenarios.

- *Spatial structure service:* The storage of spatial structure should not be handled by the structure service itself. Instead, we want to use a structure store, which is a database management system tailored to store structure. In this way, we hope to gain all the benefits of transaction management, reliability, notifications, access control, etc., normally associated with databases. Also, the separation of the structure service and the structure store introduces the potential of distribution; replication; use of multiple structure stores, etc.

  The structure store should be able to store many kinds of structure from various hypermedia domains. It does so by offering storage of generic structure units. The structure service then becomes a client to the structure store, using the generic structure units to store its spatial structure.

The notion of (spatial) structure units having various (domain specific) characteristics (e.g., position, colour, borders, etc.) leads us to the idea of defining generic storage units as an arbitrary set of characteristics.

The following are issues that arise when designing a system consisting of the components mentioned above.

- *Openness:* From the previous two paragraphs, we see that a three-tiered architecture has emerged: The top layer consists of spatial clients; the second of spatial structure services; and, the bottom layer features structure stores. This gives us a three layer architecture that is very similar to the CB-OHS framework (Section 3.1.1). In fact, the CB-OHS framework supports all the desired features we require, which is why we choose to base our system on it, and the CB-OHS framework will be discussed extensively throughout this thesis.

  To support the aspect of openness, we will define an interface for the communication between the structure server and the applications, thereby making the set of applications supportable by the structure server open and extensible.

  A mechanism for naming objects in a distributed system, such as the use of URLs, is also an issue. Users need to be able to exchange references to objects stored in the system using names that can be translated to machince objects.

- *Interoperability:* Our support for interoperability is based on the use of generic structure units to store domain specific structure. When performing structure processing, we want to keep all important information about a persistent unit. This leads to using translation or interpretation as methods by which to do structure processing. Conversion may, as we have explained, destroy "domain irrelevant information" of a structure unit that is not irrelevant at all when used in other contexts.

- *Spatial parsing:* The spatial structure service must support spatial parsing, since this generates structure that is needed to provide continuous visual structure feedback to *all* users. Designing a spatial parser includes deciding what kinds of structure should be recognised, and how to do it. The structure found by the parser is represented explicitly and stored in the hyperstore.

  To do spatial parsing quickly and automatically, parsing should be incremental, in the sense that a re-parse only processes the parts of a space that may have changed. To ensure consistency between the structures produced by the users and the parser, parsing should be triggered by user actions/events. Subsequently, updates to structure made by the parser should be broadcast to interested clients.

- *Communication patterns:* The communication needs hinted at so far suggest the use of an event based communication style between applications

and the structure service, which in turn requires subscription to events. In combination with multi-user facilities, this also solves the distribution problem, according to the multi-user scenario (i.e., changes performed by a user (add new icon) are submitted to all the other users). Indirect communication between the applications via the structure server is chosen.

As mentioned earlier, the multi-user facilities extend the functionality of the spatial structure server by defining some ordered way of interaction between multiple users. Several aspects are involved, such as session management, shared data management, shared user interface management, and, access control.

Additional communication channels are a common requirement in collaborative systems, particularly when offering synchronised collaboration. They enhance the coordination of user's work, and, in the case of spatial hypermedia, enhance the common understanding of the information space.

- *Preferences:* Preferences cover the concept of tailoring the way a system presents or handles spatial hypermedia to users, with the goal of increasing user conception of the problems at hand. Preferences are usually a set of parameters defining how to present or handle structure. In our case, one could imagine preferences specifying how to present structure, and preferences defining what properties the parser should take into consideration when parsing for explicit structure.

2.4 Summary

The previous section discussed several aspects to be considered in the design of a cooperative spatial hypermedia system. These aspects can be described by a set of issues that can be classified into three categories: system architecture, spatial functionality and collaboration support. Here, we present the categories and list the issues in each one.

- System architecture
  - Openness and interoperability
  - Structure processing
  - Structure persistence
  - Naming
- Spatial functionality
  - Spatial expressiveness
  - Parsing
  - Visual structure feedback
  - Preferences

- Collaboration support
  - Session management
  - Shared data management
  - Shared user interface management.
  - Additional communication channels

The next chapter presents a survey of literature that discusses these issues. In the thesis (Chapter 4) we refine the list of categories, based on previous work and experiences described in the literature.

Chapter 3

Literature Survey

This chapter gives a survey of the literature on which our thesis work is founded. The chapter is devided into three sections: Firstly, open hypermedia systems are described. Secondly, the spatial hypermedia domain is addressed. Thirdly, computer supported cooperative work is discussed.

3.1 Hypermedia system architecture

This section looks at the architectures of hypermedia systems. Firstly, we introduce a terminology by providing a walk-through of the evolution of hypermedia systems. Secondly, we describe implementations of open hypermedia systems, based on the terminology, and link them to the different phases in the evolution.

3.1.1 Terminology

The terminology of open hypermedia systems has emerged through the evolution of the systems themselves. Therefore, we present an evolutionary walk-through of the field of hypermedia systems, introducing the main concepts (which will be printed in bold when introduced in the following).

Monolithic systems

The first hypermedia systems were, from an architectural point of view, monolithic systems, effected as a single process, combining the user-interface, linking mechanism and store in one program. This made the (mainly navigational) hypermedia facilities inflexible, since it forced the users to use the enclosed user-interfaces.

Open hypermedia systems

Soon afterwards, the desire to open up these systems with respect to the user-interface appeared. This abstraction of the user-interface allowed an open set of applications to be used as clients that make use of the system's hypermedia facilities. This separation of the hypermedia systems allows explicit distribution of applications [NLS97].

Additionally, the linking mechanism and the hypermedia store was abstracted, allowing an open set of hypermedia stores (now called **structure stores**) to serve the linking mechanism (now called **link service**). More and more structure stores were backed by databases, thereby allowing transaction management, notification and access control of the stored structure. This separation obviously allowed the storage layer to be distributed.

This gave rise to two threads of research [WL97] on open hypermedia systems. The first thread was concerned with the link service providing hypermedia linking functionality to an open set of third-party applications independent of their data model and display functionality [DHH+92, WL97]. The second thread focused on **structure aware stores** or so called HyperBase Management Systems (HBMSs) providing services such as versioning and access control, for *both* structure and data. These systems present the first attempts at providing hypermedia-in-the-large [LS94]. The merge of these two threads of research led to the open hypermedia community [WL97] of which the Open Hypermedia Systems Working Group (OHSWG) is a product.

The original purpose of the OHSWG was to define an open hypermedia protocol for open hypermedia system (OHS) clients. This purpose has evolved into an effort of standardising *general* aspects of open hypermedia systems. Other attempts have been made to provide guidelines for hypermedia systems (e.g., the Dexter Hypertext Reference Model [HS94]).

The OHSWG specifies two main requirements for **open hypermedia systems** [Wiib]:

- An open hypermedia system must allow an open set of applications to participate in the hypermedia services.

- An open hypermedia system must support an open set of data model formats.

The first item implies the existence of a well-defined interface to the system, so that arbitrary applications can be enabled to use the provided services. The second item implies the separation of structure and content, because embedded structures alter the data format of an application, which in turn requires modifications to the application. This separation expresses the **orthogonality** of structure to the data storage (i.e., structure is superimposed on data). Often, structure is promoted to a first-class object, enabling it to be referred to and enjoying the same status as data. For a typical hypermedia architecture of this type, see Figure 3.1.

Although this definition is (more or less) accepted throughout the hypermedia community [DHH+92, ATW94, GHMS94] there is still a need for consensus to satisfy the overall goal in the design of OHSs: "... the design of an OHS is to produce a general set of hypermedia services that can be used by other applications, programs and services in the computing environment" [Wiib]. This goal requires **interoperability** between OHSs. It is the interoperability requirement that implies some common guidelines and standards. This is the main aspect addressed by the OHSWG.

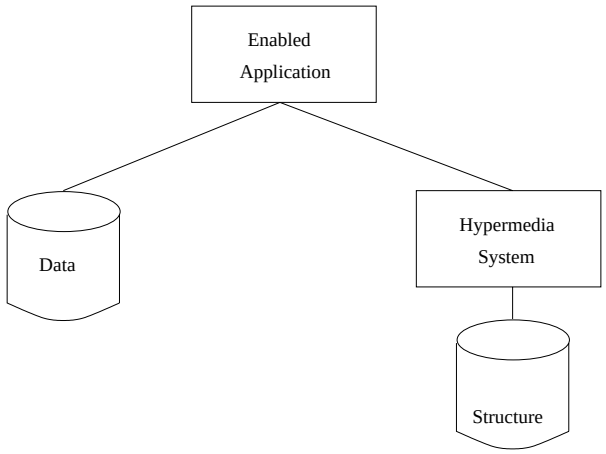

Figure 3.1: *A traditional open hypermedia architecture.*

Component-based open hypermedia systems

The emergence of new hypermedia **domains** (e.g., information analysis, classification and argumentation support), led to the acknowledgement that different domains *share* general structural problems such as integrity, versioning, access control, etc. [NWL98]. This, in turn, led to the idea of providing these services in a generic way so that servers effecting the abstractions of domains could access them, resulting in domains being architectural peers to each other. The introduction of **"generic structure"**, common to all domains, allowed the provision of these shared services in a hyperbase. The generic structure is a kind of structural atomic building block from which all other abstractions (including data) are derived [NLS97].

The concept of *generic structure* reflects a third thread of research in open hypermedia systems suggesting *hypermedia as a computing paradigm* [NLSS96]. In this field, hypermedia domains are viewed as specialisations of **structural computing**, which is the philosophy of computation in which structure is more important than data ("primacy of structure over data" [NLS97]).

The structural computing philosophy, in addition to the desire of interoperability, resulted in a recently suggested **component-based open hypermedia system** (CB-OHS) framework, proposed by the OHSWG. This framework represents the next logical step in the evolution of hypermedia systems, in that it opens the link service layer (now called **structure services**). In other words, it is now possible to support an open set of structure services. Domain specific **structural abstractions** are specialisations of generic structure maintained by the hyperbase.

Each of these structure servers resembles, somehow, a traditional OHS link server, in that it serves an open set of applications with tailored domain specific structural abstractions. Multiple structure servers use the common structure aware hyperbase that provides common services on generic structure. This common base layer provides one possibility for the alleviation of the interoperability

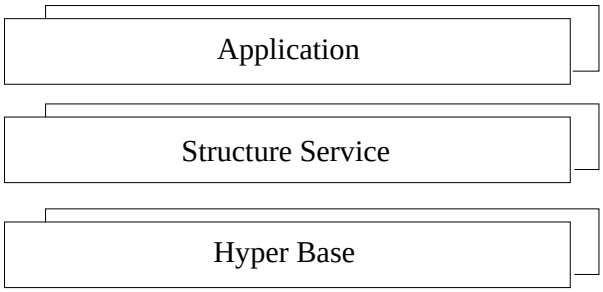

Figure 3.2: *The CB-OHS architecture.*

problem, as it allows structure servers to read other structure server's products. Thereby, the generic structure of the hyperbase somehow resembles an interchange format as part of a hypermedia-in-the-large framework, introducing "new" problems [LS94].

Figure 3.2 depicts the architecture of a CB-OHS. It is a general flexible and extensible component-based open hypermedia framework, complying with all the OHSWG standards, reference models and reference architectures [WN98]. According to the framework, there are three points to address: the functionality of OHS **backends** (the hyperbase), the functionality of the **middleware** (the structure servers), and how components communicate the provided functionality (interfaces of the hyperbase and the structure servers).

The structure service can be further abstracted into a behaviour component and a structure processor component [LS94, NLS97]. In this case, the structure processor (or **Sproc**) defines domain specific structure (the syntax), while **behaviour**s define computations over this structure (the semantics).

As mentioned before, there are still some problems that need to be further addressed. Firstly, there is the **structure processing problem**. How should the generic structure be handled? Secondly, the **integrity problem** is concerned with modifications on data that is not maintained by the HBMS, but is referenced by structure in it.

Hypermedia operating systems

Following up on the CB-OHS architecture, it has been suggested that the common structural building block should be integrated into the computer's operating system [NLSS96]. Thereby, the operating environment allows not only abstractions such as data (files) and data sets (directories, which actually are some kind of structural abstraction), but also structure and structure sets.

Facilities like creation, deletion, manipulation, mapping between data and structural objects, locking and versioning should all be supported by the operating system.

This, in turn, leads back to a two-layer architecture, in that the HBMS is integrated into the operating system, leaving the structure service and application as the main components of a hypermedia system.

The merge of the hyperbase with the operating system implies that users

always access resources (data or structure) through the same interface (the operating system), thereby making it much easier to guarantee/enforce integrity and consistency of data and structure. This is so, because the operating system now maintains both data and structure, thereby being aware of changes in either one of them, and in turn, able to handle possible implications on the other.

An example of another advantage of hypermedia operating systems, aware of structure, are new means of *semantic locality* [NLSS96]. If an object is read into memory, it is likely that objects referred to are accessed next. Thus it is possible to design pre-fetching algorithms based on structural relations instead of traditional operating systems that base locality on distance in logical memory.

Furthermore, integration of well-known structure services (e.g., the navigational domain) into the operating system would provide even higher structural abstraction levels to applications. This would result in navigational linking facilities that, because they are basic operations, are able to link between files, directories, structures and structure sets from the operating system shell (such as cut, copy and paste [Mey86]).

As new hypermedia systems and domains emerge, new problems will emerge, too. What we need now are experiences concerning the degree to which these new systems help or solve known problems.

Summary

To summarise the evolution of hypermedia systems and what "components" were abstracted we refer to Figure 3.3. Monolithic hypermedia systems (HS) were characterised by bundling tools, link mechanisms and storage. The first abstraction was aimed at the application (APP) layer, allowing for an open set of tools that could make use of the hypermedia facilities. Abstraction of the storage layer resulted in an open set of hyperbases (HB) that provided the link service (LS) with domain specific structural abstractions. This architecture represents typical OHSs. The next step was to open up the link service layer, resulting in structure servers (SS) each providing different domain specific structural abstractions. These are tailored generic structural abstractions, provided by hyperbases. CB-OHSs base their architecture on this type. The last step has been suggested to be the integration of basic structural abstractions into the operating system (OS) thereby making it structure aware, resulting in a hypermedia operating system (HOS).

3.1.2 Systems

The following systems all assert themselves to be open hypermedia system to some degree. We will take a closer look at these systems by looking at their architecture; their structural abstractions (model); some characterising or additional features; and, the evolutionary level to which they correspond to. The systems are presented in no particular order.

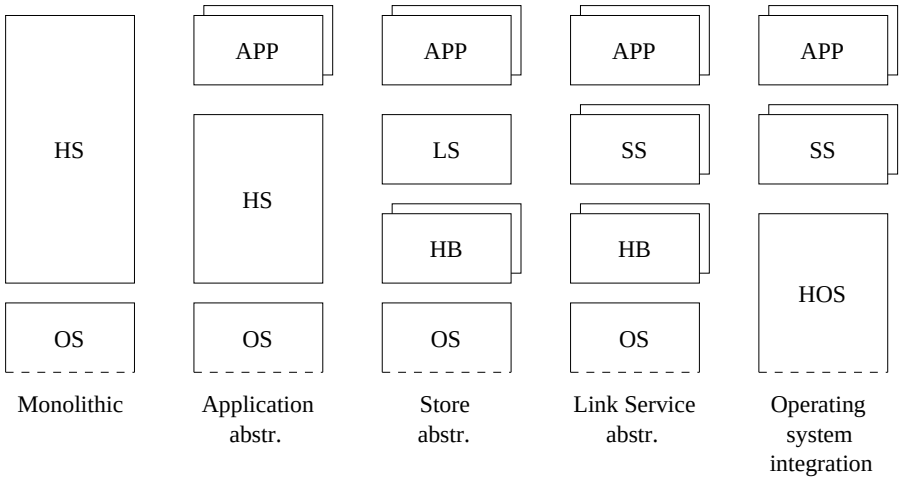

Figure 3.3: *The Evolution of Open Hypermedia Architecture.*

Chimera

Chimera [ATW94] has been developed at the University of California at Irvine. It is designed to capture and visualise relationships in software development environments (e.g., relationships between code, flow diagrams, test information, etc.).

ARCHITECTURE. Chimera is called an open hypermedia system, because it allows an open set of viewers (applications) to communicate with the Chimera server (link server) and it maintains the orthogonality with respect to structure and data [DHH+92, WL97]. Because Chimera's application program interface uses remote procedure call mechanisms (represented by stubs), it is language dependent, but the designers have supported various programming languages.

The focus of Chimera is on the link service that handles external structures (called hyperwebs) consisting of objects, views, anchors and links. These structures are stored in a separate persistent store managed by the Chimera server. The store is not structure aware [Nür97], thereby resembling some kind of "naive" hypermedia store, unable to offer structure related services. All data referred to by the structure is stored in external application specific stores. Chimera has no control over this data.

MODEL. Links are first class structures, and can thereby be anchored. This allows links to links, which in turn allows creation of complex structures. By links being first class structures, it is possible to visualise them through viewers as well.

ADDITIONAL FEATURES. Since Chimera maintains information about its connected clients (users), it is able to filter the served structures, thereby allowing different views on the same structure, depending on the filtering level stated by the clients.

EVOLUTION. Based on all these observations, the Chimera hypermedia system has to be related to the first thread of research on open hypermedia systems, which mainly focus on the openness of the link service. On the other hand, Anderson et al. [ATW94] address the need for a hyperweb server concentrating on storage management. This would open up for the opportunity to extend the Chimera architecture to allow several Chimera servers to communicate to several hyperweb servers (opening the storage layer). This shows the interest in opening the storage layer, which is described as the next step in the evolution. Additionally, the versioning problem of external data is addressed.

Microcosm

Microcosm [DHH+92, DKH94] was developed at the University of Southampton in the United Kingdom. It is designed to minimise the requirements on applications for integration. Microcosm is an open link service system [WL97], trying to deduce the minimal requirements for third-party application integration.

ARCHITECTURE. The architecture of Microcosm is similar to Chimera in that it consists of a link service managing the external structures superimposed on documents (enforcing orthogonality). This link service consists of three types of filters: the linkbase filter, which communicates with the linkbase; the linker filter, which handles creation of links; and, the computed linker filter that handles generic and local links. To some degree, these filters represent the behaviour of the system [NLW98]. The fact that the link service is extensible with respect to filters allows for some degree of tailorability of behaviour.

This architecture is extended by a universal viewer, which allows integration of applications that cannot be enabled via direct communication to the Microcosm server. The viewer is some kind of wrapper program that takes over the communication with the link service.

MODEL. Microcosm's structural abstractions are based on the associative domain and are therefore presented by abstractions such as anchor and link. Links contain the endpoint information, which makes anchor objects obsolete. Links are not first-class structure, which makes linking to links impossible.

ADDITIONAL FEATURES. Microcosm introduces a new (behavioural) abstraction: the generic link. A generic link specifies only a keyword and a destination. If a document is accessed that contains a word that matches a generic link keyword, a link marker will be generated at run-time referring to the specified destination. A local link is similar to a generic link, but limits the keyword search to a specific document.

EVOLUTION. Microcosm, like Chimera, focuses on the openness of the link service and on the integration of third-party applications. This is why Microcosm is placed in the first thread of research, which is mainly concerned with the openness of the link service.

Davis et al. address a variety of problems and possible solutions [DKH94]. For instance, one way of enforcing link integrity is to compare dates of links

and referred documents. If a link is newer than the documents, its integrity is confirmed.

DeVise Hypermedia

DeVise Hypermedia (DHM) [GHMS94, GT94] has been developed at Aarhus University in Denmark. It is a prototypic implementation of an object-oriented hypermedia framework based on the Dexter Hypertext Reference Model [HS94]. The DHM system is designed to support collaborative work of large engineering projects [GHMS94]. The intention was to implement a system satisfying such demands as a shared database, access from multiple platforms, portability, extensibility and tailorability.

ARCHITECTURE. DHM is one of the few hypermedia systems based on the Dexter hypertext reference model (henceforth simply referred to as "Dexter"). It relies on generic object-oriented classes modelling Dexter concepts such as component, anchor, composite, etc. The architecture is three-layered in agreement with Dexter. There is a storage layer (HBMS), a run-time layer (link server) and a within-component layer (applications, called editors), representing a client/server architecture. Distribution of the storage layer is supported by TCP/IP communication [GHMS94].

As with the previous systems, openness is assured through integration of third-party applications maintaining their own data objects. Structure objects are stored in the HBMS, which also allows data objects to be stored.

MODEL. Tailorability of the link server is realised by specialising (subclassing) the generic classes that model the Dexter concepts. This allows further restriction of these concepts (e.g., binary links can be specified by subclassing the generic n-ary link concept).

ADDITIONAL FEATURES. The HBMS is based on an OODB augmented with event notification and locking facilities. The OODB is *not* structure aware, which in turn allows notification and locking services to be available for every object (both data and structure) in the OODB [GHMS94]. On the other hand, the unawareness of structure restricts the HBMS, since it can not provide *structure* services such as reference integrity.

Cooperativity via shared material is supported through notification and locking mechanisms, provided by the OODB. Group awareness, for instance, can be realised by notifications of user actions.

EVOLUTION. The DeVise Hypermedia research has focused on the Dexter reference model. It has uncovered some problems and provided extensions especially with respect to cooperation (i.e., notification and locking of the storage layer).

DHM is a traditional OHS, but it has also contributed with experiences in the field of collaboration. Hyperbase management, as well as tailorability, have received the most attention, which places it in the second thread of research, focusing on generic services provided by the HBMS (although in this case, the

hyperbase is not structure aware).

DHM makes a dangerous assumption by presenting structure unaware storage as an advantage: "Its [event-notification mechanism] implementation directly within the OODB implies that we can support event notifications for arbitrary objects and classes independent of their declaration, that is, event notifications are not bound to objects inheriting from a special superclass." [GHMS94] It is true that this enables the OODB to support notification and locking on all its objects, but this could be done even if the hyperbase was aware of object differences. The latter would add the possibility of supporting more fine-grained notification and locking, as the next system will show.

HyperDisco

HyperDisco [WL96] has been developed at Aalborg University in Denmark. It is a descendant of the HyperBase [Wii93] and HyperForm [WL92] systems also developed at Aalborg University. The HyperDisco approach is to provide an open hypermedia platform, which is able to integrate distributed heterogeneous tools and data formats, extending the integrated tools to handle collaboration and versioning of multiple workspaces distributed across the Internet [WL97]. Emphasis has been put on the concepts of extensibility and tailorability.

Architecture. The Tool Integrator [WL96] (structure server) is the central component of the architecture. It allows distributed heterogeneous tools (applications) to participate in hypermedia services and control access and operation of multiple HBMSs. In other words, the architecture supports an open set of applications, an open set of structure servers and an open set of HBMSs.

Model. Tailorability is provided by two different layer models that allow specialisation (and/or extension) through object-oriented inheritance, by using a similar mechanism as DHM. There is a "data model" and a "integration model" implementing methods supporting generic services for HyperBase Management Systems (HBMSs) or Tool Integrators respectively. (DHM only allows tailorability of what corresponds to the integration model.) The integration model is used to specify the structural abstractions and services provided by a structure server, thereby specifying the protocol between it and the applications. The data model is used to specify the structural abstractions and services provided by the HBMS, thereby defining a protocol between the structure server and the HBMS. The HBMS provides various services on its stored objects, like notification, versioning and locking. Even these services are tailorable to match the properties of a specific object. This tailorability is realised by specialisation of multiple generic service classes (multiple inheritance). This makes the HBMS structure aware.

Additional features. The approach focuses on the openness of services in general. This is a major step towards the goal of interoperability of systems, because it stresses the orthogonality issue of all kinds of services, not only hypermedia specific services. The implementation of HyperDisco shows that it is practical as well.

EVOLUTION. The evolutionary level is clearly that of a CB-OHS [NWL98], since the framework allows HBMSs to communicate with an open set of tool integrators (structure servers) that, in turn, communicate through specialised protocols to applications.

Despite the high degree of tailorability, there are still some limitations on the model. The integration model, for instance, is based on the concepts of anchor and link, thereby resembling navigational abstractions which are not convenient abstractions in other domains. This reveals the intention of HyperDisco to support mainly the navigational domain.

Construct

As mentioned earlier, the OHSWG has recently suggested a general component-based open hypermedia framework. A public domain implementation of this framework, called Construct, that complies with all OHSWG standards, reference models and reference architectures [WN98] is being implemented. Construct [WN98] is a merge of HyperDisco [WL96, WL97], HOSS [NLSS96, Nür97] and DHM [GHMS94, GT94].

ARCHITECTURE. Because Construct is an implementation of the CB-OHS framework, it has all the characteristics mentioned earlier. The architecture is three-layered consisting of an open set of hyperstores, an open set of structure servers and an open set of applications. This makes the system extremely flexible.

The hyperstores provides persistent storage services, such as notification, access, and version control. The elements stored are generic structure units, allowing the hyperstores to be structure aware.

Hypermedia domains are represented by structure servers that mediate interaction between the applications and the hyperstores. This mediation includes specifying the generic abstractions (both structural and functional) of the hyperstore to domain specific abstractions.

The application layer is represented by either customised or "enabled" third-party applications. These applications communicate with an open set of structure servers.

MODEL. The basic building block of the model is the generic structure unit stored in the hyperbase. It is this unit which is tailored to domain specific structural abstractions, appropriate for specific hypermedia services, by the structure servers. This requires the structure unit to be capable of expressing all different kinds of abstractions, which in turn, allows use of structure across domains.

ADDITIONAL FEATURES. The main advantage of Construct is its extensibility and tailorability. This allows, on the one hand, new domains to be integrated into the system while on the other hand allowing specialisation of structural abstractions that fit the new domain. The integration of several domains in one system opens up new possibilities such as cross-domain linking, cross-domain collaboration and reuse of existing structure in other domains. In other words, it supports the idea of interoperability.

Evolution. Construct is under development right now, which is why experiences with the framework are still lacking. The idea is to get "hands on" experiences with a framework that builds on generic structure elements as atomic units allowing interoperability between various components, cooperativity support, and other things still to come.

The system is clearly a CB-OHS, following the reference architectural closely. Compliance of OHSWG standards is another main issue of the framework, contributing to the interoperability of hypermedia domains. Experiences with systems like Construct are still scarce, but will be invaluable for the next step of the evolution of hypermedia systems: a hypermedia operating system.

HOSS

HOSS is a hypermedia operating system simulation prototype [NLSS96, Nür97] that has been developed at the Texas A&M University. It is a descendant of the SP3/HB3 system [LS94], also developed at the Texas A&M University. HOSS implements a general hypermedia operating system architecture designed to demonstrate the convenience and efficiency of a structure aware operating system, thereby building on the philosophy of hypermedia-in-the-large [LS94].

Architecture. The architecture of HOSS is, roughly speaking, similar to that of a CB-OHS, but implies more requirements on the operating system to be able to simulate a hypermedia operating system. For this reason, there are provided some toolkits (HCMT[1] and HPMT[2]) supporting appropriate communication and process models [NLSS96]. In addition, the architecture presents a HBproc, which simulates a kind of HBMS integrated into the operating system. It is what maintains all stored resources. As mentioned before, structure servers can be abstracted further into Sprocs and behaviours; this is what HOSS does. Behaviours are implemented as light-weight processes (threads) inside Sprocs. Finally, the application layer, which communicates with the Sprocs, provides different services on domain specific structure, through behaviour.

Model. HOSS's three basic entities are data, structure and behaviour. The relation between them is as follows: data and structure are associated by structure (this reveals the first-class membership of structure), while behaviour defines the semantics of structure. This is similar to the philosophy of CB-OHSs. The special thing about this system is that data as well as structural abstractions are part of the operating system. This leaves only the semantic part to be defined by the user, while OHSs still need to specify the structural abstractions

Additional features. The awareness of structure in the operating system alleviates several problems. As mentioned in the terminology section, issues like semantic locality, integrity and consistency of data and structure are now in reach, either because of the centralised provision of both data and structure or through the awareness of structure in the operating systems

[1] HOSS Communications Model Toolkit
[2] HOSS Process Model Toolkit

EVOLUTION. Evolutionary, HOSS is obviously in the hypermedia operating system category. It is the first attempt to go beyond the typical assumptions of given operating systems and tries to motivate for a new operating system paradigm.

Although HOSS shows that it is possible to simulate a hypermedia operating system (to some extent), we still need more experiences on what atomic building blocks are essential, how structure services make use of these, how to process structure from generic to tailored abstractions and vice versa, what kind of services a hyperbase management system should provide, how to handle concurrent access of structure services, etc.

A lot of these missing experiences can be obtained through work with the CB-OHS architecture because of its similar philosophy of structure domains and the modularity in general (components).

3.1.3 Summary

In this section we have introduced terminology and a survey of several systems concerned with aspects of openness. We have tried to categorise them by placing them in a historical context. Various architectures have been described, of which the component-based open hypermedia framework represents the state-of-the-art concerning openness. This new framework will hopefully help us to solve or at least alleviate known problems. On the other hand, it will certainly also present new challenges that have to be addressed in the future.

Open issues

We outline some of the addressed issues concerned with component-based open hypermedia [NLW98]:

- *Interoperability:* Interoperability has become a central issue of hypermedia. Even though it is currently a subject of research, it is a complicated issue that should be expected to remain difficult for the foreseeable future.

- *Structure processing:* Structure processing as a subject has been known for a long time, but it is getting more and more attention, because of its close connection to interoperability. Specifically, structure processing of generic structure units that are able to express all kinds of structural semantics is a major topic.

- *Human computer interaction:* Human computer interaction issues have always been an active part of the hypermedia field. New domains are only one aspect of HCI, that has affected hypermedia systems and will continue to do so. With the potential of interoperability new aspects like presentation of multiple structure models simultaneously appear.

- *Standardisation:* Standardisation has presented an effective method for "global" availability of services, but it seems like it has taken a considerable time for the OHS community to realise the need for it. It is only recently

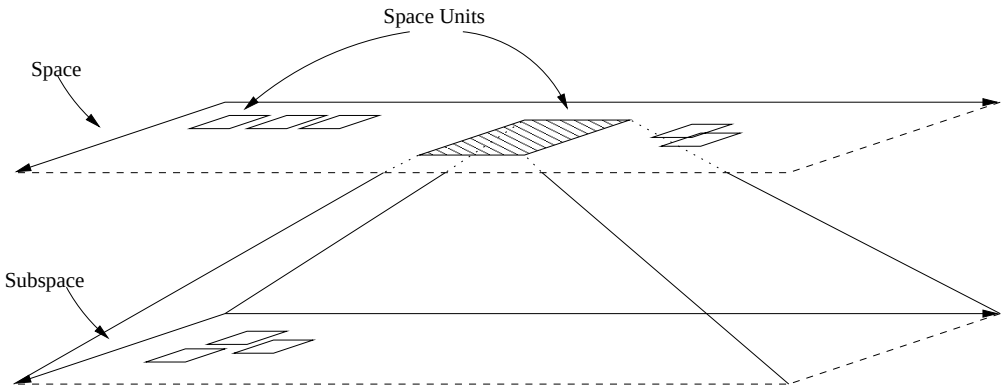

Figure 3.4: *A space and a subspace.*

that attempts for standardisation have been made (OHSWG), again to promote the interoperability of system.

- *Application integration:* Application integration has been an issue for a long time, but now interoperability allows multiple structure models, stressing new issues such as what integration levels are shared across structure domains? Is it possible to define a general integration framework for all domains (integration reuse)?

- *Backend functionality:* Backend functionality is concerned with how much and what kind of functionality can be put into the hyperbase management system, thereby offloading the structure servers of generic functionality.

3.2 Spatial hypermedia

This section will survey the literature on spatial hypermedia. Concepts used and apparent advantages of spatial hypermedia will be addressed. Firstly, a terminology that introduces the concepts used in the area of spatial hypermedia will be presented. This will serve as a common basis for further treatment and discussion of spatial hypermedia. Secondly, a characterisation of spatial hypermedia will argue that spatial hypermedia possesses the ability to support working informally with emerging structure. Thirdly, this section will present a chronologically ordered review of hypermedia systems that over time have contributed with spatial abstractions to form spatial hypermedia as an area of its own belonging to the domain of information analysis. Methods for spatial parsing will also be treated in this section.

3.2.1 Terminology

This terminology will be used to describe the spatial hypermedia domain, by describing its constituents. The defined terms are an accumulation of concepts introduced in various articles and systems.

content unit A content unit holds the content (e.g., text) of a space unit, or a reference to its content.

space unit A space unit has a location in a space; a size; and, visual properties that specify its appearance. It refers to a content unit, and represents that information in a space. Space units can be structured by means of their visual properties, placement and proximity to each other.

virtual copy A space unit that shares its content unit with another space unit is called a virtual copy.

space A space is an infinite spatial area in n dimensions, which can contain space units.

subspace A subspace is hierarchically related to another space by containment (see Figure 3.4). Subspaces can be used to model hierarchic structure.

visual structure The hierarchical structure of space units, that constitutes the bare organisation of space units made by the users. There is no notion of structure interpretation in visual structure.

space structure A space structure is a (hierarchic) structure that relates spaces according to subspace containment.

implicit structure An implicit structure is a visual pattern of space units that is expressed informally by means of proximity, alignment, repeated patterns and visual properties of space units.

explicit structure An explicit structure is an explicit representation of the implicit structure.

structure indicator A structure indicator visualises explicit structure, thereby providing feedback to users.

spatial parser A spatial parser produces an explicit structure by interpreting the implicit structure in a space. A definition of implicit structure embodied in the parser defines the interpretation.

spatial relations The following spatial structure types represent typical ways people have been found to use for expressing spatial relationships (see Figures 3.5 and 3.6).

- *Relationship by spatial arrangement*: This kind of relationship is expressed by proximity between objects. It can take several forms. For example, horizontal lists, vertical lists, matrices, and, stacks, each of which requires a certain alignment and visual uniformity of objects to express ordered groupings. Piles require objects to be partly overlapping, and can express unordered groupings.

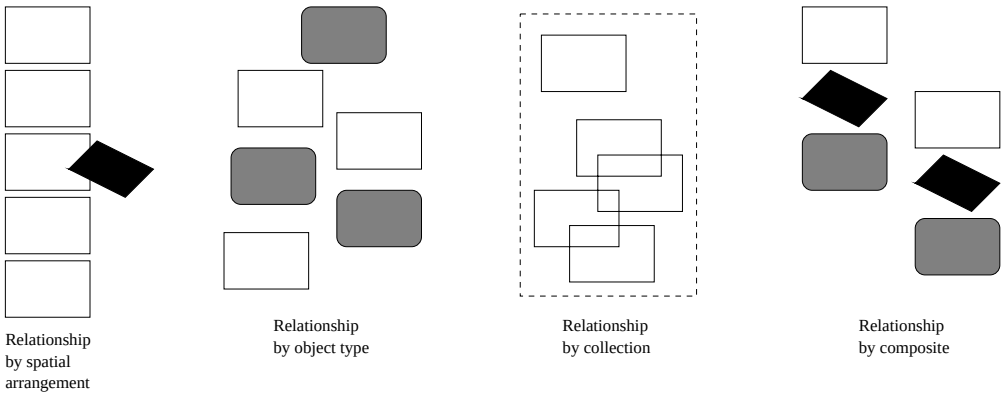

Figure 3.5: *Spatial relationships.*

- *Relationship by object type*: This kind of relationship is expressed by visual similarities between objects. The objects are perhaps dispersed, but similar looking objects will be perceived to have some relation. In this way, object type relationships can express category membership that cross-cuts spatial positioning. Visual similarities can be expressed by many primitives (e.g., colour, font, extent, shape, border width, etc.).
- *Relationship by collection*: This kind of relationship is imposed on a set of objects to categorise them according to some criteria. A collection can reside in a separate space, perhaps a subspace referring to or containing the set of objects that are related by the membership of that collection. Subspaces can express hierarchical ordering of collections.
- *Relationship by composite*: Composites provide users with the ability to express relationships between different types of objects. Composites can reveal themselves as recurring patterns of objects, where the immediate constituents of the composites can be of arbitrary type, form and extent.
- *Relationship by activity-related areas*: A space is said to be organised into activity-related areas when objects are clustered (by spatial proximity) in such a way that objects in the same cluster relate to the same task. Only the clear spatial separation of objects into clusters is significant, not structure within the clusters.

spatial hypermedia Spatial hypermedia systems provide visual means to structure space units residing in a space implicitly. Spatial structure can be implicit (i.e., by means of proximity, alignment, repeated patterns and visual similarities of space units), or explicit (i.e., by structure indicators resulting from spatial parsing of space units).

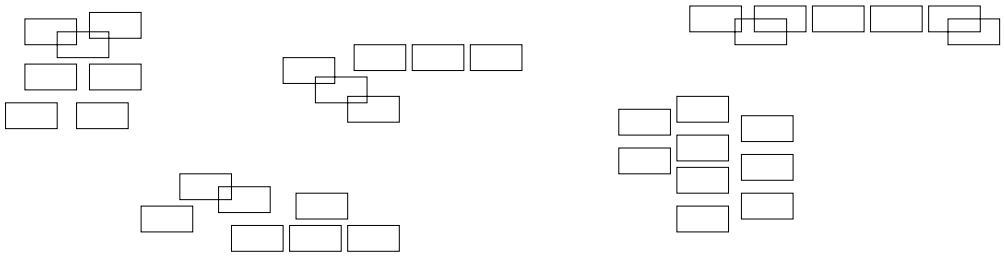

Figure 3.6: *Relationship by activity-related areas.*

3.2.2 Characterisation

Given that it is possible in spatial hypermedia to express visual structures through placement, alignment, visual properties and containment of objects, it is possible to characterise spatial hypermedia by the following properties. These properties also represents the "state of the art" in spatial hypermedia, and highlights what should be expected to be supported from a general spatial hypermedia system today.

Structure creation

In spatial hypermedia, focus is on active creation of structure rather than reading and traversing structure [MSC94]. Although spatial tools can be used to browse spatial structure, users are encouraged to grab objects and move them around, to understand and express connections among objects.

Emerging structure

Spatial hypermedia allows structure creation even when users only have a fuzzy idea about what kind of structure to create, thereby creating unclear structures that can be ambiguous. Spatial hypermedia allows the user to create collections before naming and typing them; thereby explorative development of informal emerging structure is supported. This can be seen as a kind of stepwise refinement of structure, without changing the integrity of the structure [MS95], which is a valuable alternative to creation of explicit structure at once. In [MS97] it is mentioned that people's organisational strategies change over the duration of a task as they become more familiar with the material to be organised. This stresses the need for emerging structure in tasks involving structuring.

Flexibility

Spatial hypermedia allows people to record various interpretations of bodies of material, by letting them arrange material in multiple ways. The ability to show structure on the fly as objects are moved around means that modification of structure is immediate, because structure results from placement and proximity of objects. To develop a personal understanding of a collection of information, one sorts and marks up material with annotations, underlining,

etc. This stresses the importance of the ability to express meta communication about material. Along with spatial properties, flexibility should also be supported by the ability to change colour, border, shape, font, etc., on objects. For instance, one could express more relevance of an object by thicker borders, and less relevance by thinner borders. Under time constrained work with huge amounts of data, attributes as colour, shape, etc., help people quickly to express relationships between objects [MS97]. The flexibility of spatiality makes it easy to express relationships such as similar-to or different-from [MS95].

Common understanding

Spatial hypermedia allows people in a group setting to discuss appropriate structures, by moving objects around in a space, suggesting new structures. In this way a common understanding of structure can be developed, over time. The materials are themselves of limited utility without this shared understanding. It is the perception of the structure spatial hypermedia wishes to address. The difficulty lies in expressing and recording mutually-defined, ever evolving interpretive structure. One's personal reflections, understanding and interpretation of a collection of information can be communicated to others through spatial structure [MS97].

Handling large structures

A problematic aspect of handling a large amount of data is that structure grows large. The chance that users lose track of their position and context while they navigate or structure it is present. Spatial hypermedia allows people to structure large amounts of data in a comprehensive way.

Some methods for showing an overview of structure have been shown to be inappropriate when data grows large. Networks get confusing when there are many nodes or links, because the graphical representation of nodes and links blur the picture [Fei88].

What we want is to keep an overview of structure while dealing with it. Losing context when navigating structure is a serious problem today, since there are millions of resources available on the net, and browsing has become a daily routine for laypeople. This was already mentioned back in 1991 as the navigation problem [BBF+91].

Spatial hypermedia allows a representation in which recognition of spatial placements of objects is central. In spatial hypermedia, one can inspect structure by navigating up or down through spaces referring to each other. Subspaces allow the user to immerse himself in local meaning without losing global context [MS95]. The perception of which way one directs attention becomes more clear when only two directions are at hand. Either one enters a subspace, or one moves up to the parent space. This way of navigating has parallels with the use of Walden's paths [SFB+98], in which only two directions are present, back and forth. Walden's paths have shown useful in settings where information is picked from many contexts, and people have to keep track of where they are in the process of browsing the structure [SFB+98].

The use of structure in which nodes have one parent and many children has the characteristics of a hierarchy. The space subspace relationship is actually the model of a taxonomy. Specialisation is represented by subspacing and generalisation is represented by the containing space. Spatial hypermedia can be seen as a cousin to taxonomic hypermedia because the primary structure is a taxonomy. The approach of dividing material into nested subspaces also includes aspects of taxonomic reasoning which was described by Parunak [Par91].

Spatial parsing

Spatial parsers interpret visually expressed structure and create explicit structure, that can be used to provide feedback to the users about what the system has understood as some kind of structure. When multiple users work together on the same space, it can be very useful if the system can suggest what should be understood as structure and what should not. In this way, the system provides means for common understanding of the structure. This is a way to make the dialog uniform between users of the system, as spatial hypermedia layout in some cases implicitly can be interpreted differently among users.

For a hypermedia system to perceive structure expressed spatially, we need a method of understanding spatial relationships. The method should tolerate people's different ways of expressing spatial structure, and be able to interpret it. The method must not over-interpret spatial structure; it should only extract intended structure. This is a difficult task, because ambiguity and lack of clarity is inherently a part of spatial hypermedia. Borderline cases, in which whether something is structure or not is a subjective matter of personal opinion, are abundant. Therefore, methods typically builds on heuristics for general recognition patterns.

Literature on practical spatial parsing is relatively sparse. Research on visual parsing of visual languages deals with spatial layout of objects. Often, though, these parsing strategies are rule based, which do not suit our task of extracting emerging structure from arbitrary positioned objects. Rule based parsing intends to recover structure of visual languages that are constructed based on a given grammar, which usually embody little ambiguity, and do not differ among different users [IMM95]. Actual implementations of spatial parsers are presented in VIKI [MSC94], Web Squirrel [ES95] and a prototype of an adaptive recognition algorithm [IMM95]. The methods will be described in the presentation of the different spatial hypermedia systems in the next section.

3.2.3 Systems

This section presents a chronologically ordered survey of hypermedia systems that have contributed with spatial abstractions to the area of spatial hypermedia. The review will not make a comparative analysis of systems. Rather, it will address various spatial concepts to systems, and describe how some spatial concepts emerged and were realized through use of the systems.

NoteCards

NoteCards [HMT87] a hypermedia system from the 1980's, aimed at easing the task of idea processing. Idea processing is divided up in three phases: acquisition, analysis and exposition. Idea processing progress through manipulation of tangible objects, that represent or embody ideas and their interconnections.

Basic concepts. NoteCards has four basic objects. A *notecard* is an editable object that can contain substance, in the form of text, graphics, etc. *Links* are typed and directed relations between two notecards. The type of a link is simply expressed through an annotated textual label. A *browser* is a special notecard that contains a structural diagram of a network of notecards. It is possible to edit structure in a browser, as well as deleting and adding objects to the displayed structure. *Fileboxes* are special notecards that are used to organise large collections of notecards. Fileboxes can be hierarchically nested, and can represent an alternative way to structure notecards. Fileboxes can be filed, making structure persistent. Notecards is quite flexible with respect to expressiveness, since graphical appearance and behaviour can be modified.

Using NoteCards. Navigating structure in NoteCards can be done in either of two ways. It is possible to follow links from card to card, at each card deciding which link to follow. It is also possible to create an overview browser for some subnetwork, and then traverse the links from the browser to the referenced cards.

The rich expressiveness in NoteCards revealed an unexpected result, as users started to use NoteCards in an unintended way. Notecards provides an object-oriented, spatial and direct manipulable navigational interface, not only to information, but also to structure. This way of manipulating tangible objects encouraged users to an alternative use of NoteCards. Users found it difficult to plan and specify an appropriate structure with which to start. Therefore, they started to use objects spatially, by placing them in proximity to each other, thereby expressing implicit structure. NoteCards was not prepared for this, so explicit structure could not be recorded by the system [MS93].

Three issues. In [Hal88], experiences with NoteCards led to suggestion of 7 issues that later became a milestone in hypermedia. Three issues are of special interest when we deal with spatial hypermedia:

Composites—augmenting the basic node and link model. . NoteCards does not have a proper composition mechanism for dealing with groups of nodes and links as unique entities. The filebox is a way to group nodes, but it fails to take into account the difference between *reference* relations (ordinary links) and *inclusion* relations (part/whole relationships). Halasz argues that *composites* should be added as a primitive to basic hypermedia models, and the *inclusion* relationship of composites should be implemented within, and not on top of, all hypermedia systems.

Virtual structures for dealing with changing information. . Halasz mentions that NoteCards suffers from the problem of premature organisation. He suggests virtual structures as the solution to this. Structure should

be defined intentionally. That is, structure should be dynamically determined from a description. This would make the structure dynamic with respect to the structure description. Modifications would be easier because it is more tedious to change an extensionally defined structure than an intentional description of the structure. Spatial hypermedia is later suggested as a promising approach to solving the problem of premature structure organisation. Spatial hypermedia is not intentional though. Instead, it is characterised as extensional, along with being implicit [MS93].

COMPUTATION IN (OVER) HYPERTEXT NETWORKS. . NoteCards was frequently augmented with extra computational engines for particular applications. These extensions to the system made the handling of NoteCards structure (navigation or editing) depend on the added computations. Halasz suggests that systems should be capable of incorporating components specifying computations over structure, thus separating structure from behaviour (also addressed in [KL91, NLS97]). In spatial hypermedia, structure can be expressed implicitly. What is needed for a spatial hypermedia system to make the structure explicit, is a spatial parser, which actually is a behaviour that makes computations over structure.

Boxer

Boxer [dA86] is a programming environment developed in the 1980's at MIT and Berkeley. It is a research tool for educational studies, with the intention of exploring what a programming language intended for laypeople might be like.

THE BOX. Boxer introduces the notion of a box as a structured notion of containment. Displayed boxes have computational semantics and package local data and procedures. Computations can be done by sending messages to boxed procedures.

STRUCTURE. Boxer makes use of a spatial metaphor by representing computational objects in boxes as regions on the screen that contain text, graphics, or boxes-within-boxes that represent hierarchical structure. The Boxer language makes use of naive realism by showing the computational world in its entirety on the screen. The user sees computational objects directly, not just interfaces to them. Anything that appears on the screen is available for the user to "run" or modify. Boxer has the notion of "porting" which allows linking into hierarchical structure, thus allowing a view on a box at some other place in the system to enable cross-referencing and sharing.

CONTRIBUTION. Boxer contributed with the spatial metaphor of boxes, which themselves may contain boxes, thereby modelling hierarchical structures. This metaphor has later been used in other systems. VIKI has, for instance, the notion of collections, which are visually-nested hierarchical structures that provide a navigational mechanism allowing people to move between focal collections [MS95]. Also the IGD system [Fei88] makes use of roughly the same metaphor, by expressing tree structure through spatial containment. In IGD, pages are nested inside parent pages or chapters in a strict hierarchy.

gIBIS

gIBIS [CB88] is an Issue Based Information System that is designed to facilitate the capture of early design deliberations. The system offers argumentation support through *Issue*, *Position* and *Argument* objects. These objects can be related to each other by links, to form argumentation structures. A browser makes it possible to view the argumentation structures that are spatially presented in a two dimensional space. The system makes use of an overview picture of the complete structure, with an indicator showing where the user is. It is possible to zoom in, thus examining local aspects of discussions. Colours are used to type links visually, as people easily adopt and remember mappings from colour to meaning.

THE SPATIAL ASPECT. When dealing with large structures in gIBIS, the advantage of placing objects and links spatially was revealed. The problem of users getting spatially disoriented was overcome, as users by themselves started to divide their networks into activity related areas that were meaningful in terms of the problem they were working on. This technique had several advantages. Users reported that the effort of coming up with a layout revealed aspects of their problem that were not obvious beforehand. The problem of deciding where to place new issues was eased, and users who had forgotten keyword information on a local discussion, and thus could not find it again by means of a supported textual search, were suddenly able to find it by searching visually through spatial structure, while remembering and recognising visual structures and spatial placements.

Virtual NoteBook System

VNS is a page-oriented hypermedia system that supports display and manipulation of various types of information objects like text, images, video, audio, page links and action links. In [MS93] it is mentioned how users can spatialise text, by dividing text into pieces that can be manipulated moved around and placed spatially. In VNS, spatiality is used to create structure orthogonal to navigational structure. References are still created by means of links, but structures are composed through spatial layout and graphical notation. The example shows that spatial layout can be used to express certain aspects in a hypermedia system that otherwise are based on traditional linking.

Aquanet

Aquanet [MH+91] is a browser-based tool that allows users to represent information graphically in order to explore its structure. Aquanet was released in 1991, and builds partly upon experience with NoteCards [HMT87] and gIBIS [CB88].

BASIC CONCEPTS. To model hypermedia structures in Aquanet, one has to be familiar with the following concepts taken from [Mar92]. A *basic object* is a typed entity that represents types of information; it is the same as an information object. Information is contained in name-value pairs called *slots*.

Every object is an instance of a type. Type definitions are organised into a multiple inheritance hierarchy. This means that objects of a given type include both its own slots and the slots inherited from their super type(s). A *graphical appearance*, is a graphical entity that has extent and placement in Aquanet's information space. In the information space, there can be multiple references to the same basic object. Multiple references are called *virtual copies*; they all share the same graphical appearance. Basic objects can be related by forming *relations* between objects, which are nondirectional links. Relations and graphical appearances are defined by a modifiable *schema*. The schema dictates which types of relations will be available between basic objects when modelling structure. Schemas also specify graphical appearance of objects, concerning colour, border width and type.

Structure through constrained relations. Aquanet was originally designed to support knowledge structuring tasks like analysis and argumentation. Users are supposed to constrain relations, making them into argumentation structures, by specifying in a schema which kinds of relations should be available. An example of an argumentation structure constrained by specific relations is a Toulmin structures [MH+91]. Toulmin structures can be specified by means of Aquanet's schemas. The idea of modifiable schemas is that one can start out by specifying a scheme for the structuring problem at hand, and then engage in the task of knowledge structuring. As experience with the structuring task at hand grows, one may find the structuring model insufficient. One can then further constrain or extend the original knowledge structuring schema to highlight the phenomena of interest better. This way to specify which knowledge structure to support is also seen in COWFISH [WH97].

Structure overview. In the early 1990's, hypermedia systems had a tendency to center interaction on objects, nodes and documents. Examples of systems that do not provide any structural overview are Hypercard and KMS. In these systems, authors are expected to focus on textual or graphical content of nodes, and to move from node to node by navigating through a network [MH+91]. Like gIBIS and NoteCards, Aquanet departed from that tendency and made it possible to center interaction around a network overview of emerging hypermedia structure. This presented spatial advantages such as the ability to express implicit lightweight structures in terms of proximity and alignment. In early phases of the task of structuring information, this has proven useful as a means to avoid the problem of premature organisation [Hal88]. If later, in the task of information structuring, spatial structures tend to be repeated (e.g., certain spatial patters emerge) one can consider making an Aquanet relation to express the nature of the spatial pattern. Spatial proximity and Aquanet relations are by nature similar, as both approaches are able to express nondirectional relations between objects. Thus, one can argue that it should be fairly easy to abstract between them.

Experiences. Though steps are taken in Aquanet to loosen up the tyranny of links, the problem of premature structure still exists. In general users found it difficult to come up with a set of types with which to start. This encouraged

the idea of making it possible to define and handle objects before typing them.

Aquanet was originally build on the assumption that users would be able to classify the nature of problems at hand, by identifying general characteristics in a top down approach. Experiences showed, though, that users tended to start recognising idiosyncratic properties on small subsets of the problem, and wanted to create new instances with little commitment to what type of object the instances would eventually end up being. Practical use of Aquanet was therefore characterised by a type hierarchy that had to be changed often. Users usually ended up shoehorning new sources into the structure based on the first set of sources they came up with [Mar92].

The idea of making an information space in Aquanet, in which users can survey their structure in a general spatial overview, revealed new advantages. Instead of expressing relations between different objects by using Aquanet's relations and types, users conveyed implicit relations through the use of spatial layout [Mar92]. This was actually not too surprising, as it is a common practice to categorise materials by putting it in piles, heaps, lists, etc. People do not want to express complex relations from the start; instead, complex relations grow out of simple relations, as knowledge about the information increases over time. Aquanet's virtual copy mechanism was used (by inventive users) to reference the same objects in different spatial contexts.

VIKI

VIKI [MSC94] builds upon experiences with Aquanet, NoteCards and VNS. It was the first system that unified aspects of spatiality from different systems in one system. VIKI has for some time represented the state of the art concerning spatiality.
Spatial hypermedia in VIKI is guided by three pivotal concepts.

1. Interaction must be informal.
2. Users must have access to a variety of representational modes.
3. Additional structure must be available on demand, either through development of types from examples and prototypes, or through recognition of implicit structure.

Realisation. Informal interaction is realised through the nature of spatial hypermedia. Structure results from proximity of objects, and can exist without being strictly classified, typed and annotated.

Access to various modes of representation is realised by presenting multiple choices for the user to express the look of an object. Colour, border thickness, font, form and types are applicable on bodies of information. Types, though, do not limit the flexibility of the system, as VIKI allows any visual symbols appearance to be manipulated independently of its object type.

Additional structures are realised through the ability to define higher structural abstractions in shape of composites, by placing objects in proximity to each

other. Composites can be found and accepted by the user by letting a spatial parser suggest collections in an information space, based on spatial proximity and types of the objects.

DATA MODEL. VIKI's data model contains three basic kinds of elements: objects; collections; and, composites. Objects contain information, or links pointing on some external information resource and capable of spawning some viewer when clicked. Objects are represented by visual symbols that have various visual characteristics: *bounds*, *shape*, *background colour*, *border colour*, *border thickness*, *text content*, *text font*, *type*. It is also possible to make extra characteristics in form of *field definitions*. Multiple visual symbols (virtual copies) can represent the same object, for the reason that one may wish to express the situation in which the same object has different meaning in different contexts [MS95]. Collections are (sub)spaces containing objects, composites and other collections, thereby representing hierarchical abstraction layers in the model. Composites are higher-level structures that can be used when repeated patterns of structure are needed. Composites can be defined by users. Alternatively the system can be asked to find and suggest composites of objects.

NAVIGATION. Navigation of the spatial structure at hand is realised by zooming in and out on collections, which are acyclic, visually-nested hierarchical structures. Zooming in makes a collection fill out the whole application window, reminding one of diving into a subspace that can contain new structure elements. Zooming out is done by clicking on a bar on top of the window, thereby making the parent collection fill out the whole window. Subspaces with collections thereby specify that part of the spatial structure that models layers and composites of objects specify structure at a certain layer.

SPATIAL PARSING. The result of spatial parsing in VIKI is revealed for the user on demand. This means that users interact with implicit structures. Inter-object abstractions in form of explicit structure are only shown on the users initiative.

VIKI performs a bottom-up hierarchical parse using empirically-determined heuristics, implemented as structure experts. Recognition is based on three attributes of visual symbols; position, extent and object type. In VIKI, the order of structure recognition is as follows: Aggregates based on overlaps are found first. Then sets based on homogeneity and alignment are found. Then composites are located. Heterogeneous structure types like composites assume a combined type, consisting of the types of their immediate constituents. Found structures are re-parsed according to the same rules to find higher level structures [MS93].

VIKI can recognise relationships by spatial arrangement. The relationships can be of the following basic spatial structure types (see Figure 3.7):

- *Horizontal lists*: Objects found to be proximately aligned horizontally.
- *Vertical lists*: Objects found to be proximately aligned vertical.

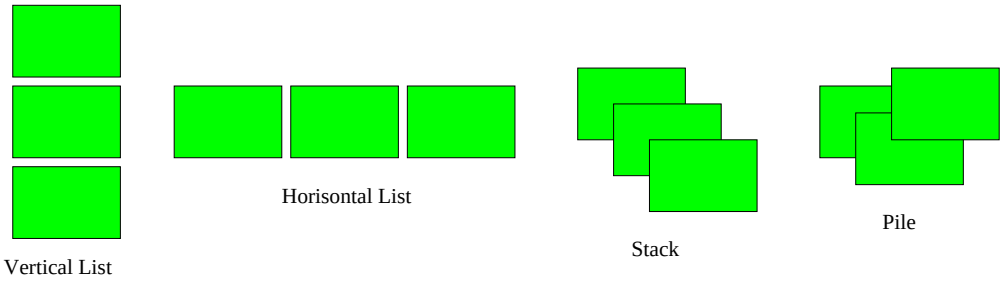

Figure 3.7: *Basic spatial structures in VIKI.*

- *Stacks*: Objects found to overlap and aligned in some direction.
- *Piles*: Objects found to overlap.

Based on the spatial structures, VIKI can suggest:

- *Collections.* Objects found to be in a certain spatial structure, which can then be moved to a subspace.
- *Composites.* Objects that are recurring patterns of clustered objects in the space, possibly consisting of nested structures. Composites can be used as building blocks if accepted.

VIKI has hierarchic click-selection implemented, which has at least two immediate advantages. Firstly, users may select objects that are part of partially hidden structures without having to scroll or traverse from the current view. Secondly users may select partial structures in areas where objects are too densely packed for sweep-selection [MSC94].

Storyspace

Storyspace [Joy91] is a hypermedia system centered around the concept of a writing space, which correspond to a space unit in our terminology. Writing spaces contain tangible objects, which can be picked up, arranged spatially, and rearranged as ideas change or as new information is discovered. Organising, clustering, and revising is natural, because users can move writing spaces around and because links retain connections automatically. In the data model, writing spaces are characterised by a title, some content (e.g., text, pictures and other multimedia material, and a set of embedded writing spaces). Thereby, Storyspace unifies content and a set of embedded writing spaces in one unit.

Relations in Storyspace are modelled as conventional directed links. Linking is done from words in the content of a node to another node. The ability of a writing space to contain other writing spaces makes it possible for spaces to be hierarchically nested. Nested spaces resemble piles, as noted by Rosenberg [Ros96]. Navigating the spaces can be done by entering/leaving writing spaces. Entering enlarges the space to occupy the whole screen, thereby revealing subordinate spaces and perhaps new spatial structures and content. Leaving

makes the space the users was just inside a part of the visible structure in the parent space. It is possible to view the hierarchical structure as a tree, thereby creating an overview of the complete structure, which in part is hidden in the other representation form. Storyspace also gives the user the possibility to make paths through spaces of information, thereby emphasising some links in favour of others. Even paths following links across different hierarchical nested spaces is possible. In Storyspace, implicit structuring can be expressed by spatial arrangement. Explicit structuring can only be achieved through linking, path definitions and hierarchical nested spaces.

A prototype of an adaptive algorithm for spatial parsing

This system is a prototype of a spatial hypermedia system, where the focus is on tests of visual spatial parser algorithm. All objects in this system are similar. Objects are visualised as fixed size boxes of the same colour with no content. This leaves no room for expressing relations except for spatial positioning in form of proximity, alignment and area clustering. Spatial structures found by the parser are visually indicted by graphical frames, that contain the related objects.

SPATIAL PARSING. In [IMM95], an alternative recognition algorithm for finding spatial structures is presented. The algorithm elaborates on the work presented in [MS93, MSC94]. The new parsing strategy recognises implicit structure by calculating the strength of connections between objects. An automatic parameter tuning mechanism is used to make the parser adapt to user's personal preferences with respect to conception of structures. The recognition process is, as in VIKI, started on demand by the user. The parsing process is carried on in three phases:

1. Sort all objects by upper-left x-coordinate.

2. Find the candidate proximity objects for each object by comparing x-coordinates of objects.

3. For each object, check the candidate objects, and create links to the nearest candidate in each of eight surrounding regions in a grid.

This results in a diagram, which can serve as basis for further calculations. A strength value (proximity of nodes) and a list-level value (list likeliness) is calculated by functions for each link. These functions can adapt to users behaviour by automatic adjustment of parameters. The strength values are now recalculated through repression and reinforcement, to check the regularity of the links in the diagram. The most stable links are finally selected, and objects connected by these links, are grouped together in separate structures. Finally, it is determined whether the separate structures can be typed as vertical lists, horizontal lists or just clusters. Like in VIKI, click selection can be used to navigate and examine hierarchies of explicit structure.

A comparison reveals that the spatial parser algorithm in VIKI [MS93] has more expressive types of structure than the adaptive algorithm does. For instance stacks and composites are part of VIKI's recognisable structures. The adoptive algorithm, though, targets more ambiguous layouts, as it is tuned to the preferences of the user.

Web Squirrel

Web Squirrel [ES95] is an information organising tool with the intension of making it easy to keep track of large numbers of URL's. Items are objects that describe a place on the net. They can be created by dragging or copy-pasting URL's onto Web Squirrel's canvas. Double-clicking on the items makes an associated web browser follow the links described in the items. In items, links to resources like web sites, newsgroups, ftp servers, gopher servers, email addresses and file names can be specified.

Web Squirrel can create spaces with items by entering a URL of a web site. The links on that site will then automatically be made into items on the canvas. Bookmarks in browsers can likewise be made into items automatically. It is also possible to make items by hand.

Web Squirrel offers the possibility to organise items into neighbourhoods, which are groups of items clustered around a label. Lists are piles of items stacked together, and have to be made manually. Agents are items that can be set up to scan the space constantly for items that meet some requirements; they are a kind of continuous search facility. When clicked on, agents can reveal labels for items that currently meet some given requirements (e.g., containment of certain words or characters).

Navigation in Web Squirrel can be done either by using the scroll bars, or by grabbing the background of the space, and moving it around. It is also possible to pick a neighbourhood from a menu, and then fly to the chosen neighbourhood in the space. While flying past, the intervening territories is shown. This helps the user keep in touch with the spatial relationships in the space. It is possible to zoom in and out on the space details, thereby making it possible to get an overview of the whole space, and also to dive into details of local areas.

SPATIAL PARSING. Neighbourhoods in Web Squirrel are explicit spatial structures based on spatial proximity of the constituents. A visual indicator in the shape of a frame is positioned around a neighbourhood. Neighbourhoods can be nested inside each other. The size of the neighbourhood is decided by a spatial parser in Web Squirrel. The neighbourhood is the only explicit structure abstraction available in Web Squirrel. Other types of structures can only be expressed implicit [ES95]. The parsing strategies are adjustable through parameters. Structure found by spatial neighbourhood parsing can be exported as explicit structures in form of HTML.

Hot Sauce

Hot Sauce [HS97] is a graphical browser that presents labelled nodes in multiple colours in a three dimensional space. Nodes can link to external resources, such

as web pages, gopher files, ftp files, desktop files, email and database structure. A node can also hide/reveal other nodes to which it links to; this can be used to express hierarchical relationships (e.g., a tutorial presented an information space modelling the class hierarchy for the java-class library). Hot Sauce presents content in a view where objects are floating freely in space. It is possible to use the mouse to fly through the space in 3D, quickly navigating between branching structures. The effect is like flying a spaceship towards elements that reveals more and more detail as one get close to them, but suppress details as one navigates away.

The intention of Hot Sauce is to provide an adequate framework for people to be able to remember placement and structure of a collection of resources. The alternative presented in Hot Sauce is different from the previously mentioned systems. The spatial three dimensional effect has the advantages of expressing structure in a world that we as humans are used to navigate, and orientation is therefore easier to maintain during navigation.

Self

Self [SU91] is an object oriented programming language that presents its objects in a two dimensional space. The Self environment is very different from conventional programming environments. The Self interpreter runs while users program; any changes applied to code pieces apply immediately both visually and functionally. This, along with the ability to inspect and change objects while one's application is running, makes programming a more dynamic and interactive task. Furthermore, it helps debugging code.

Self is intended to make users feel that they manipulate real objects. Both the visual appearance of an object and the code block belonging to it are presented as manipulable items in the space. Dragging, replacing, resizing, copying, etc., is immediate, which means that the "object in hand" feeling is enhanced. This makes the Self system an excellent tool for making prototypes of system applications, as users "just go out and play" with objects.

The ability to place items spatially in Self is useful when many objects are on the canvas. Typically, one selects an area of the canvas, in which to work with components of a prototype. In this way, the canvas is partitioned up into activity related areas. This was also noted as a general behaviour of users in Aquanet [Mar92]. In Self, it is possible to make clickable buttons that can teleport you to another area in the space, thus navigation of activity related areas is supported. To keep a general overview in the Self world, there is a window with an overview picture of the space. Like in gIBIS, a graphical indicator box tells the users where they are currently in the space.

Inside regions, it has been shown to be useful to be able to place objects containing code spatially in proximity to each other when navigating code pieces. In [Mar92], Smalltalk is mentioned as an example where spatial arrangement is useful in programming contexts. Spatial arrangement can express structural relationships between code pieces. Also, it is useful to have the visual object created by some code in spatial proximity to the code, as a programmer typically wants to express that a given code belongs to a given object. The programmer

also wants to see the changes made to objects immediately [SMU95].

3.2.4 Summary

In this section, we have introduced concepts that characterise spatial hypermedia and aspects of these concepts. We have classified different ways to express spatial structure, and mentioned advantages of using spatial hypermedia.

The above mentioned systems each made contributions to the area of spatial hypermedia. Spatial hypermedia is today a broadly accepted area of the information analysis hypermedia domain.

Future systems will perhaps reveal new aspect of spatial hypermedia, and new contexts in which it can be used. As seen in recent systems, current trends are heading towards experiments with spatial hypermedia in three dimensions, and spatial characteristics to pervade alternative areas of computing (e.g., programming languages).

This section described characteristics of the area of spatial hypermedia; aspects of the area will be described in the following chapter.

3.3 Computer supported cooperative work

In [EGR91], groupware, of which collaborative systems are a subset, is defined as follows:

> computer based systems that support groups of people engaged in a common task (or goal) and that provide an interface to a shared environment.[3]

The term *Computer Supported Collaborative Work* (CSCW) is used for technology that provides a computerised environment enabling people to work together despite being geographically dispersed and/or working at different times.

Hypermedia systems have attempted to support collaboration among users ever since seminal systems like NLS [EWN73], which supported sharing and collaborative authoring of nodes their content. Today, systems are deployed in a distributed setting, supporting data sharing among users. Adding support for multi-user collaborative settings is a natural next step. This is reflected by extensive discussions in the literature on supporting collaboration in contemporary hypermedia systems [WN98, SK+96, HW98, WL93, SH+92, SGHH94]

The focus of this section is to present a survey of examples in the literature of combining hypermedia and CSCW functionality. Some CSCW-specific articles have also been studied, and are referenced, due to certain field-specific concepts discussed in them.

This section will begin by introducing terminology relevant to CSCW-enabled hypermedia systems. Subsequently, we use that terminology to describe and compare a number of such systems that are relevant to our work, and characterise the way in which CSCW functionality is implemented.

[3] To illustrate this definition, note that it does not include: 1) multi-user time-sharing systems, because there is no common task or goal; 2) e-mail, because there is no shared environment between users.

3.3.1 Terminology

Basic terms

event An event is a state change occurrence in an entity that is of potential interest to parties outside the entity.

message A logical unit of information sent between two entities. Messages may be characterised according to their purpose (e.g., requests, responses, notifications, etc.)

notification A message that reports an event in some entity to interested parties outside the entity.

request A message that asks the destination entity to perform a computation and/or to return a response.

response A message that contains the information asked for in a previous request.

event subscription Clients can subscribe to notifications about selected events from an entity, thereby stating their interest in particular state changes in that entity.

session A session specifies and structures the interactions between a group of collaborating entities working together on some shared data.

synchronous work Synchronous work specifies a group of users engaged in a cooperative work session, working concurrently on shared resources.

asynchronous work Asynchronous work specifies a group of users working cooperatively on the same resources, but not concurrently.

group focus The properties of and mechanisms in a shared user interface that permit a user to be aware of the work of others, in such a way that focus remains on the task at hand.

group awareness Some indication mechanism that makes a user aware of the presence of other users with whom data are shared.

group dynamics An indication mechanism that makes a user aware of changes to the group the user is coupled to.

coupling mode A coupling mode is defined in terms of a minimum set of event types to which all coupled entities must subscribe. It specifies the minimum coupling that is required between a set of entities such that some desired functionality can be effectuated.

loose coupling mode A coupling mode intended for maintaining a consistent view of shared data between the coupled entities.

tight coupling mode A coupling mode intended for maintaining a consistent view of shared data through a shared user interface that provides a high level of feedback of the actions of other coupled entities. It is usually a superset of the events specified by an accompanying loose coupling mode.

locking Locking is a mechanism for preventing access conflicts to objects; the owner of a lock on an object is granted certain privileged access rights to that object.

floor control Floor control specifies a turn-taking protocol in a shared user interface for concurrency control of updates to shared objects.

Application structure

The general architectural structure of CSCW-enabled systems often fall into the following categories [CM+90], or hybrids thereof:

centralised Applications provide a view into some common state provided by a common application server. Applications send user requests for modifications to the server, and the server sends notifications to the applications whenever the common state changes. All concurrency control over the common state is managed by the server in this model.

An example of a system belonging to this group is a shared window system.

replicated A system consisting of replicated applications does not have a central coordinating component. The system is considered to be a single multi-user application, with one replicated application instance per user. Each instance maintains its own copy of the shared state. The input to the application is the union of input to all instances, with some suitable ordering applied. All instances distribute their own input to all other instances; this enables each instance to build a copy of the input to the application. Shared data consistency is maintained in all instances because each instance applies the same operations on the shared data based on the collective input.

The major disadvantage of this model is the effort it takes to maintain an identical state across all instances of the application [CM+90].

Session management

SESSION. The concept of a session is used for specifying and structuring the interactions between a group of users and/or computational entities working together on some shared data. Sessions are usually defined as follows [HW98]:

1. a group of users/clients;
2. the shared object(s) worked on; and,
3. a specification of the mode of cooperativity.

SESSION MANAGEMENT. The term session management refers to the actions of starting, stopping, joining, leaving, and browsing sessions.

Where to handle session management in the overall system architecture is an open issue—the most common points of view in the literature are:

- *Application-centric:* A CSCW-enabled application is seen as a multi-user application, which is like a traditional single-user application, but with more input and output channels to control and coordinate user actions [PH+90]. Session management is built into applications to manage the interactions between simultaneous users [PH+90, WL93, HW98].

- *Environment-centric:* The ability to perform session management is an inherent property of the environment in which applications run, and is performed automatically to a certain degree, depending on the shared objects in use [Edw94].

The problems of how new sessions are initiated and how an existing session is joined (called *rendezvous*) are non-trivial, and depend on the architecture of session handling, as described above. A taxonomy of these aspects of session management can be given as follows [Edw94]:

explicit This form of (so-called *heavy-weight*) session management requires users to handle session creation and rendezvous themselves. This category can be subdivided as follows:

Initiator-based: Some user sets up a session and explicitly invites other users to join it. This method servers the goals of notifying users of the existence of the session, as well as providing a means for rendezvous with it.

Joiner-based: An initiating user sets up a session. Other participants perform rendezvous by browsing the available sessions and explicitly choosing the one to join. This method handles the rendezvous problem, but requires some other means to notify users of the existence of the session.

Collaboration systems using explicit session handling tend to be *application-centered*—that is, session management is implemented at a (low) level in the application itself.

implicit This form of (so-called *light-weight*) session management [Edw94] is based on applications volunteering information to an external session manager about *activities* of users—who they are, what application they are using, and what objects they are working on. When the session manager detects concurrent activities on the same object, it sets up a cooperativity session, notifies the involved activities (i.e., the applications the involved users are using) of the session and invites them to rendezvous with it.

Because it is the objects, or rather, activities on objects, that trigger session management, this method is also called *artifact-based*. One sees

that it does not necessarily require explicit actions from users to set up or join sessions; hence the name "light-weight".

Explicit session management handles the problems of initiation of new sessions, notification of existing sessions, and rendezvous well, but, it requires some means for users to control their policy with respect to session participation. Also, it requires active voluntary participation in session management from applications.

Implicit session management is an attempt at making session handling *environment-centric*, by providing an *open* mechanism for session management usable by all applications in various contexts.

Also, allowing users to join sessions dynamically brings up the problem of how new users in a session catch up with the others

Explicit session management seems most useful in situations where a certain degree of formality and/or planning is involved in the cooperative activity. Implicit session management seems useful for allowing collaboration of a more serendipitous and transient nature to take place.

Shared user interface aspects

On the user interface side, finding the exact set of requirements is a much-debated subject in the literature. Many contributions consider human-centred aspects of collaboration [RG92, SM93], in order to discover the nature of cooperation between humans and thus improving our sense of the nature of the requirements. Also, there have been technical contributions on various aspects (e.g., user interface coupling [DC91]), and kits for building user interfaces (e.g., GroupKit [RG92] and Coast [SK+96]).

Some practical design issues to be considered when designing CSCW systems are:

group interface What particular aspects of the user interface are there to support aspects of the multi-user environment, and which should we use? Are they extensions of well-known single-user features, or are they new features specifically created to satisfy special needs of collaborating users?

notification control This covers the mechanisms of being given notifications from the system when other users make changes that affect one's work. The exact nature of the notifications depends critically on how the system is positioned in the space-time taxonomy.

The three C's of cooperativity

We will characterise the problem of providing collaboration support to hypermedia environments along the lines set out in [EGR91] and [HW98] combined. According to this view, there are three major facets of CSCW, on which all collaborative systems must take a position and with which they must deal accordingly. A summary is given in Figure 3.8, and the explanation of it follows.

1. **Collaboration**
   - Shared data management
     - Resolving concurrent write conflicts
     - Event notification
   - Shared user interface management
     - Modes of cooperation
     - Group interface support
   - Persistent session/collaboration information
2. **Communication**
3. **Coordination**

Figure 3.8: *Summary of the three C's, issues to consider for support of CSCW functionality.*

COLLABORATION. The support for collaboration can be subdivided into the management of three smaller problems:

1. *Management of shared data objects:* Support of (concurrent) access to data shared between several users, as well as for maintaining state consistency. Some aspects of this problem are [WL93]:
   - *Resolving concurrent write conflicts:* There are two ways to achieve this goal:
     - *User-controlled fine-grained shared locking:* In addition to locking mechanisms provided by transactions in the underlying store, users should be able to control locking of hypertext objects on demand, to indicate the intention of modifying the objects. The locks should be fine-grained enough to allow maximum cooperativity of work on relevant objects. They should be shared in the sense that even if an object is locked, it is always possible to read it.
     - *Immutable versioned objects:* All hypertext objects are considered immutable, but are versioned. Updating an object implies creating a new version of it. Simultaneous updates are allowed by branching the version tree, but this introduces the problem of how to merge the branches later.
   - *Fine-grained event notification:* This allows collaborators to monitor changes made to hypertext objects in real time. It should be fine-grained enough for clients to be able to distinguish what operations have been performed on the modified objects.
2. *Management of shared user interfaces:* This is the support for shared

	Same time	Different times
Same place	face-to-face	asynchronous
Different places	synchronous distributed	asynchronous distributed

Table 3.1: *Taxonomy of interaction in groupware by space and time.*

display states, to some degree, depending on the coupling wanted between users. It involves some propagation of updates between collaborators.

- *Modes of cooperation:* The technical demands put on the functionality of a collaborative system can be roughly characterised according to a *space-time taxonomy* [EGR91], as given by Table 3.1. Of these, synchronous distributed interaction is of course the most demanding in terms of system features needed to support it.
- *Group interface support:* These are the features of the user interface that support collaboration. They can be tailored versions of single-user features, or new features created to satisfy special needs of collaborating users.

  Issues like group focus, group awareness, group dynamics, and notifications, belong in this category.

3. *Persistent collaboration information:* To facilitate server crash recovery, collaboration information (i.e., active sessions, consisting of records of users, event subscriptions and user-controlled locks) should be stored persistently.

Providing shared data management by itself is termed *information sharing*—it can be provided by having shared storage servers. To support user interface management and group awareness also, it is necessary to employ *session sharing*, which is the topic of the following section.

COMMUNICATION. This specifies how collaborators communicate apart from the shared workspace. This may take place inside (integrated audio/video channels) or outside (e-mail, telephone) the shared environment

COORDINATION. This specifies the aspect which prevents users from engaging in conflicting and/or repetitive actions. Examples include informal plans agreed upon by co-workers and formal work flow methods. Also, user-controlled locking of hypermedia objects is a method of enforcing a simple coordination policy.

Finally, issues such as access control, security, etc., are important in all multi-user systems, but are peripheral to the problem of providing cooperation support as such, and will not be discussed here.

The next section is a discussion of systems described in the literature that combine hypermedia functionality with cooperation support, with emphasis on the aspects relating to the terminology introduced in this section.

3.3.2 Systems

COAST

COAST[4] [SK+96] is a middleware toolkit for developing synchronous groupware applications. It features an object-oriented architecture designed to handle many aspects of providing synchronous collaboration:

ARCHITECTURE. COAST uses a replicated architecture approach. Each application operates on exactly one document, with fully replicated document data on each application instance.

SESSION MANAGEMENT. A central session manager allows users to create, join, and leave sessions. Editing of a shared document part implies creating or joining a session. The session object that represents the session is shared between all users in the session. Dynamic sessions (i.e., entry of new users to an ongoing session) are also supported by an invitation mechanism.

COLLABORATION. Collaboration support in COAST can be described as follows:

SHARED DATA MANAGEMENT. An extension of the Model-Control-View design pattern is used to handle the sharing of objects. The model (the document contents) is shared and replicated on each client; the control and view objects are unique to each client. Access to the model from the view and the control goes through the session object, which coordinates coupling between application instances.

Shared data consistency is handled by using transactions to modify shared object fields. The optimistic concurrency control mechanism permits a local commit of update before it has been distributed between all application instances and committed globally. The local commit allows applications to continue working without delay, at the risk of having updates undone later if a transaction is cancelled globally.

SHARED USER INTERFACE MANAGEMENT. Provision is made for opening and closing application windows that provide a view of the shared model automatically when users enter and leave sessions.[5] Control and view objects access the model through a shared collaboration session object, which coordinates group awareness and the coupling between users.

Coupling modes can be defined dynamically, by specifying which aspects (object fields) of the model should be coupled between clients in the session. Provision is made for enabling smooth transitions between coupling modes.

PERSISTENT COLLABORATION INFORMATION. Session objects in COAST are persistent and are stored with the document.

COMMUNICATION & COORDINATION. These aspects of CSCW are not within

[4] CoOperative Application Systems Toolkit

[5] One may argue that this is an (unnecessary) design premise of COAST, since it seems to prohibit using it in a straightforward way for making computerised collaborators.

the scope of the COAST toolkit.

SEPIA

SEPIA[6] [SH+92, HW92] is designed to be a tool for collaborative authoring of complex hypermedia documents. Its hybrid architecture shares a common centralised store between applications. Applications are replicated and share notifications and updates between themselves to maintain a common state.

A work session in SEPIA uses explicit session management by invitation. The set of authors in a session is fixed. SEPIA has no provision for persistent session management.

COLLABORATION. Collaboration support in SEPIA can be characterised as follows:

SHARED DATA MANAGEMENT. Concurrency control and data coherency are handled by sharing of a single multi-user DBMS, using its transaction management capabilities. Additionally, user locking of objects is provided by CHS[7], which sits on top of a DBMS. Notification of updates to shared objects are broadcast among the clients themselves.

SHARED USER INTERFACE MANAGEMENT. SEPIA supports an asynchronous work mode and two synchronous work modes (loosely and tightly coupled).

The collaboration mode may change as users work—care has been taken to make the transitions between coupling modes smooth. Some coupling mode transitions are triggered automatically (e.g., when a second user starts working on a node, that user automatically switches from individual to loosely coupled mode with the first user). Others are user-initiated (e.g., leaving a tightly-coupled session).

Group awareness is provided by a status line indicating the status of each participant in a collaborative effort with respect to their collaborative mode. A "door bell" sound is use to indicate the arrival of a new user.

Group focus in the tightly coupled work mode is handled by providing a WYSIWIS shared view to all participating users. Also, concurrent telepointers are provided. To coordinate their work, users are expected to use communication facilities.

COORDINATION. There is no formal support for work coordination in SEPIA.

COMMUNICATION. Facilities are provided for communication (one digital audio channel and one analog audio/video channel) intended for use in tightly coupled sessions.

[6] Structured Elicitation and Processing of Ideas for Authoring
[7] Cooperative Hypermedia Server—a successor to the German HyperBase

DOLPHIN

DOLPHIN [SGHH94] is a successor to SEPIA. With DOLPHIN, the focus has shifted from cooperative authoring of hypermedia content to providing collaboration support between participants in meetings.

ARCHITECTURE. DOLPHIN utilises a client-server architecture to allow multiple distributed clients to share common hypermedia objects stored in a cooperative hypermedia engine server. The architecture is closed— there is no possibility of adding DOLPHIN functionality to external applications, and it is not possible to extend the hypermedia engine by adding new structure servers. Later versions of the system are based on COAST [SK+96], and derive many of their properties from it.

SESSION MANAGEMENT. DOLPHIN allows multiple sessions per hyperdocument.

COLLABORATION. Collaboration support in DOLPHIN can be described as follows:

SHARED DATA MANAGEMENT. The system allows multiple sessions per hyperdocument, and full concurrent sharing of objects in workspaces.

SHARED USER INTERFACE MANAGEMENT. Attention has been paid to providing private as well as public workspaces for users. For public workspaces, an explicit sub-objective of the system is to cover multiple cooperative work modes in the time-space taxonomy, and in particular the transitions between them; that is, users of the system can create hypermedia structures before, during, and after a meeting, working alone or in groups.

Group awareness is handled extensively through indications of coworkers present in a workspace, as well as through telepointers. Various interface details, such as private pointers as well as telepointers are also addressed for. The system supports unusual hardware (liveboards) and input methods (pen with gesture recognition) which enhance the interface in meeting situations.

COORDINATION. The hypermedia objects created by users are used not only to represent the subject matter of cooperation, but as a medium for cooperation and coordination [Str94]. Apart from that, there is no formal support for coordination in DOLPHIN.

CHIPS

CHIPS[8] [HW97, WH98] is a successor to COWFISH[9] [WH97] that focuses on supporting coordination (e.g., in business processes).

ARCHITECTURE. CHIPS features a three-layered architecture. The bottom layer is a basic CSCW support layer that is based on the COAST framework for synchronous and asynchronous groupware. The middle layer is a hypermedia

[8] Cooperative Hypermedia Integrated with Process Support
[9] COoperative Work in Flexible Information Systems using Hypermedia

layer that provides shared hypermedia object spaces. The two lower layers together provide cooperative hypermedia. The top layer provides process support services.

SESSION MANAGEMENT. Session management functionality is inherited directly from the functionality provided by the COAST framework.

COLLABORATION. Collaboration support in CHIPS relies on the same mechanisms as those introduced in SEPIA and DOLPHIN—the COAST framework handles all aspects of shared data management and persistent collaboration information. Shared user interface management is supported through sessions that couple hypermedia model objects to their view and control object(s).

COMMUNICATION. For direct communication, audio/video connections between participating workstations or sites can be set up, providing people with an informal communication channel. Indirect communication can be achieved through shared hypermedia documents. This enables users to use social protocols to coordinate their work.

COORDINATION. The top layer in the CHIPS architecture models work processes and shared artifacts using cooperative hypermedia. The rationale behind this design decision is that the logical structure of both workflow processes and hypermedia networks (in their model) can be seen as directed graphs; the straightforward correspondence between concepts in the two fields justifies using hypermedia networks to model workflow processes.

Care has been taken to support workflow processes at various degrees of formality. The process of gradually evolving informal workflow processes through refinement to is also supported, thus allowing formal processes to emerge gradually from informal origins. Also, processes can consist of subtasks of varying formality (e.g., manual tasks that include automatic subtasks).

Tasks and activities represented formally as workflow processes can be executed by the computer, performing such tasks as data hand-over in the process. Hypermedia structure serves as both a process definition formalism and as process data manipulated by the processes. The ability to switch between those interpretations enables easier handling of exceptions to, and, adaptations of processes.

DeVise Hypermedia

In [GHMS94], an implementation of a cooperative hypermedia system, called DeVise Hypermedia (DHM), is described. It is based on the *Dexter Hypertext Reference Model* [HS94] and it provides cooperativity support as follows:

ARCHITECTURE. DHM features a centralised architecture. A component called the *Run-time Process*[10] (RP) is the link between applications and a central hyperstore. An RP serves hypermedia facilities to all applications in use by *one particular* user. It acts as a session manager to the user's applications, and it

[10] It performs much the same function as the Tool Integrator in HyperForm

is a client to the hyperstore.

COLLABORATION. Collaboration support in DeVise Hypermedia can be described as follows:

SHARED DATA MANAGEMENT. Shared data management relies on the use of a shared hyperstore that is based on a general-purpose OODB developed in parallel with DHM. Structure coherence is enforced using the transaction management facilities in the hyperstore. Support of shared data management through versioning is planned.

Whenever a user starts the RP to begin hypermedia work, it always starts a transaction with the hyperstore which lasts as long as the RP runs. An object in the hyperstore, and its transitive closure, can be (read/write) locked by being involved in a hyperstore transaction. Two features help management of access contention during long-lasting transactions; firstly, locks are shared; secondly, locks may be changed dynamically by request. The latter feature meets the same needs as those provided by user-controlled locking from a frontend perspective, but they differ in the backend—user-controlled locking is meant to eliminate long-lasting transactions, whereas lock changing is meant to circumvent the problem of long-lasting locks caused by them.

SHARED USER INTERFACE MANAGEMENT. All asynchronous cooperativity modes described in Table 3.1 are supported by the use of event notifications (and locks on hyperstore objects). Synchronous cooperativity support is planned, and will be done through the use of a shared window system (i.e., a centralised approach).

Hyperstore transactions provide a source of events, allowing all interested parties to be notified when the transaction modifies objects. The minimum granularity of notifications is to report modifications to objects. Another higher-level event source allows RPs to monitor the start, commitment, and abortion of transactions executed by other RPs.

PERSISTENT COLLABORATION INFORMATION. The RP does not support persistent sessions.

COMMUNICATION. Communication facilities between users is a planned extension that will be added together with synchronous collaboration support.

COORDINATION. There is no formal support for work coordination in DHM.

BSCW

The BSCW[11] Shared Workspaces [B+97] from GMD-FIT[12] is a CSCW system which attempts to utilise the WWW to support cross-platform cooperative work.

COLLABORATION. Collaboration support in BSCW can be described as follows:

[11] Basic Support for Cooperative Work
[12] Institute for Applied Information Technology

SHARED DATA MANAGEMENT. This is done by copy or by reference; that is, content is entered into or uploaded to the system server, or it is referenced via URLs. There is no provision for concurrent access to shared data, and objects in the system can not be locked.

SHARED USER INTERFACE MANAGEMENT. This is achieved using ordinary WWW browsers; the server is an ordinary HTTP server, on top of which the BSCW system runs as CGI scripts. The nature of WWW browsers implies that the system cannot support synchronous collaboration, due to the lack of coupling between users (HTTP is a sessionless and stateless protocol), and of a mechanism for automatic propagation of updates.

GROUP AWARENESS. Support for group awareness is an extension available for more advanced users. It consists of a Java applet which provides information about logged on users, as well as their current status (present or "away").

COMMUNICATION. Communication is handled by several mechanisms. A threaded discussion forum is an integral part of the system. The system can schedule meetings, using tele-conferencing software to bring participants in touch if required. In addition, the aforementioned Java applet provides a live chat facility.

COORDINATION. The system tracks the actions of users. In the threaded discussions, every create, change, and read action is logged. Other group members can see the log.

HyperBase

HyperBase [Wii93] is a stand-alone hypertext database system that server navigational hypertext to hypermedia-enabled applications. It is designed to support cooperative use.

ARCHITECTURE. The HyperBase is an implementation of a hyperstore that uses a file system for persistent storage and builds a complete hypertext database from scratch on top of that. It is a layered system with the following layers: basic entities (file system storing and retrieval), basic services (operations on entities) and multiuser services (locking and event handling).

The navigational hypertext model provided by the basic entities is a simple data model that offers nodes and links, each consisting of fields that contain various information.

SESSION MANAGEMENT. HyperBase uses an event-driven approach to public data distribution. There is no concept of groups of users that form sessions with associated group properties.

COLLABORATION. Collaboration support in HyperBase can be described as follows:

SHARED DATA MANAGEMENT. Hypertext content, as well as link information, is stored in the HyperBase; this ensures that a hypertext is not changed

outside the control and awareness of the system. Concurrent data access is handled by fine-grained user-settable locks on whole objects or object fields. Setting a write lock on an object does not prevent reading of it (shared locking).

There is versioning support for nodes, but not on links. The versioning system provides a global and multiple local versions for each node. A global version is visible to all clients. A local version enables clients to work in privacy until it is checked in as a global version.

SHARED USER INTERFACE MANAGEMENT. An event mechanism allows clients to be notified whenever (fields of) objects of interest change. This barebones mechanism is sufficient to allow clients to operate in all four quadrants of the space-time matrix, including synchronous distributed interaction.

Since there is no concept of cooperativity sessions in the system, there is no explicit representation of workgroups and the coupling between its members; this makes it harder to provide group awareness mechanisms, etc. However, it is possible to identify owners of locks. This provides the basis for a simple group awareness mechanism.

COORDINATION. The locking mechanism is the only formal coordination mechanism in HyperBase.

EHTS

EHTS[13] [WL93, Wii93] is a collaborative hypertext authoring system that is an application sitting on top of HyperBase.

ARCHITECTURE. EHTS consists of two tools: a multiple window text editor and a graphical browser.

SESSION MANAGEMENT. There is no explicit concept of groups, or sessions that group users, in the HyperBase or EHTS systems.

COLLABORATION. Collaboration support in EHTS is designed to rely on the mechanisms of the underlying HyperBase in the following way.

SHARED DATA MANAGEMENT. The underlying HyperBase provides shared data management through its fine-grained locking mechanisms. Locks are allocated and released when the user switches between browse and edit modes in some node.

SHARED USER INTERFACE MANAGEMENT. A simple kind of group awareness is provided to all readers of an object; the readers are notified whenever the data they are viewing may change. Four different modification notifications are issued: Intention, when someone obtains a lock on an object; update, when a writer actually modifies an objects, the new contents is distributed; completion, when a write lock is released; and, deletion, when a field is deleted.

COMMUNICATION. Group members can communicate in real-time and send asynchronous messages within the EHTS environment.

[13] Emacs HyperText System

HyperForm

HyperForm [WL92] is a sequel to HyperBase. The primary goal of its design is a re-structuring of the facilities found in HyperBase, in such a way that the system becomes more extensible.

ARCHITECTURE. HyperForm is a centralised hyperbase server implemented around the Elk[14] interpreter. Sophisticated object-oriented programming is implemented as an extension to the language. All classes and instances are treated as objects that are stored in the database. Dynamic extension of the system is possible through a similar language feature.

A component called the Tool Integrator (TI) is an extensible, tailorable interface between the HyperForm server and participating hypertext applications. It provides a basis for running hypermedia applications written in Elk internally; or, it can provide local caching for external applications that choose to access HyperForm through the TI.

SESSION MANAGEMENT. HyperForm uses an event-driven approach to public data distribution. There is no concept of user groups that form sessions with associated group properties, in either HyperBase or the Tool Integrator.

COLLABORATION. In the following we will describe the properties relating to collaboration based on the standard facilities in HyperForm. Because it is highly extensible, it can support any kind of collaboration.

SHARED DATA MANAGEMENT. The basic locking mechanism provided in HyperForm is an adopt of the locking mechanism found in HyperBase. Updates to hyperbase objects can be grouped in transactions, to make several small updates appear atomic. A versioning system inspired by RCS allows a version tree of an object to be stored. An access control mechanism allows permissions for users/groups/others to get, set and delete instances and fields to be controlled.

SHARED USER INTERFACE MANAGEMENT. The notification control in HyperForm is based on experiences with HyperBase. It is unusual in the sense that the set of required events, as well as the circumstances in which they occur, are specified using Scheme expressions; thus, the system can provide much richer and better targeted event filters. It is possible to generate notifications on events unrelated to field or object updates, as also required.

COMMUNICATION AND COORDINATION. These aspects are not catered for in the standard HyperForm system, but can be added as an extension.

[14] A dialect of Scheme

HyperDisco

HyperDisco[15] [WL97] was implemented using the HyperForm[16] hypermedia system development environment. This section describes features found in the standard system. Due to its highly extensible and tailorable nature, practically all "missing" collaboration functionality can be added.

ARCHITECTURE. The HyperDisco is a three layer architecture OHS consisting of participating applications, the Tool Integrators (TIs), and the HBMSs. The latter two components are extension of similar components inherited from HyperForm. The HyperDisco HBMS implements the data model layer and provides storage of objects in this layer (i.e., nodes, links, composites, anchors, or subclasses thereof). The Tool Integrator implements the integration model layer, and is the central component of the architecture. It supports multiple integration models that allow distributed, heterogeneous tools to participate in hypermedia services using multiple HBMSs.

SESSION MANAGEMENT. The tool integrator maintains communication sessions with its connected clients, but they don't appear to be used for CSCW purposes in the standard system.

COLLABORATION. The basic style of cooperativity support mechanisms in HyperDisco are adopted from it predecessors, notably HyperForm, in the following way.

SHARED DATA MANAGEMENT. The basic model of a HyperDisco storage server is that it serves a single *workspace*. This has implications for the Tool Integrator because it allows clients to work on multiple workspaces distributed over the Internet [WL97]. This is an enhancement of the abilities for shared data management over those found in HyperForm.

The Tool Integrator mediates access to one or more HBMSs for integrated clients. It handles retrieval, manipulation, and storage of linking and anchoring information; and, distribution of events. It can also handle caching of data for its clients.

SHARED USER INTERFACE MANAGEMENT. The mechanisms in HyperDisco for providing shared user interface management are essentially the same as in HyperForm.

COMMUNICATION. There is a facility in the Tool Integrator whereby clients connected to it can communicate using an event-based mechanism.

COORDINATION. There is no support for formal coordination of work in HyperDisco.

[15] *Hyper*media platform for *Dis*tributed *co*llaborative computing environments.

[16] The series of systems from Aalborg discussed here are all attempts at making a general-purpose, extensible hyperbase system for serving (customisable) hypertext structure to hypermedia-enabled applications. The later systems (HyperForm and HyperDisco) are extensible enough to satisfy all kinds of cooperation over hypermedia structures.

OHSWG

The OHSWG is engaged in research of the CB-OHS architecture. As part of this effort, it has been suggested [WN98] that generic hypermedia functionality should be part of the architecture. This will ease the task of making structure service components so that it primarily handles the adoption of CSCW functionality to its particular domain, thus enhancing system modularity and re-use of shared facilities.

In [HW98], it is suggested that a major step towards collaboration support in OHS can be taken by integrating session management and structure service. By considering several alternatives, the author concludes that the most advantageous style of integration is to add a session manager as an integral part of each structure service, because of the following:

- The exact nature of the session management can be tailored to the specific needs dictated by providing collaboration support for the associated structure service.

- Transition management between coupling modes is easier to handle within the same session manager rather than between session managers.

According to this view, applications enter a session whenever they communicate with a hypermedia structure server. This forces them to participate in some (minimal) cooperation protocol with some degree of coupling to other users.

In short, applications must speak a session management protocol in order to be allowed to speak a hypermedia structure exchange protocol.

The ITU T.120 standards

An implementation of the session concept, as described in Section 3.3.1, will require some multi-point communication protocol between the server and its clients. We note that some ITU[17] protocols in the T.120 series [18], particularly T.122 and T.125 on the MCS (Multi-point Communication Service), may help in this direction. They were originally developed for teleconferencing applications. Thus, some of the services offered by these protocols may satisfy communication requirements for tightly coupled collaborative sessions.

3.3.3 Summary

This section introduced standard terminology from the CSCW field that is used to characterise various properties of collaborative systems. Subsequently, we presented a series of examples of prominent hypermedia systems that include collaboration support. Each of them was described using the terminology given earlier, and their merits and shortcomings with respect to CSCW functionality combined with hypermedia were discussed.

[17] International Telecommunication Union
[18] Available from the ITU's web site: `http://www.itu.org/`

The survey of the field of collaboration enabled hypermedia systems shows that a reasonably firm understanding of the infrastructure requirements for collaborative hypermedia systems has been established. Issues such as shared data management, notifications, locking are well understood; session management, general application architecture, and scalability issues still open issues. Front-end issues in particular are still a research subject in general, not only within the area of collaborative hypermedia systems.

Some of the reviewed collaborative systems provide functionality similar to what is required in a collaborative spatial hypermedia systems (e.g., DOLPHIN [SGHH94]). However there have been no attempts so far at making a full-scaled collaborative information analysis system that includes a spatial parsing.

3.4 Summary

This chapter started by giving a short account of the evolution of OHSs, the relevant terminology, followed by a survey of systems pertaining to the concept. It is considered in Sections 5.1 and 6.1, where we discuss the architecture of our system.

Then, Section 3.2 surveyed a number of spatial hypermedia systems; this analysis is considered in Sections 5.2 and 5.3, where we present our spatial model and the spatial server, to motivate the design desicions.

Finally, Section 3.3 briefly introduced the field of CSCW, with emphasis on its relation to the hypermedia field. A number of systems that combine CSCW with hypermedia were characterised. Our design of CSCW features in Section 5.3.3 will be motivated partly by considering the lessons learned here.

Chapter 4

Thesis

In the preceding chapter, we presented important previous work in the fields of hypermedia system architectures, spatial hypermedia, and cooperative hypermedia.

This chapter presents the concise list of issues (Section 4.3) that we have decided to cover throughout the rest of this thesis. This list is an outcome of: an analysis (Section 4.1) of the issues presented in the summary to the scenario (Section 2.4), by contrasting them with the information presented in the literature survey; and, a synthesis (Section 4.2) that considers the merge of the areas analysed previously. Occasionally, we will choose not to address certain topics any further. Thereby, we narrow down our focus.

4.1 Analysis

In this section, we reflect on the issues presented in the scenario section, with the lessons learned in the literature survey in mind. Some delimitation of our project is done based on the issues that are analysed.

4.1.1 System architecture

With respect to the system architecture issues of Section 2.4, this section will consider each one of them in greater detail, pinning down more specific issues with the intention of providing guidelines for the design of a spatial hypermedia system.

Openness and interoperability

As mentioned in the introduction (Chapter 1), there are some separate evolutionary directions with respect to hypermedia systems. One direction is concerned with system design issues of mainly navigational hypermedia systems, while another direction is concerned with problem domains that are different from the well-known associative domain (e.g., information analysis, taxonomic reasoning, and argumentation support). Recently, system designers have suggested CB-OHSs, allowing a distributed, flexible, and powerful framework that

is extensible and tailorable with respect to new hypermedia domains. This is an attempt to close the gap between system and domain researchers [NLW98].

Since our attention is on a spatial hypermedia system that is open and allows interoperability, we design ours to be part of a CB-OHS. A CB-OHS, as described in Section 3.1.1, consists of three layers where the storage layer is common to all structure servers. By assuming the provision of a CB-OHS infrastructure (including a hyperstore), this means that we "only" have to design a spatial structure server to build a spatial hypermedia system. To demonstrate the facilities provided by the spatial structure server, we will also design a spatial client that makes use of the services provided by the structure server.

Structure persistence

Persistence of structure has always been an important issue in hypermedia systems. Nevertheless, the persistence issue has changed character over time. The introduction of CB-OHSs puts new requirements on the persistent store, storing the structure information. Stores are now required to provide complex services on generic structural abstractions, such as access control, version control, notifications, and now, structure aware services.

With the assumption of a provided CB-OHS infrastructure (including a hyperstore), we do not need to concern ourselves with the issue of how to store structure. This, in turn, leaves us "only" with the issue of *what* to store.

Naming

Naming concerns the identification of objects, documents, components, and the like. Traditional OHSs typically use internal naming conventions incompatible with other systems. Once again, the introduction of a CB-OHS puts new requirements with respect to naming on the infrastructure [NLW98]. In CB-OHSs, names of objects have to be valid across structure servers. Additionally, because of the openness of the different layers, naming of components (what port to connect to, etc.) has to be taken care of.

Since we assume that a CB-OHS infrastructure is provided, we do not have to worry about these issues—we consider it a part of the given infrastructure.

Structure processing

As described in the OHS terminology (Section 3.1.1), interoperability between OHSs (therewith the ability to reuse structure from them) has become an increasingly important issue [WN99, Wiib]. This leads to the question of how to provide structure produced by one OHS to another OHS working on the same or other structural abstractions. We have to address this problem if we want interoperability to be realised.

Introducing a CB-OHS makes this requirement even more obvious [NLW98]. If there is a system designed to support interoperability, the need for a structure processing policy is required. In what other ways are inter-domain authoring tasks [NWL98] handled?

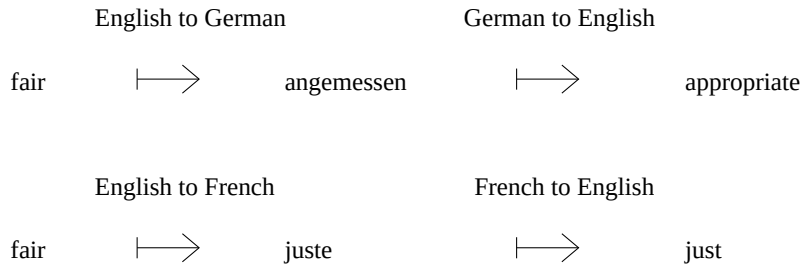

Figure 4.1: *This figure shows the effect of "noise" in the process of translating words. The example uses AltaVista's translation service, resembling some possible structure processing services.*

To give a better understanding of structure processing, we will try to analyse different approaches. There are various aspects of structure processing (see also [LS94]). Here, we will look at the following:

structure openness This is the ability to allow an open set of structure domains to participate in structure processing. Note that this definition is different from openness of an OHS.

accuracy This is proportional to the inverse of the number of structure processes necessary to get the desired representation, presuming that the quality of structure is never improved by structure processing.

Ideally, structure processing should be both open and accurate. The problem is that these two aspects are somehow incompatible. On the one hand, openness requires some kind of common/generic object capable of expressing all possible structures, representing some kind of interchange format. This implies that at least two structure processes are required (indirect) (e.g., processing from domain A to the interchange format and from the interchange format to domain B). On the other hand, accuracy aims at minimising the number of structure processes, since this always includes some kind of interpretation which in turn introduces the possibility of misinterpretation (noise, see Figure 4.1). According to this, structure processing from domain A directly to domain B should be more accurate. The problem with direct structure processing is that a domain (structure server) has to know in advance which domains it wants to support and how to convert these domains, reflecting inflexibility in the case of new emerging domains.

Generally, there are three different ways to handle structure processing (see also Figure 4.2):

interpretation Structures are expressed using one (generic) structural abstraction that is interpreted by entities using it.

translation Foreign structural abstractions are translated into native structural abstractions.

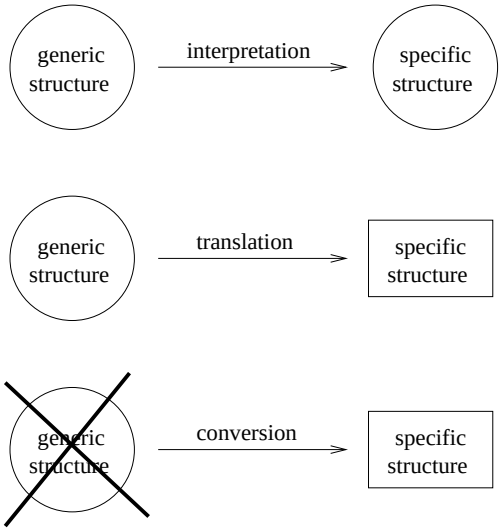

Figure 4.2: *Visualising the differences of structure processing mechanisms.*

conversion Foreign structural abstractions are changed or modified to become native structural abstractions.

What describes the distinction of the three processes best is the level of "egocentricity". Structure conversion, for instance, does not consider other interested parties and modifies an object to fit its own perception. Often, irrelevant information for the destination format is discarded, revealing a high degree of egocentricity. Structure interpretation, on the other hand, adapts a (foreign) structural abstraction, accessing and modifying only the characteristics relevant to it. This shows a low degree of egocentricity in combination with the policy of information preservation (irrelevant information is kept untouched).

With respect to CB-OHSs, structure processing will obviously be performed in the structure server or more accurately the structure processor. As mentioned in Section 3.1.1, a structure server is some kind of a mediator between generic and domain specific structural abstractions.

In our case, we have decided to use structure translation, thereby generating spatial structural abstractions based on the generic structure stored in the hyperstore. Additionally we ensure that our translation preserves information irrelevant to spatiality.

When considering how structure servers are going to process structure generated by other structure servers, we are entering an even more complicated area. Much more experiences with CB-OHSs supporting multiple structure servers is needed.

Although our project focuses on the ability to use spatial structure in other contexts (domains), we will consider the issues of how to process structures between domains (or via the hyperstore) as outside the scope of our project. In other words, we will focus on providing spatial structure as generic structure,

potentially being (re-)used by other domains, but *not* how to process existing structures generated by other domains.

4.1.2 Spatial functionality

In this section, we will take a closer look at the issues related to spatial hypermedia listed in the summary section of the scenario (see Chapter 2).

The VIKI system described in Section 3.2.3 has for some years represented the state of the art concerning spatiality The system will in this thesis serve as a basis for discussions concerning spatiality, and we will frequently compare VIKI's ideas with our own ideas.

Spatial expressiveness

The literature survey (Chapter 3) listed several aspects that characterise spatial hypermedia. It was mentioned that spatial hypermedia focuses on structure creation by means of emerging structure. Flexible means of expression, along with the ability to enhance common understanding, was mentioned as some of the strengths of spatial hypermedia. VIKI presents a high level of expressiveness concerning structure creation. Apart from structures expressed by means of position, proximity and alignment, VIKI also supports expression of structure using colours, shapes, border thickness of items, etc. This level of expressiveness is a pivotal aspect of spatial hypermedia today, and certainly should be incorporated into every spatial hypermedia system.

Although VIKI is a structurally expressive tool, we think that we can simplify its data model and still keep the same level of expressiveness. VIKI differentiates between collections, composites and space units. These structural abstractions can be unified into one structural abstraction, capable of expressing the same as the previous three abstractions. The manner in which one can do that is described in the conceptual design (Chapter 5).

VIKI does not support creation of external links across data objects, as in Boxer and Storyspace. This could be useful in some situations, in which lack of expressiveness calls for a combination of characteristics from different hypermedia domains. The idea of mixing structural abstractions across structural hypermedia domains is interesting, but it is not something we will treat in this thesis.

In our spatial system, we will aim at maximising the ways by which users can express implicit structure in the information analysis domain.

Spatial parsing

For a system to do spatial parsing, thereby producing explicit structure from visual structure, we have to decide what kind of relations should be reflected in an explicit structure representation. As mentioned in the previous section, many ways to express relationships exist in modern spatial systems.

What relationships should be found and expressed through explicit structure? The extreme case is to use all visual aspects of visual appearance, since all of it can be crucial for the understanding of the problem. In this case, a parser

should use all aspects of what is expressed visually to create an exhaustive explicit structure.

One may argue that structure expressed by position, proximity and alignment, etc., is primary structure. This means that nuances, expressed by differences or similarities between colours, shapes, border thickness, etc., are secondary structures. The rationale is that the task of sorting objects spatially by means of proximity, alignment, etc., is similar to the task of arranging papers into a structured arrangement of piles and lists of papers on a desktop. This kind of structuring can express clustering of objects into (nested) collections. In this context, colours, shapes, etc., seem better suited for expressing nuances of the primary structures (e.g., relations like "part of" versus "not part of" or "good" versus "bad") [MS97].

In this thesis, we will look at explicit structures derived from "primary" spatial properties of objects, such as position, proximity and alignment only. The "secondary" aspects of visual structure (called "tacit codings used implicitly by the user" [MS97]) are not used for the generation of explicit structure, but are stored in the hyperstore.

Visual feedback of explicit structure

We focus on providing general explicit structure via a hyperstore, so that the general structure can be used outside the information analysis domain. This suggests the usefulness of a feedback mechanism that can tell the user how an implicit structure has been parsed and stored as explicit structure by the system—this enables the user to verify that what is stored is what was intended.

Storing explicit structure persistently is not only justified by reuse of structures across domains. Persistence also ensures that visual feedback of explicit structure is straightforward to display, without the need for a possibly time consuming reparse of the complete visual structure.

In VIKI, explicit structure is not shown directly. Instead, users can ask the system to suggest composites, or invoke hierarchic click selection on specific objects to reveal explicit structure. The user thereby experiences a kind of shift in the perception of structure, from implicit to explicit structure. One reason for hiding explicit structure in VIKI could be that users would be information overloaded if they could see both implicit and explicit structure at the same time. On the other hand, if focus is on the created structure, it is nice for users to know which structures they have actually created. We will provide this visual feedback to users and look at how and when a spatial parse should be triggered to start a parse.

Preferences

In the scenario chapter, we briefly mentioned the provision of personal preferences in a spatial multi-user context. Here, we will elaborate on the subject.

Regarding preferences for presentation of structure in our system, one could imagine a user that prefers to see all lists as vertical lists. The reason could be that vertical lists express the same alignment and ordering as horizontal lists,

but working with both structural abstractions takes up too much real estate in the space. The user therefore wants to constrain himself to deal only with one structural abstraction of lists. Doing so means that the system will need a way to convert horizontal lists to vertical lists and at the same time maintain the original structural meaning of the original structure representation.

Regarding preferences for spatial parsing, it could be useful for a user to specify preferences for parse strategies. For instance, if a user tends to create lists in spatial settings with relatively larger distances between list items, the parser could be tuned to accept the spatial arrangement as a list anyway. The structure that should be found by spatial parsing depends on the subjective opinion of users, as mentioned in the literature survey (Section 3.2.2), and therefore, spatial parsing is well suited for adjustment by personal preferences.

Tailoring a system in the above mentioned ways can enhance the personal conception of the structuring problem at hand. On the other hand personal preferences can also cause problems. In the context of multi-user systems, personalised views are difficult to translate between different views specified by different users. Objects and structures can have different appearance (e.g., colour or shape) from view to view, making it difficult for the users to talk about the space. A simple request like "move the green icon in the vertical list", could be misunderstood because the users have different colour and structure preferences.

A solution that could support the notion of preferences in a spatial multi-user context could be to relate a set of preferences to either an information space or a session. This means that if users want to work with a space or join a session, the related preferences can be adopted. All users working in a space or sharing a session must share the same set of preferences. Changes to preferences are communicated to other participants. If information spaces and sessions are stored persistently in the hyperstore, preferences related to them can also be stored permanently.

Regarding preferences, we have chosen not to treat it further in this thesis. The purpose of this section was to highlight the issues that emerged while considering the subject. Implementing preferences in CAOS could be a interesting experiment, but will be postponed to future work.

4.1.3 Collaboration support

As concluded in the literature survey (Section 3.3), a number of hypermedia systems have been developed that incorporate CSCW support. Most systems have concentrated on backend support for collaboration. We have been unable to find systems that attempt to support collaboration in the information analysis domain.

In the following, we will discuss the issues concerning CSCW support that were hinted at in the scenario (Section 2.4), using appropriate terminology from Section 3.3.1, and partly under the assumption that we have already chosen to target our project at a CB-OHS (see Section 4.1.1).

Session management

Many authors in the field have pointed at session management as being key to providing collaboration support [HW98, PH+90, SK+96, WN98], since it is the concept that specifies and structures a group of collaborating users. Session management need not be a backend issue—it is perfectly possible for applications to perform session management between themselves. However, it is a backend mechanism in an *open* collaborative system, in which we would like heterogeneous applications to be able to coexist.

It is still an open issue whether to use a general, open mechanism for (implicit) session management, or whether it should be integrated with the hypermedia system itself. If session management is part of the hypermedia system, at what architectural layer should it be put?

Current CB-OHS research seems to favour integrating session management within hypermedia structure service components [HW98, WN98]. However, this raises the question of how to handle cross-domain applications (i.e., applications that use hypermedia from more than one domain). At the time of writing, actions are underway, designing and implementing an collaboration component handling open light-weight session management. The component is a peer to other structure servers, allowing both structure servers and applications to make use of its session management services.

In the following, we will discuss various concepts related to session management, and present our intentions, with respect to our project, for each of them.

Coupling modes. What modes of cooperation need to be supported in spatial hypermedia? Are the lonely, the loosely, and the tightly coupled modes enough for all structure services [HW98], or should one allow each structure service to provide its own, tailored set of modes?

Finding the set of coupling modes can be approached in two ways. Firstly, one could write scenarios that specify which ones should be provided, and what their characteristics are. Secondly, one could design the spatial structure service so that it can supply an extensive set of events; then, experiments with composing coupling modes as subsets of the range of events can be performed, in order to find a useful set of coupling modes. The former approach may yield the most imaginative and original ideas, by not being restrained by a particular approach of doing things, whereas the latter is useful for gaining practical use experiences with various modes.

Having found the set of coupling modes, there still is an issue of how to handle transitions between coupling modes, and how to bring newcomers to a session in progress up to date with the other participants.

In our project, we will take a closer look at what kind of coupling modes should be supported in a spatial environment, and implement some of them. Transitions between coupling modes will not be addressed further in this thesis.

Application architecture

The decision to use a CB-OHS enforces some architectural constraints on our collaborative spatial structure service. Storage of the common state is centralised via the structure service, so changes to the shared state are communicated via the server.

On the other hand, we are still free to choose a "replicated" application architecture to handle shared user interface management. The information that is shared between clients operating in the more demanding collaboration modes may not be very relevant or useful to the structure server (e.g., mouse movement). We propose an architecture in which the structure server acts only as a central information base that coordinates demanding cooperation between clients, by performing collaborative session management.

In our project, the application architecture is distributed, since it relies on a structure service to perform shared data management. We note that this architecture has to be taken into account when considering tight coupling modes later in this report.

Shared data management

Shared data management has been researched extensively in the hypermedia field, as we saw in the literature survey (Section 3.3.2). Many issues have been solved by building hyperstores on top of a general DBMS, utilising its transaction management facilities to provide basic structure coherency in the presence of multiple clients. Besides shared data management, notification mechanisms and user-defined locking are also extensively treated in the literature.

Since we assume the presence of a CB-OHS framework, including a hyperstore, we expect the availability of basic transaction management and notification from it. This helps us make a structure service that can supports shared data management reliably. We will not address user-defined locking.

Shared user interface management

The challenge in making collaborative spatial hypermedia is mainly in the frontend, in the design of the shared user interface. Many traditional concepts from the CSCW domain (e.g., group awareness) are not trivial to apply to spatial hypermedia in a sensible way.

Designing a shared user interface for collaborative spatial hypermedia can be roughly divided into two separate aspects:

1. Mechanisms that distribute and display changes to shared data.

2. Group interface features that support group awareness, group focus, and, group dynamics, etc.

In sophisticated shared user interfaces, the two may not be independent, but we will consider them to be so, for the sake of simplicity. In the following we will consider each aspect separately.

Distribution of structure changes. This is the most basic functionality in collaborative hypermedia systems. We aim for the ability of multiple users to observe the changes made to a space in real time.

Group interface features. It is not trivial to design the more advanced group interface features when they are applied to spatial hypermedia, due to a lack of suitably detailed use scenarios and experiences with working systems. The scenario(s) we give in Chapter 2 do not suffice for this purpose, because they do not specify precisely the mechanisms upon which the collaborators rely. Here, we will present some general issues concerning group interface features in spatial hypermedia.

Group awareness. Group awareness is a mechanism that reflects the presence of others, either because a location is shared, or because it tracks the whereabouts of the scattered members of a group. But the location of a user can be hard to define for some applications.

Consider a user X who is viewing and editing some space. The group awareness functionality reflects the presence of X to other users in the same space. X can also view and edit subspaces of the current space. For example, X can move items in the current space into subspaces. Does this mean that X is present there, too? If so, what if subspace structure can become cyclic?

Group focus. How can the system help a group of collaborators to stay focused on their work? Is the focus mechanism connected to the location of the collaborators? If so, the difficulty of defining the location of users (see the example above) has aggravating consequences for this feature. In particular, what if the focus involves several spaces?

Group dynamics. Dynamic joining and leaving of collaborators in ongoing sessions requires the ability of the system to bring newcomers up to date with the others, and to ensure that leaving collaborators exit cleanly. User interface mechanisms may also be used to signal the arrival and departure of session participants.

Again, these issues are tightly connected to coupling modes and will therefore be further discussed in the proceeding chapters.

Communication. Collaborative spatial hypermedia systems that permit synchronous work on shared data would benefit from having external communication facilities (e.g., chat, audio/video) when working synchronously, like any other synchronous collaborative systems. So far, we have not identified any special properties of collaborative spatial hypermedia that makes the choice of one particular type of media preferable to others. We will consider this provision of communication as being out of the scope of this report.

Coordination. Supporting coordination among collaborators in a collaborative spatial hypermedia system in terms of formalised work processes is partly against the spirit of such a system. It is designed to allow tentative informal structuring of information according to the subjective preferences of the ana-

lyst. So, why would users accept having formalised work processes that guide their informal analysis? We expect that user-defined locking and general access control mechanisms and agreement on structuring between collaborators will be the most useful coordination facilities among collaborators.

4.2 Synthesis

So far, the analysis section has only considered issues related to general aspects of CB-OHSs, spatial hypermedia, and collaboration support. This section looks at the synergy of combining these three aspects into one system, and the new issues that emerge due to it.

4.2.1 Motivation

An open spatial hypermedia system supporting collaboration in a CB-OHS framework will be a powerful tool for information analysis. In addition to the advantages each area contributes in itself, the very combination of them will further benefit a spatial hypermedia system in the following ways.

- Spatial tools will also become collaborative tools. It will be possible to organise spaces in cooperation with other users synchronously, thereby building a common understanding of information.

- Persistence of both visual and explicit structure allows asynchronous collaboration as well.

- By providing a standardised interface to the spatial server, interoperability of applications is allowed, enabling various applications to use spatial hypermedia services. This is called presentation interoperability [WN99].

- The provision of a generic backend interface allows various structure servers to (re-)use structure stored in the hyperstore. This leads to the possibility that spatial structure can be used as a basis for other hypermedia domains. This is called storage interoperability [WN99].

- A common backend will also open the possibility of collaboration across domains.

- The existence of different domains at the same time permits applications to support various domains simultaneously. This allows, for instance, navigational links to or from other domain specific documents, such as a space. We call this cross-domain linking.

- By incorporating our spatial hypermedia system in a CB-OHS framework, it inherits the ability to distribute its components, because they are all autonomous entities.

4.2.2 Analysis of issues

The combination of CB-OHS, spatial hypermedia and CSCW results in both simplifications of issues and emergence of new issues. The simplifications are the result of overlapping issues from each of the three areas. For instance, the assumption of a provided hyperstore, as part of a CB-OHS, makes some spatial hypermedia storage mechanism issues obsolete. Similarly, some collaborative issues, such as centralised versus replicated application architecture, will be influenced by the fact the spatial server suggests a centralised architecture with respect to structure sharing.

New issues emerge as a result of new requirements that the different areas impose on each other when combined. In the following, we will discuss a typical spatial hypermedia system (e.g., similar to VIKI) with an integrated spatial parser and consider what new issues arise when adding support for collaboration and interoperability?

Collaboration and spatial hypermedia

Having several clients working together on the same space introduces well-known collaboration issues, such as shared data management, shared user interface management, and session management. Does this have implications on our spatial structure server?

Consider a user editing a space; to maintain consistency between the spatial server and all its clients, the change has to be broadcast to all collaborating clients. In the presence of a spatial parser, the change of the visual structure can change the explicit structure—therefore, these changes have to be broadcast, too. This suggests a parser, triggered by user actions, that is capable of changing the explicit structure. We call this an automatic and continuous parser, whose role is to enforce consistency between the visual and the explicit structure at all times.

Since the task of organising information in a spatial tool is closely related to moving space units around and watch the emergence of implicit structure, this will result in frequent reparsings. In multi-user scenarios, this problem will worsen. For this reason, we consider the design of an incremental parser that parses only the regions actually affected by the user's changes.

Another issue is the consistency of the perception of the users working together. Especially in the field of spatiality, where structure is expressed through implicit structure, it is important to have a clear and, in the case of collaboration, common understanding of a space. This can be emphasised by visually indicating the implicit structure derived by the spatial parser. This supports the other considerations about visual feedback.

CB-OHS and spatial hypermedia

The introduction of the spatial information analysis domain in a CB-OHS obviously requires a spatial structure server. This puts new requirements on our spatial server, because it now has to base its structural abstractions on basic building blocks provided by the CB-OHS, that is, generic structure units.

Thereby, some structure processing is introduced to the spatial service concerning how to translate between spatial and generic structure. In general, the decision to adopt a CB-OHS framework has a major impact on many architectural design issues of the spatial service (e.g., the spatial parser becomes a behaviour).

Another issue that has to be considered by the spatial server is the provision of a generic spatial interface, since interoperability is one of the issues stressed by the use of a CB-OHS. This should allow applications enabled for spatial hypermedia to communicate with different spatial servers, supporting so called presentation interoperability [WN99].

4.3 Our focus

Our main topic is advanced spatial hypermedia, in the context of CB-OHS and CSCW. This section presents a concise list of issues, based on the previous analysis and synthesis of this chapter, that we have decided to focus on.

- *Openness and interoperability:* We will implement an open spatial hypermedia structure service, designed as a component for use in a CB-OHS framework, on par with other structure service components. In addition to open access to spatial structure services, re-use of spatial structure in other contexts/domains, and vice versa, will be encouraged by the architecture.

- *Continuous and automatic incremental spatial parsing:* Our spatial service component integrates a spatial parser that will maintain an explicit structure representing the implicit structure in the visual structure. Because we want both structures to be consistent with each other at all times, reparsing will be triggered automatically by changes to the visual structure. For efficiency reasons, parsing will be incremental.

- *Persistence of all structure:* All structure produced during work with the spatial structure service should be stored persistently, for possible re-use in other contexts. This not only includes the visual structure produced by users, but also the explicit structure produced by our spatial parser.

- *Visual feedback of parse:* We will make a spatial editor, based on the design of VIKI. Additionally, it will feature explicit visual indication of implicit structure, based on the explicit structure produced by the parser. This enables users to engage in a "dialog" with the spatial parser, because they are able to inspect immediately whether the explicit structure found is what they intended. This supports the process of refining an evolving structure, by continually showing the structures present at any given stage of the process.

- *Shared data management:* The adaptation of cooperativity to the spatial hypermedia field will be initiated by supporting synchronous editing of shared spaces. The spatial structure service will extend and tailor generic

collaboration mechanisms provided by the backend, and provide clients with collaboration mechanisms adapted to spatiality.

- *Shared user interface:* Our spatial editor will support a common view of shared data by allowing multiple editors to view the same space. Some basic functionality with respect to coupling modes, group awareness, and the like will be provided.

A system satisfying all these characteristics would share characteristics with systems like VIKI, DOLPHIN and HOSS (see Chapter 3). In this case, VIKI represents the spatial functionality, DOLPHIN the collaborative features and HOSS the interoperability properties.

Much of our spatial editor design is based on VIKI, in that many of its properties are inspired by VIKI. Its emphasis on the importance of emergent structure was an eye-opener. Our focus is mainly on extending the applicability of spatial structure.

The focus of DOLPHIN is on the support of collaboration between participants of meetings. Our focus is not on a specific event such as a meeting, but more on general aspects of collaboration in the context of spatiality. Since the support for collaboration in DOLPHIN is rather sophisticated, and since its user interface is what most closely resembles a collaborative spatial system, we have based our collaboration support on its concepts.

A similar attempt at integrating a spatial information analysis domain in a CB-OHS has only been performed by the HOSS project, where a simple spatial structure server was implemented, mainly for reasons of justifying the concept of hypermedia operating systems (roughly equivalent to the CB-OHS architecture). In our case, the focus is mainly on the consequences for spatial hypermedia in the context of CB-OHS and collaboration.

The resulting open spatial hypermedia system, supporting collaboration, and the individual solutions in themselves, is what our project contributes to advance the state of the art of both the hypermedia field in general, and the spatial hypermedia field in particular.

4.4 Summary

This chapter reviewed issues concerning the design of an open spatial hypermedia system with collaboration support. Simultaneously, we pinned down the issues that are addressed by the rest of this thesis. New features and issues were introduced that originate in the merge of such diverse fields as spatial hypermedia, collaboration, and CB-OHS. Some techniques for solving the issues were suggested, such as the automatic incremental parser. Finally, we listed the issues this work intends to contribute to. The relationship between a system solving these issues relative to representative systems concerned with spatiality, collaboration and CB-OHS was mapped out.

The aim of our project is to implement and experiment with a system capable of handling the issues mentioned in this chapter. This will be the topic of the following chapters.

Chapter 5

Conceptual Design

This chapter describes the conceptual design of our cooperative open spatial hypermedia system. The goal of this system is to provide a spatial environment that enables interoperability with applications, and that provides support for collaboration among applications (and their users).

Firstly, we take a look at the overall component-based framework, by splitting the system into encapsulated components. Secondly, we discuss the information analysis domain specific structural abstractions. Thirdly, the spatial server, corresponding to a structure server providing spatial structural abstractions, is specified. Fourthly, we present properties of spatial clients in general as well as for a more specific client: a spatial editor. Finally, we present the interfaces between the spatial structure server and its clients.

5.1 Open hypermedia architecture

As hinted at throughout the report, we base our system on, or rather extend, a CB-OHS. The CB-OHS framework was suggested by the OHSWG and is considered state-of-the-art. The framework has been briefly introduced in the literature survey, where we have given an outline of the architecture. This section gives a more detailed description of the framework, describing the facilities we assume will be provided, and the ones we will provide, resulting in a CB-OHS supporting spatial hypermedia abstractions.

5.1.1 CB-OHS

Figure 5.1 depicts a typical component-based open hypermedia system framework. It consists of three layers: the storage (backend) layer, the structure model (middleware) layer, and the application (frontend) layer. Each layer is represented by an open set of components. A component of the storage layer is called a hyperstore; a component of the structure model layer is called a structure server; and, a component of the application layer is called an application. The philosophy of a component-based framework is to try and identify autonomous sets of functionality that can be encapsulated in or assigned to components. This gives a clean conceptual division with respect to responsibility

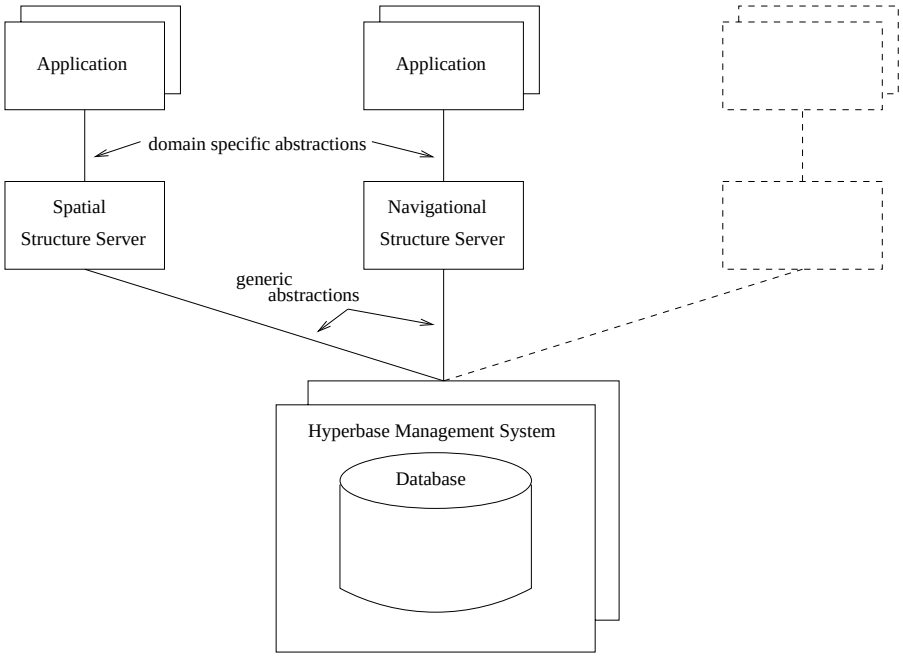

Figure 5.1: *A typical CB-OHS.*

and functionality.

Backend

The backend represents the layer that provides the basic structural building blocks of the framework. The fundamental structural abstractions are defined in the storage layer and used by the structure model layer which tailors them to domain specific structural abstractions.

Besides providing the basic building blocks of the system, the storage layer obviously has the purpose of storing structure and data persistently.

Middleware

The middleware represents the layer that provides domain specific abstractions to the applications. The structural abstractions are derived from the generic structure, provided by the backend. Therefore, a structure server is a type of mediator between applications and hyperstores.

Frontend

The application layer represents the hypermedia enabled tools used by the users. The available structural abstractions are provided by structure servers, each representing a hypermedia domain.

5.1.2 Our CB-OHS

Our project focuses on the implementation of an open spatial hypermedia system. The CB-OHS framework argues that this is done by building a middleware component that tailors the existing facilities, provided by the backend, to the information analysis domain. This offloads the developers, because they can reuse the backend layer. Therefore, we consider a hyperstore (backend) that complies with the OHSWG (de facto) standards, as given. Interoperability is partially achieved through the use of the CB-OHS framework.

The following three sections describe the three components that make up our spatial hypermedia system. Firstly, our expectations regarding the hyperstore are described. Secondly, our requirements for the spatial structure server are outlined. Thirdly, a spatial client is presented.

Hyperstore

If a hyperstore has to provide all its clients (the structure servers) with a common atomic structural building block, then there has to be a notion of a *generic structure*.

In addition to providing a generic structure abstraction and basic storage functionality, the storage layer should also provide services such as access control, versioning support, transaction support, notification control, scripting services and concurrency control [Wiia]. The fact that all these services are orthogonal to the abstraction being stored allows the hyperstore to provide these services, without altering the stored objects. This is similar to hypermedia systems that provide hypermedia services to applications' without affecting the applications services or data format. The storage layer groups all these orthogonal services into one central service provider: the hyperstore management system.

Since the hyperstore only works on one type of object (the generic structure abstraction), and an essential property of this object is the ability to refer to other objects, the services provided by the hyperstore can take this into account. This requires the explicit specification of an object's relational fields, thereby, allowing the storage layer to become *structure aware*, which in turn, allows more powerful services. For example it is possible to enforce "indirect" access control by setting permissions on access to objects [Nür97].

Because of the fact, that the provided services are applied on a generic structural abstraction, common to all structure servers, the services themselves are common to structure servers. This results in a storage layer that provides a generic structure abstraction *and* a variety of services common to all structure servers.

Before continuing, we have to address another mechanism that is sometimes used for storing structure. Instead of a generic structure unit (resembling some kind of inter-change format), one could use the object-oriented paradigm. It states that all objects (in our case structural elements) are accessible as such, because they are specialisations of the common superclass of all objects, often called `Object`. This would allow arbitrary structure servers to be able to access

arbitrary units of type `Object` from the object-oriented database (OODB). The main advantage of this mechanism is the ability to store units as domain specific structural abstractions, while at the same time allowing access to the unit via its `Object` type. Thereby, the structure server's task of processing structure from generic to domain specific structure would vanish. Unless structure from another domain is accessed, in which case a problem occurs. How should a structure server be able to determine which fields of a foreign object represent structural relations and which represent attributes? If this cannot be determined, the structure server cannot traverse foreign structure? These are still open issues in the field of OODBs. Another drawback of OODBs is its relatively bad performance in contrast to relational databases.

The remaining chapters of this report assume the hyperstore to be based on a relational database that provides a generic unit tailorable by all structure servers.

Spatial structure server

The main characteristic of a spatial structure server is the provision of domain specific structural abstractions and services to an open set of applications. This is done by tailoring the existing facilities provided by the backend. Therefore, the structure server has to comply with the hyperstore's interface to become a part of the CB-OHS.

As mentioned in the literature survey (Chapter 3), a structure server can be thought of as a structure processor (Sproc) and a set of computations over structure (behaviour). This splits our spatial structure server into a structural processing part, tailoring the generic abstraction to spatial abstractions, and a behaviour part, representing the spatial parser, which extracts implicit structures.

In addition to these services, our spatial server supports collaboration of spatial applications. This requires some kind of session management and shared data management. Again, the hyperstore can provide basic services that can be tailored to enforce domain specific policies.

All these services are finally provided through the interface of the spatial structure server. For interoperability reasons, this interface is an attempt to define a general spatial interface. If this interface is complied by other spatial structure servers, it allows spatially enabled applications to communicate with different spatial structure servers, one of them being ours. This addresses the presentation interoperability aspect [WN99].

Spatial client

To make our CB-OHS extension complete, we have developed a spatial client, complying with the spatial structure server's interface. This allows us to verify some of our assertions about what problems a cooperative component-based open spatial hypermedia system can alleviate or even solve.

Outcome

The resulting extension represents a component-based open spatial hypermedia system with all the commensurate benefits [WN99]. Openness is assured through the typical provision of a well-defined interface and the orthogonal storage of structure. Interoperability is alleviated by the infrastructure (i.e., the compliance of the hyperstore with the OHSWG standards and the attempt to provide a generic spatial interface for the structure server). Domain specific collaboration support is handled by the spatial structure server. The applicability [WN99] of the system is supported through the pluggable nature of CB-OHSs, consisting of autonomous components.

5.1.3 Summary

In this section, we have introduced the general architecture of a CB-OHS system. This was done by presenting the different components and their purpose with respect to the overall system. Furthermore, we introduced our spatial components as part of a CB-OHS.

In the following sections we will take a closer look at: the spatial abstractions used by the structure server and the application; the composition of the structure server (i.e., a spatial structure processor, a spatial parser and spatial cooperativity[1]); and, the requirements imposed on the spatial client.

5.2 Spatial model

In the design of our spatial hypermedia model, we want to define a model, capable of expressing as many properties of spatiality as possible. In other words, we want to define some kind of "generic" spatial model that describes a maximal set of spatial structural abstractions.

The spatial model of VIKI allows users to use *item* and *subspace* objects (see Section 3.2.3). In VIKI, items and subspaces are separate entities. VIKI's items have visual properties, and can refer to objects with content. Multiple items can refer to the same content object (such items are also called virtual copies). Subspaces contain a spatial arrangement of items and nested subspaces.

The basic unit of our spatial structure abstraction is the space unit. Our space unit abstraction is a generalisation of the two concepts defined by VIKI. It unifies both the concept of an item and a subspace, thereby eliminating the need to distinguish between the two. Multiple space units can refer to the same content object, thereby making our model capable of expressing virtual copies. Separating content (data) from the space unit (structure) makes our spatial model open in the sense that it is possible to change structure, without changing data (expressing the orthogonality of data and structure). External structure makes our system an open hypermedia system [NWL98].

Adopting this model over VIKI's model has certain benefits. Firstly, it is simpler, introducing only *one* object type that structure servers and applications have to relate to. Secondly, it is more powerful, in that it becomes possible to

[1] CSCW functionality adapted to work processes in spatial hypermedia

use the content of a space unit as an annotation to a subspace. Thirdly, it allows subspaces to be part of explicit structure, extracted by the parser, because there is no difference between items and subspaces.

The next section describes what makes up a space unit, followed by a section on how to compose space units into spatial structure—this includes explaining the distinction between visual and explicit structure units.

5.2.1 Spatial structure abstraction

A space unit is represented using an adaptation of the general structure model offered by the hyperstore. We assume a generic structure element to consist of an identifier and a set of characteristics. Each characteristic is either a set of attributes or a set of references. The attributes are used to contain information about the state and properties of the elements, while the references refer to other generic structure elements.

Field name	Field type	Meaning
`parent`	REF	The parent of this space unit
`children`	REF	The subspace of this space unit
`content`	REF	Content text
`kind`	ATTR	What kind of space unit this is
`title`	ATTR	Title text
`bounds`	ATTR	Position and size
`bg`	ATTR	Background colour
`border`	ATTR	Border colour
`shape`	ATTR	Border shape
`thickness`	ATTR	Line thickness of border

Table 5.1: *The fields of a space unit.*

In our space unit, a field is a characteristic with a predefined meaning in the spatial information analysis domain. The set of fields defined for a space unit is shown in Table 5.1. In addition to these fields, clients may define an arbitrary set of extra fields. The hyperstore may provide metadata for each space unit (e.g., creator, creation date, permissions, etc.).

We use the `kind` field to signify whether the space unit represents a visual space unit or an explicit representation of some implicit structure. A spatial parser may define several `kind` field values to differentiate between the implicit structures it is able to recognise (for examples, refer to Figure 3.7).

The `parent` field of a space unit refers to its parent space unit—as can be seen from the name, our model allows only one parent per space unit. The `children` field refers to the space units contained in the unit's subspace. This represents a `1-to-n` directional relation (i.e., each space unit has exactly one parent and `n` children).

The visual appearance of a space unit in its parent's space is defined by its `bounds`; its colours (`bg`, `border`); and, its surrounding frame (what `shape` the frame has, and the `thickness` of it). The set of possible shapes is open to

whatever shapes clients want to use. Every space unit can specify a `title` as well.

A space unit can refer to content, which is defined by the `content` reference. The content unit is represented as a generic structure element with one characteristic, namely `text` (see Table 5.2). The content of the field can contain either direct content (e.g., plain text) or indirect content (e.g., a textual representation of an URL specifying some external stored document of arbitrary media). The content unit resembles data in that it has no further references (so called "degenerated structure"). In other words it resembles some kind of a structural "dead-end", which is what data looks like from a structural perspective.

Field name	Field type	Meaning
`text`	ATTR	The content string.

Table 5.2: *The field of a content unit.*

5.2.2 Spatial structure

Due to the restriction that a space unit only can have one parent, and because a subspace is contained in a space unit, it follows that our spatial structures will become trees. Note, that we do not regard content relationships as being part of spatial structure.

We distinguish between two types of spatial structure:

visual structure This constitutes the structure of space units produced by clients of the spatial structure server. The implicit structure that clients wish to express is embedded in it.

explicit structure This is an explicit representation of the implicit structure in the visual structure, as determined by a spatial parser.

How these two kinds of spatial structure are related and used is described next.

Visual structure

What we call visual structure is a tree of space units in which the `kind` field value express the fact that the user created it directly (see Figure 5.2). It is the structure produced by users when they analyse their information spatially. In the absence of a spatial parser, it would be the only structure served by our spatial service.

Explicit structure

Visual structure is augmented by explicit structure to represent explicitly the implicit structure found by a spatial parser. An example is given in Figure 5.3.

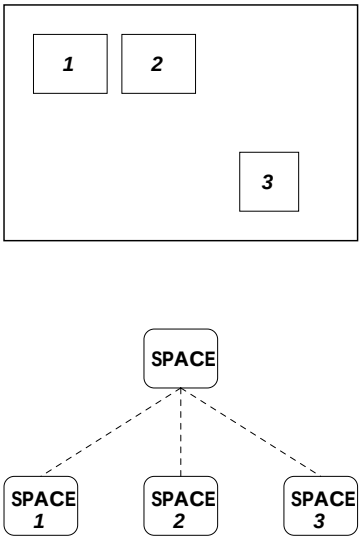

Figure 5.2: *An example space and the visual structure representing it.*

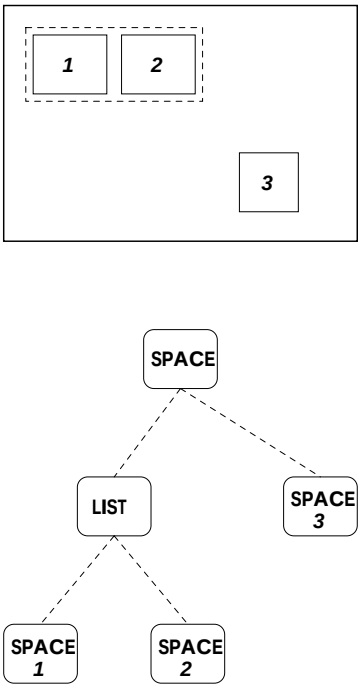

Figure 5.3: *The same example space as in Figure 5.2 and the explicit structure representing it.*

Explicit structure is constructed from visual structure by inserting explicit structure units between visual structure units. Explicit structure units are characterised by having the `kind` field value set to a value representing an implicit structure. This is done in such a way that the original visual structure is contained in the explicit structure. Here, containment means the following: given a space unit u and one of its children v in the visual structure, we say that *if* there is a path $[u, v]$ of units in the visual structure tree, *then* there is a corresponding path $[u, e_0, e_1, \ldots, e_n, v], n \in N_0$ in the explicit structure, where e_i denotes explicit structure units representing implicit structure.

5.3 Spatial server

This section describes the design decisions we have made for our spatial structure server. Firstly, we present the internal architecture of our server, consisting of a structure processor, a cached model, and a behaviour component (representing the spatial parser). Each component of the architecture is shortly described. Secondly, we take a closer look at the behaviour of our spatial parser and the decisions we have made. Thirdly, the collaboration support of our spatial structure server is discussed.

5.3.1 Spatial server architecture

As mentioned in Section 5.1, our spatial hypermedia system is based on a CB-OHS. Therefore, our job is to design a structure server providing spatial services to a set of applications. Previously, Section 3.1.1 addressed the architecture of a structure server consisting of a structure processor and an open set of behaviours. In our case, a structure-cache is introduced for efficiency reasons. The following sections describe the functionalities of the components contained in a structure server.

Structure processor

The structure processor has the task of mediating between generic and domain specific structure units. As mentioned in Section 5.2, it is given generic structure units, each assumed to consist of a set of characteristics, and transforms them to space units and vice versa. The generic unit can contain characteristics not used in the spatial information analysis domain. To enforce information preservation as discussed in Section 4.1.1, we will restrict our server to modify only spatially relevant information. In other words, the foreign characteristics are not modified by the spatial structure server, preserving the foreign properties of the structure unit.

Structure cache

The structure cache is a container for structure, accessible and modifiable by users as well as behaviours. Our cache contains a spatial model (see Section 5.2). It is convenient, because it allows provision of model information without requiring communication with the backend, if the information resides in the cache. If the information is not residing in the cache, it is fetched from the hyperstore and kept in the cache for possible future requests.

In the case of a modification on the cached spatial model, the structure server should support a write-through mechanism to ensure consistency between the persistent structure in the hyperstore and the cached structure in the structure server.

If the persistent model is modified (possibly via another structure server), notifications from the hyperstore ensure that the structure server can take care of the inconsistency between its cache and the persistent structure.

Behaviour

According to the literature [NLS97], behaviours represent computations over structure. This is a very broad definition, covering many application areas. Here we will only consider behaviours working on space units (domain specific spatial structural abstractions). These types of behaviour resemble spatial services, not provided by the structure processor. For instance, the spatial parser represents a behaviour, traversing a model in search for spatial regularities of space units. In this case, the parser even creates new space units, extending the original

model and resulting in a model presenting the spatial parsers view. This view should be accessible for interested parties such as users or other behaviours.

Architectural considerations

The separation of the structure server into three kinds of components (structure processor, structure cache and behaviour) introduces the question of how to design the internals of a structure server.

From a conceptual point of view, it seems as if behaviours can generate whole new structures apart from the original structure it computes over. Technically, this introduces new problems. Consider the situation of multiple behaviours, each generating new structures. This would require requesters (both applications and behaviours) to specify which structure they are interested in, that, in turn, would require the requesters to know what behaviours are supported—is that desirable and/or necessary?

We have decided to use *one* central structure-cache, maintaining an accessible and modifiable spatial model, applicable by all behaviours. In this scenario, a behaviour (e.g., a spatial parser) modifies the given structure by extending or removing parts of it. The only thing that has to be guaranteed is that the resulting structure is of the same kind as the original structure. In our case, both have to consist of space units. Otherwise, behaviours and applications can not be sure of what type of structure is provided by the structure-cache.

Still, the spatial model has to be powerful enough to express all kinds of spatial abstractions. In our case, this is possible by specifying the `kind` field of a space unit, allowing behaviours to differentiate between units.

Another noteworthy issue is that the result of applying behaviours *can* depend on the order in which they are applied. For instance, a behaviour counting lists in the spatial model somehow requires a parse (capable of creating list space units) of the model before traversing it, or no lists will be found. Who is responsible for this kind of ordering? There are three possibilities: the users; the structure server; or, the behaviours themselves. Imagine a behaviour that is dependent on previous computations. In that case, it would be convenient for a behaviour to be able to force required computations to be executed beforehand, introducing relations between behaviours.

In general, the solution to the addressed problems depend heavily on the nature of a behaviour (i.e., is it dependent on other behaviours, does it modify the model, etc.). Another observation shows that all these problems arise when introducing multiple behaviours to the structure server.

In our case, the parser can actually be looked at as a set of behaviours, each designed to find a specific regularity of space units in a space. In that way, we have vertical list behaviours, horizontal list behaviours, etc. (later called experts), that can produce respectively vertical lists, horizontal lists, etc., of space units. This makes the parser look as some kind of scheduler, deciding the order in which the different behaviours are applied.

A more technical issue is the question of what communication mechanisms to use. One way is to let the structure processor, the structure-cache and each behaviour be represented by a process. This, in turn, would lead to a great amount

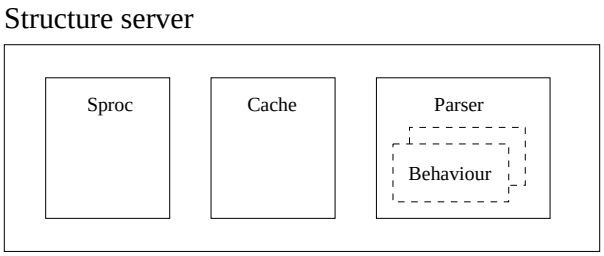

Figure 5.4: *Internal composition of a structure server.*

of inter-process communication (IPC), because of the fact that behaviours work on structure provided by the cache. Another way, is to represent all components as threads (light-weight processes) inside a structure server process [NLSS96]. This would allow for efficient communication between the components, because they all would share a data space. In our case, several behaviours are encapsulated in the parser, presented by a single thread, thereby even avoiding thread synchronisation issues that would arise.

Based on these thoughts about the architecture of structure servers, we have decide to build a spatial structure server consisting of a structure processor, a central structure-cache and a spatial parser (resembling multiple behaviours). The parser should be implemented as a thread in the spatial structure server process, which maintains the cache, while the structure processor provides the server with spatial structure. Figure 5.4 depicts the architecture.

One noteworthy problem in this regard is that our spatial model is restrictive with respect to its expressiveness. The fact that our model only allows space units to have *one* parent makes it impossible for a space unit to be part of multiple "parallel" spatial structures. For instance, the result of a spatial parse cannot express a space unit to be part of a vertical and a horizontal list at the same time. Another limitation is that subsequent behaviour executions can destroy previously found structure. Consider a parser only interpreting horizontal lists followed by a parser only interpreting vertical lists. A space unit located such that it could be part of both, will only be represented in the vertical list, since space units only can refer to one parent (in this case the vertical structure derived). This is not to be confused with nested spatial structures, which our model is designed to express.

5.3.2 Spatial parser

Spatial parsing produces explicit structure by interpreting visual structure. A definition of implicit structure embodied in the parser defines the interpretation. This section will explain the conceptual design that underlies spatial parsing in our server.

In the literature survey, we presented three systems that did spatial parsing. Two of these systems, VIKI and the "adoptive algorithm prototype", presented ideas that we have chosen to design our spatial parser upon. The conceptual framework will mostly be inspired by VIKI.

Structures

The design of a spatial parsing algorithm includes considerations about what aspects to take into account when finding spatial structures. Proximity, alignment, visual similarity, repeated patterns and grouped areas of space units are some of the characteristics that can have influence on the perception of structure. We have chosen to infer structure based solely upon measures of proximity, alignment and extent. Visual similarities (e.g., shape, colour, etc.) do not have influence on decisions made by our parser.

Spatial structures that our parser can recognise are vertical lists, horizontal lists, heaps and matrices. Structures such as lists are type checked in the sense that a list only can contain elements of the same kind. VIKI differs between space units and subspaces. Subspaces are not taken into account when parsing the space for in-space spatial structures. Only space units have influence on explicit structure on a certain hierarchical space level in VIKI. As described in the previous section, we have chosen to model a space unit that refers both to content and a set of children space units (defining its subspace) at the same time. Therefore, it is natural for us to let subspaces (which are part of space units) to be taken into account in the spatial parsing, and thereby be part of explicit spatial structures.

Using space-subspace relations to express hierarchical structure on top of the spatial in-space structures requires us to choose a parse strategy concerning how to deal with hierarchical nested spaces when we parse. We have chosen to do spatial parsing discretely per space, which means that we confine our spatial parser only to find structure inside a current visual structure's subspace, as opposed to finding spatial structures spanning or cross cutting multiple spaces. Explicit spatial structures are inserted into an explicit hierarchical structure (modelling the space-subspace hierarchy structure much like a taxonomy), as described in the previous section that describes the spatial model.

Visual structure feedback

Our system will represent a different approach than VIKI's when it comes to visual feedback of found structures. We will make continuous visual structure feedback reflecting the found explicit structures. Visual feedback is presented in form of visual structure indicators, graphically presented as frames of some colour depending on the found structure and surrounding the structured space units. Visual structure indicators are continuously rendered in the space as the results of the event-triggered incremental parser are received. Given that notifications are sent to spatial clients whenever structure changes occur, it is a relatively simple matter for a spatial editor to maintain visual indicators representing explicit structure based on the notifications received.

Visual feedback makes the user aware of what structures are currently explicitly and thereby persistently stored in the hyperstore. Visual feedback can also help users to maintain a spatial sense for object relationships that are only partially visible in the current window view. Visual structure indicators surrounding a partially visible object will reveal it to be part of a greater structure,

typed by the colour of the indicator.

Parser architecture

Conceptually, we have chosen to design an event triggered bottom-up spatial parser. The parser is event triggered in the sense that the user can cause a parse of the current space simply by manipulating space units. This means that the user does not explicitly ask the systems to perform a parse of some visual structure; rather parsing is a "silent" action, continuously working in the background. The parser is bottom up in the sense that we start out by building simple structures out of space units; repetitive parsing can then build upon both space units, and previously derived explicit structures, to find high-order structures. Repetitive parsing continues until no further structures can be derived according to the structure criteria in the parser.

Conceptually, the structure criteria in the parser is each taken care of by a structure expert. A structure expert is basically a piece of code that is designed especially to find a specific structure type (e.g., we have the notion of a horizontal list expert). Experts are parameterised and can therefore be adjusted to scale some of the criteria by which the expert decides on specific structure (e.g., list experts have parameters defining the maximum distance between objects in the list, and the maximum displacement of objects to still be considered aligned). Structure experts are called in turn by the main parser program. The experts are called in the following order: heap parsing, horizontal list parsing, vertical list parsing and matrix parsing. In our framework, matrices have to be checked for last, because they build upon lists (i.e., a matrix structure can replace a list of lists, if the list can be interpreted as a matrix by the expert). The order in which experts are called corresponds to the order in which experts are called in VIKI's spatial parser. This order was found fairly successful [MS93], although VIKI does not find matrix structures. VIKI's spatial parser includes a composite expert. Composites are useful in spatial settings as a means for users to express repetitive occurrences of higher order structures. Composite parsing is not a part of our design. The notion of experts comes from the algorithm design of VIKI. Structure experts make the spatial parser extensible for new structure experts. One only has to make a new expert component (i.e., a composite expert), and let it be invoked by the main parser program. In the following chapter on our prototypic implementation, the structure experts will be described in more detail.

Parsed implicit structure is stored continuously as general explicit structure in a hyperstore. This means that a conversion from structure server specific explicit structure is done.

Incremental parsing

Our spatial parser is designed to make incremental parses of a space. To motivate this, we have earlier described that explicit structure is stored continuously in a hyperstore. Continuously storing of found structures guarantees persistence of structure, but as a consequence, parsing is also done frequently (e.g., for every

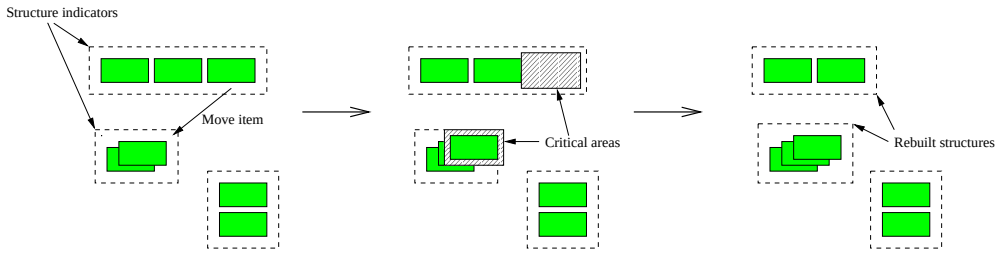

Figure 5.5: *Incremental parse triggered by a move.*

insert, move, resize or delete action on a space unit). Frequent parsing makes the parser a bottleneck when each parse has to be done from scratch every time and may take long time because the space contains large amounts of objects. We therefore aimed at reducing the time to make a single parse. We observed that a complete parse of the space, building the complete explicit structure from scratch repeatedly, contained much redundant work, when the parser was triggered by actions on one object only. We therefore limited reparsing to work only on those parts of the explicit structure in which structure necessarily should be removed and rebuilt from scratch. To realise this, we introduce the concept of a critical area. A critical area is relative to an item that is either inserted, moved, resized or deleted. A critical area is bigger than the area of the item itself; the extent of the area is defined as the area of the original item, expanded in all four directions by the maximum parse parameter for allowed distance between items in spatial structure contexts. This means that a critical area covers that area in which the item in question can affect existing structure. We use critical areas to decide which explicit structures should be removed and rebuilt from scratch. The design of the incremental parser is based on the following observations concerning critical areas (see Figure 5.5):

- *Inserting a space unit*: This causes explicit structure intersecting with the critical area of the space unit to be removed.
- *Moving a space unit*: This causes explicit structure intersecting with the critical areas of the space unit, inferred by the original position and the target position, to be removed.
- *Resizing a space unit*: This causes explicit structure intersecting with the critical area of the resized space unit to be removed.
- *Deleting a space unit*: This causes explicit structure intersecting with the original critical area of the space unit to be removed.

After the explicit structures, intersecting with the critical areas, have been removed, we are left with a set of space units and possibly explicit structure that was previously part of a removed higher order structure. These space units and structures have to be parsed again, but since they possibly constitute only a limited set of all space units, we have reduced the work of the parser to take

a minimum set of space units into account when creating new structure. This makes the spatial parser incremental, as only local areas of explicit structure are changed by each parse. Communicating removed structure and rebuild structure is handled by an event listener that distributes such events among its clients.

5.3.3 Collaboration support

It has been argued [WN98] that CB-OHS systems should include as much generic functionality as possible for supporting cooperative work. The benefit of this would be to streamline and unify the workings of the system, enhancing both system efficiency and the user experience.

Creating a new structure service for a CB-OHS is not a trivial task. Having to consider collaboration support on top of that makes it considerably more complicated, even if collaboration support can be considered a feature that is orthogonal to the structure service. This is so, because adapting a hypermedia domain to the CSCW setting raises nontrivial design issues in itself. In addition, not much of the CSCW functionality required for a domain is inherent to it; thus, having generic collaboration support in the framework will ease the task of supporting cooperativity when making a new structure service component, allowing such efforts to focus on how to adapt the domain to cooperativity.

In the following we will describe the collaboration support mechanisms that our spatial structure service will offer. To what degree we rely on mechanisms to be provided by the CB-OHS framework is also discussed.

Session management

Session management is functionality that ought to be provided in the CB-OHS framework. The exact nature of how it is provided is not crucial. The only assumptions we make is that some connection management facility is present that allows clients to join a session; that sessions include a coupling mode; and, that sessions are dynamic (i.e., clients can join and leave sessions at any time).

COUPLING MODES. The set of coupling modes we plan to provide is described in the following. The messages relating to group awareness functionality are discussed below, in the section on shared user interface management.

- *Lone coupling:* No notifications at all are sent to clients in this session, and clients are not required to send any notifications to the server.

- *Loose coupling:* Clients using this coupling mode are notified about spatial structure changes, and are required to send notifications to support the group awareness functionality.

- *Cooperative coupling:* Clients that are coupled in this mode receive the same notifications as in the loosely coupled mode. In addition, they receive events that allow them to implement a group awareness mechanism, and they are required to send events to the server to support the group awareness functionality.

It is not possible to change the coupling modes of our sessions dynamically.

NEWCOMERS. Handling a newcomer to a cooperative session in general poses the problem of bringing the newcomer up to date with the rest of the participants in the session. In our case, the nature of the communication between a spatial structure service and its clients has been designed in such a way that no special precautions need to be taken—clients act the same independently of whether they arrive first in the session or not. This applies to both shared data management and shared user interface management.

PERSISTENT SESSION INFORMATION. We do not intend to support persistent session information in our prototype.

Shared data management

This section describes how the spatial structure service relies on concurrency control mechanisms in the hyperstore to guarantee basic structure coherency. Also, it describes some mechanisms it implements on its own.

CONCURRENT UPDATES. Conflicting requests for updates are resolved using the transaction management facilities of the hyperstore. The updates will be performed inside (short-lived) transactions that are serialised and performed in some order. The exact order cannot be determined, and generally is not significant to the structure service; however, it can be discerned by examining notifications from the hyperstore, if desired.

LOCKING. Here, we distinguish between automatic locks that are set during short-lived hyperstore transactions, and user-controlled locks that are controlled by the structure service and are used to simulate long-lived transactions on objects.

SHARED LOCKS. We presume that the hyperstore implements sharing of locks, so that objects locked for update can still be read.

LOCK GRANULARITY. The granularity of automatic locking when performing updates should be designed to maximise concurrency of work. Subspaces present a special issue with respect to granularity, in that locking a space unit should not lock its subspace. For example, locking the title of a space unit for update should not prevent anyone from working freely in its subspace.

Since fields of units is the granularity at which the hyperstore can lock objects, this should not be difficult to achieve.

USER-CONTROLLED LOCKING. User-controlled locking is a mechanism provided by the spatial structure service, using primitives in the hyperstore. It gives the client that locks a space unit the exclusive right to modify the unit. Our prototype does not implement user-controlled locking.

Shared user interface management

Our spatial structure service and the accompanying editor does not provide full-fledged CSCW support, since this is a research project in itself. Instead we

focus on providing a simple form of group awareness that allows users to track the presence of other users in a spatial structure.

The mechanism is based on a voluntary protocol in which clients send *enter* and *leave* messages whenever appropriate to the spatial structure service. The structure service in turn sends *entered* and *has left* notifications to clients that use the collaborative coupling mode.

To exemplify the use of the mechanism, think of a spatial editor that can display spatial structure. Whenever a (sub)space becomes visible in the editor it notifies the structure service that it has "entered" the space, and vice versa. The accompanying notifications sent by the structure service can be used by clients to present information to users about who else is present in the same space(s) as themselves. The above definition of when to send enter and leave messages is consistent with the fact that users are only able to modify what is actually visible to them in a spatial editor. Hence, presence in a space is equivalent to being able to modify it.

5.4 Spatial client

This section will introduce general features of the clients to our spatial service component, and the requirements that they must satisfy.

5.4.1 Client view of spatial structure

The spatial structure service presents a space unit to clients as a set of fields, and possibly metadata, as discussed in Section 5.2.1.

If a space unit contains references to other space units (such as those contained in its subspace), then the server encodes these as identifiers before passing it to clients. Thus, the spatial structure service can serve individual space units without serving its containing or contained structure.

5.4.2 Spatial service communication

Connection management

When a spatial client (an application in Figure 5.1) connects to the spatial server, it first of all engages in a connection management dialog with the server, using an appropriate protocol. The dialog results in the client joining a collaboration session, through which it can communicate with the spatial service proper.

Style of communication

The client and the server communicate through message passing. Messages may be requests, responses, or notifications. The set of events passed from the server to the client is determined by the coupling mode of the session in which the client participates.

The event handling capabilities of a spatial client can be summarised as follows:

- *Minimum capabilities:* The least demanding coupling mode (the so-called lonely coupling mode) allows clients to disable all server events, and frees them from sending any events to the server.

- *Standard capabilities:* Most clients will probably choose to use a coupling mode that permits them to be aware of changes to the shared structure. Clients are not required to send any events to the server in this mode.

- *Collaboration capabilities:* A set of events pertain to shared user interface control in collaborative coupling modes (e.g., events to support group awareness). Clients may be required to send events to the server in these modes, and to other clients in the same session.

- *Custom capabilities:* In addition to receiving the events prescribed by the predefined coupling modes, clients can request that *more* events should be sent to it. Among other things, this allows clients to specify completely custom-made coupling modes by using the lonely coupling mode and adding the events required.

Use of space unit identifiers

Identifiers of space units are used in all communication with the server. Requests for retrieval of or updates to space units work on the space unit identified in the request. Event messages from the server reporting updates to space units also use identifiers to specify the affected unit(s).

Synchronisation issues

A kind of race condition can occur between the client and the server (actually, it is between clients), due to the classical problem of synchronising distributed processes. For example, suppose a notification from the server arrives at an editor which tells that a space unit has been deleted. At the same time, the user of the editor is dragging the same space unit. What should the editor do? There are three possible ways of solving the conflict:

- Undo the deletion and allow the user to finish the drag operation, and effect the movement of the space unit.

- Cancel the drag operation, and discard the dragged space unit according to the notification.

- Lock the space unit in the structure service when the drag begins (disallowing other entities to access the space unit), and keep the lock for the duration of the drag operation.

5.4.3 Caching of space units

We expect that most spatial clients will need to cache the structure they work on. Firstly, they may need to maintain a copy of the state of (some part of) a spatial structure in order to do their job at all, (e.g., a spatial editor). Secondly,

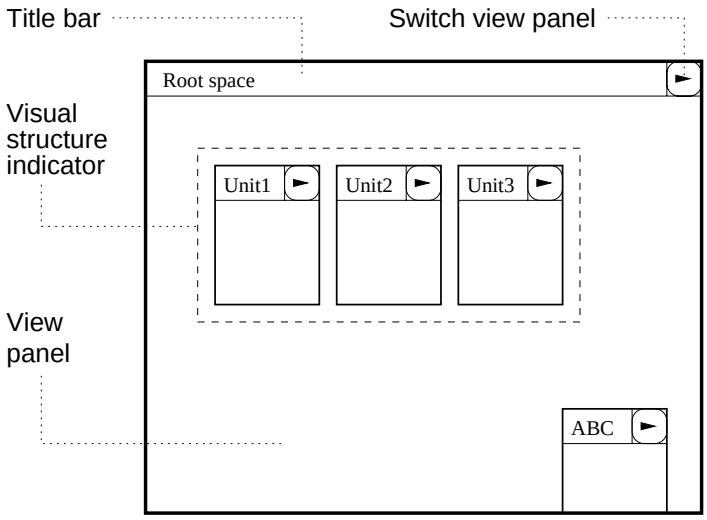

Figure 5.6: *A schematic drawing illustrating how a space unit that displays its subspace view panel is rendered in the editor.*

doing so will increase efficiency, reduce network communication and server load. The set of events from the spatial service that report state changes to space units have been designed so as to enable clients to maintain cache coherency with respect to the spatial structure of the service.

Because we expect caching to be required for most spatial clients, it would be appropriate to design a generic framework to handle such a task. However, we have not done this, because it is outside the scope of our project.

5.4.4 Our spatial editor

Our example spatial client, a spatial editor, abides by the design criteria outlined in this section. This section presents the key functionality we want the editor to support.

Visualisation of spatial structure

To perform visualisation of spatial structure, the editor maps space units to a view unit that is rendered as indicated in Figure 5.6. The view unit resembles a window in a GUI[2], and has some similar features. There is a title bar with control buttons in it (only one in our editor), and an area that shows (a part of) either the content or the subspace of the space unit that the view unit represents. The title is editable. The control button in the title bar switches between panels showing various aspects of a space unit[3]—in our case, its content and its subspace. Attributes like colours, border type, etc., are not illustrated.

[2]Graphical User Interface

[3]Note that this view structure is amenable to the enhanced spatial model presented in Section 8.2.1, in which the "contents" of a space unit can be an arbitrary set of spaces, text, etc.

Visualisation of implicit structure

The editor will visualise the explicit representation of the implicit structure served by the spatial structure service. It will do so in a way that tells what kind of explicit structure unit it is indicating, and which space units the explicit structure unit contains. An simple example of such a visual indicator is given in Figure 5.6, which shows a stippled frame around a group of space units clustered in a horizontal list.

One can imagine a multitude of different ways in which explicit structure can be visualised. Common to all approaches is that they must take into account that explicit structure can be nested. For example, one can have lists of lists. The rendering of a visual indicator around a cluster may depend on whether there is nested structure in it, and the depth of the nesting.

In our design, we chose to render a frame around all explicit structures. The various types of structure found by the parser are indicated by rendering the frames using different colours.

Editing of visual properties and content

Users should be able to set the common visual properties of space units (as discussed in Section 3.2.2). The editor defines a range of values for some characteristics whose domains are an open set in the spatial model. For example, the editor defines a number of frame shapes that are used when rendering space units.

The content of a space unit, in our model, consists of some arbitrary text. The editor should support appropriately sophisticated editing facilities for the content (e.g., a notepad-like editor).

Another part of a space unit that is the open set of characteristics that can be associated with each space unit. This set is due to the fact that we use a CB-OHS architecture where space units are modelled using generalised structure. Even if the editor only makes use of the set of characteristics predefined for space units (Table 5.1), it should permit users to edit other characteristics, too.

Creation and deletion of space units

Users should be able to create and delete space units that are displayed by the editor. Suitable default values for all fields of new space units must be given, in some way. For example, our editor will make the default textual content empty, whereas the initial bounds are defined by user interaction with the visualised spatial structure.

Making virtual copies (see Section 3.2.1) of space units should also be supported. Since only the textual content is shared between virtual copies (Section 5.2), the editor should support modification of visual properties for each individual virtual copy.

Deletion of space units could be implemented very simply in the straightforward way, by discarding them. Or, one may dump deleted space units into a *trash space*, for possible revival later on. This might be particularly useful if the deleted space units have content that might be reused in other contexts.

Editing structural relationships

Users should be able to move space units around, both within a single space, and between spaces. In the first case, the bounds of a space unit are changed, which is likely to cause the explicit structure in the space to change. Such structure changes are to be reflected appropriately by the mechanism that provides visualisation of the explicit structure. In the second case, the operation changes both the parent and the bounds characteristic. This, too, may change the explicit structure in both the source and the destination space.

A mechanism for cutting and pasting space units, corresponding to removal from the source space and insertion into the destination space, should also be provided. If a space unit is copy-pasted, a new space unit should be created that *shares* its content with the space unit copied, but can have its own visual appearance independently of the original space unit, such copies are also called virtual copies..

Navigation of spatial structure

The spatial editor maintains a notion of the "currently viewed space". For example, Figure 5.6 is currently viewing the "Root space".

Users should be able to navigate the spatial structure, i.e., to change the "currently viewed space". It should be possible to dive into a subspace shown on the screen, so that it becomes the currently viewed space. Or, one may go up in the hierarchy, to view the parent of the currently viewed space.

Instantiation of transient explicit structure units

The explicit structure produced by the parser may be considered transient, in the sense that it is persistent, but not permanent. It is stored persistently, but is created and destroyed by the parser outside the direct control of users. Sometimes, users may wish to hold on to a certain explicit structure produced by the parser. We call this "instantiation" of an explicit structure unit, since it is instantiated and becomes permanent in a way similar to creating new space units.

Some properties that may be desirable in instantiated structure units are the following:

1. They are never removed by the parser.

2. The space units contained in an instantiated structure unit become "fixed". They cannot be moved or deleted, since this might disrupt the meaning of the structure unit, in the sense that the parser would have removed the structure unit had it been transient. Also, one cannot add or move other space units so that they overlap the instantiated structure unit, since this may also disrupt the meaning of the structure.

3. They can be part of nested explicit structure, exactly as before—in other words, whether a structure unit has been instantiated or not does not make any difference to the parser when it is scanning for implicit structure.

4. One may assign a title and textual content to instantiated structure units (annotations, explanations, etc.).

We propose two methods for performing instantiation of structure units, given our spatial model.

- One may "lock" the explicit structure unit such that the space units in it cannot be moved, deleted, resized, etc., and so that other space units cannot be positions in such a way that they disrupt the structure internal to the structure unit.

- One may create a new space unit in place of the explicit structure unit and move the space units contained in it into the subspace of the new space unit.

The former method satisfies properties 1, 2, and 3 from the list above. A major drawback of it is the fact that the space units in the instantiated structure become fixed, which is contrary to the idea of allowing free and informal restructuring of all items in spatial hypermedia. The latter method satisfies properties 1 and 4 without incurring the fixation of items. However, property 3 is not satisfied, and this may not be very desirable, nor what was intended, in some situations.

Collaboration support

How should we approach the inclusion of collaboration support in spatial hypermedia? We could start by inventing ways to support CSCW features, implement them, and see how they work. Our argument against this approach is that such attempts might be misguided due to our lack of experience with basic collaboration in spatial hypermedia. Consequently, we have no real sense of what is needed, and the risk of doing futile work is increased.

To illustrate this point, we note that spatial hypermedia emerged as a result of studies of user behaviour when using Aquanet [Mar92], a hypermedia system for organising knowledge. Observation of the actual work patterns of users revealed that the sophisticated features for expressing knowledge structure in the system were not used as much as expected—instead, users mostly employed the simple ability to organise items spatially. In other words, users preferred informal and tentative means of expression in preference to features that forced them to shoehorn their work into a limiting and/or inadequate formalism.

Therefore, our approach to adopting collaboration support for spatial hypermedia is to add it incrementally by iterating over more and more advanced prototypes. Our prototypic spatial editor will provide only basic synchronous editing of shared data, and a simple group awareness mechanism, based on the concept of the "currently viewed space", as discussed above. Studies of how the basic system is used will give us a much better understanding of what the actual requirements are for shared user interfaces for spatial hypermedia.

5.5 Spatial interface

This section defines our proposal for a general spatial hypermedia protocol, based on the spatial model (Section 5.2); the specification of how our spatial structure service operates (Section 5.3); and, the requirements that spatial clients must satisfy (Section 5.4).

The spatial protocol consists of two interfaces: the server interface and the client interface. The server interface defines what operations clients can ask the server to perform, and vice versa with the client interface.

We will not describe protocol message syntax, nor the encoding, but we will focus on the logical requirements of the messages passed via the interfaces. This is done by describing some general prerequisites first:

object naming Naming is the ability to refer to an object using a human-readable name. Our structure service does not allow references to space units by name. If such a reference is required, it is assumed that an external name resolving mechanism can be used to obtain the corresponding unit identifier.

transfer of units When passing a unit as a message argument, it is transferred as a set of fields.

In the following, we will describe the server and client interfaces, given the premises above.

5.5.1 Server interface

The server interface used in our system offers the following operations to clients:

get root space unit A request from a client to send the server's root space unit to the client. There are no message parameters. It causes a `get root space unit response` message to be sent back to the requesting client.

get space unit A request from a client to send the specified space unit. The message parameter is:

- the identifier of the space unit requested.

It causes a `get space unit response` message to be sent back to the requesting client.

add new child A request to add a new space unit to the spatial structure. The message parameters are:

- the identifier of the parent of the new space unit; and,
- the new space unit.

It causes the `new child added` notification message to be sent.

add child A request to add a space unit to another space unit's child-relation, representing a spatial restructuring. The message parameters are:

- the identifier of the parent space unit; and,
- the identifier of the child space unit.

It causes the `child added` notification message to be sent.

remove space unit A request to remove a space unit. The message parameter is:

- the identifier of the space unit that has to be removed.

It causes the `space unit removed` notification message to be sent.

set characteristic A request to set some field of a space unit to a new value. The message parameters are:

- the identifier of the space unit to be changed; and,
- a name-value pair that identifies the field to be changed and the new value(s).

It causes the `has set characteristic` notification message to be sent.

enter A notification from a client to the server that tells which space unit is considered the currently viewed one by the client. The message parameter is:

- the identifier of the space unit entered.

Clients that wish to support group awareness are required to send this message at appropriate times determined by their inner workings.

leave A notification from a client to the server to inform it that it no longer considers the previously announced space unit to be the one currently viewed. The message parameter is:

- the identifier of the space unit that has been left.

Clients that wish to support group awareness are required to send this message at appropriate times determined by their inner workings.

get content unit A request to send the specified content unit. The message parameter is:

- the identifier of the content unit.

It causes the `get content unit response` message to be sent.

add new content unit A request to create and add a content unit to a space unit. The message parameters are:

- the identifier of the space unit; and,
- the new content unit.

It causes the `new content unit added` notification message to be sent.

remove content unit A request to remove a content unit. The message parameter is:

- the identifier of the content unit that has to be removed.

It causes the `content unit removed` notification message to be sent.

modify content unit A request to set some field of a content unit to a new value. The message parameters are:

- the identifier of the content unit to be changed; and,
- a name-value pair that identifies the field to be changed and the new value(s).

It causes the `content unit modified` notification message to be sent.

5.5.2 Client interface

The clients in our system can expect to get the following messages from the server.

get root space unit response A response sent to the client that sent the `create root space unit` request. The message parameter is:

- the server's root space unit.

get space unit response A response sent to the client that sent the `get space unit` request. The message parameter is:

- the space unit requested.

new child added A notification sent to inform clients that a new space unit has been added to the spatial structure. The message parameter is:

- the new space unit (with its parent field set).

child added A notification sent to inform clients that a space unit has a new child-relation to another space unit, representing a spatial restructuring. The message parameters are:

- the identifier of the parent space unit; and,
- the identifier of the child space unit.

space unit removed A notification sent to inform clients that a space unit has been removed permanently from the spatial structure. The message parameter is:

- the identifier of the removed space unit.

has set characteristic A notification sent to inform clients that some field of a space unit has changed. The message parameters are:

- an identifier of the space unit affected; and,
- a name-value pair that identifies the changed field and its new value(s).

has entered A notification from the server telling when another client has selected a new currently viewed space unit. The message parameters are:

- the identifier of the space unit entered; and,
- the name of the user that entered the space unit.

This notification forms the basis for providing a group awareness mechanism in clients.

has left A notification from the server telling when another client no longer considers a particular space as being viewed. The message parameters are:

- the identifier of the space unit left; and,
- the name of the user that left the space unit.

This notification forms the basis for providing a group awareness mechanism in clients.

get content unit response A response sent to the client that sent the `get content unit` request. The message parameter is:

- the content unit requested.

new content unit added A notification sent to inform clients that a new content unit has been added to a space unit. The message parameters are:

- the identifier of the space unit; and,
- the new content unit.

content unit removed A notification sent to inform clients that a content unit has been removed permanently. The message parameter is:

- the identifier of the removed content unit.

content unit modified A notification sent to inform clients that some field of a content unit has changed. The message parameters are:

- the identifier of the content unit affected; and,
- the name-value pair that identifies the changed field and its new value(s).

5.6 Summary

This chapter has introduced the conceptual design upon which we have built our prototypical implementation upon. This was done by outlining the design issues concerning the spatial structure server and a spatial client as part of a CB-OHS. Based on that, we presented the spatial model, the spatial server architecture, a spatial behaviour (the parser), a spatial client and the interface of server and client.

In the next chapter, this conceptual design guides the implementation of a prototype of a cooperative open spatial structure server and a spatial editor, comprising a cooperative open spatial hypermedia system in a CB-OHS framework.

Chapter 6

Prototypic Implementation

This chapter presents our prototypic implementation of a cooperative open spatial hypermedia system. Section 6.1 presents the Construct framework, on which our project work is based. The architecture of the framework is presented, along with a description of what it provides for us. Section 6.2 describes the implemented spatial model. Section 6.3 is about our implementation of a spatial structure service component for use with the framework. Section 6.4 describes our sample client of the spatial structure service, a spatial editor. Finally, Section 6.5 presents our implementation of the spatial protocol.

6.1 Open hypermedia architecture

As described in the previous chapter, we have designed a spatial structure server as part of a CB-OHS. With respect to the implementation of a CB-OHS, the enormous flexibility and tailorability is offset by a complex infrastructure. For this reason, it is essential and convenient that the implementation of the infrastructure can be, and is, implemented in a standardised way, resulting in a *one-time-only* implementation.

This section describes an implementation of a public domain CB-OHS compliant with OHSWG (de facto) standards, reference models and reference architectures, called Construct.

6.1.1 Construct

Construct implements both a framework supporting communication between the components of a CB-OHS as well as explicit implementations of components, or, in other words: the infrastructure[1]. This makes Construct a good code base for open hypermedia system developers, by providing an off-the-shelf framework [WN99], extensible and tailorable to one's needs. We have extended Construct with a spatial structure server and a spatial client, by relying on the communication framework and the hyperstore.

[1] As of November 1998, Construct consists of a communication framework, a hyperstore, a navigational structure server and a set of enabled applications.

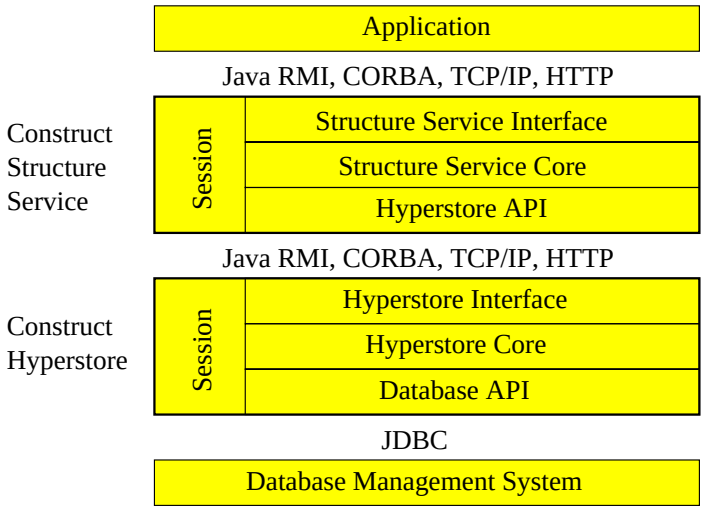

Figure 6.1: *The architecture of Construct.*

Architecture

The architecture of Construct is based on the general CB-OHS framework depicted in Figure 5.1. In Construct, both the storage component (hyperstore) and the structure server component consist of multiple process sessions (not to be confused with either communication or collaboration sessions), each of which is further separated into three sublayers of functionality (see Figure 6.1).

The hyperstore is implemented on top of a database management system (DBMS). The database API[2] provides basic storage and retrieval services to the hyperstore core by using JDBC[3] to communicate with the DBMS. The hyperstore core implements both generic hypermedia services (e.g., objects, relations, and behaviours) and generic collaboration services (e.g., nested transactions, concurrency control, notification control, access control, and version control). All these services are provided to the structure servers through the hyperstore interface.

Structure servers consist of the same layers as the hyperstore: the client library layer (called hyperstore API), the core layer (called structure service core), and the server interface layer (called structure service interface). The hyperstore API provides the services of the hyperstore to the structure service core (resembling some kind of a RPC client stub). The core tailors and extends the generic facilities provided by the hyperstore to structure server specific services. The structure server interface provides these domain specific services to applications (both custom and legacy made).

Construct[4] supports various standardised encoding formats for communica-

[2] Application Programming Interface
[3] Java Database Connection
[4] As of November 1998, Construct supports Java RMI, TCP/IP and HTTP

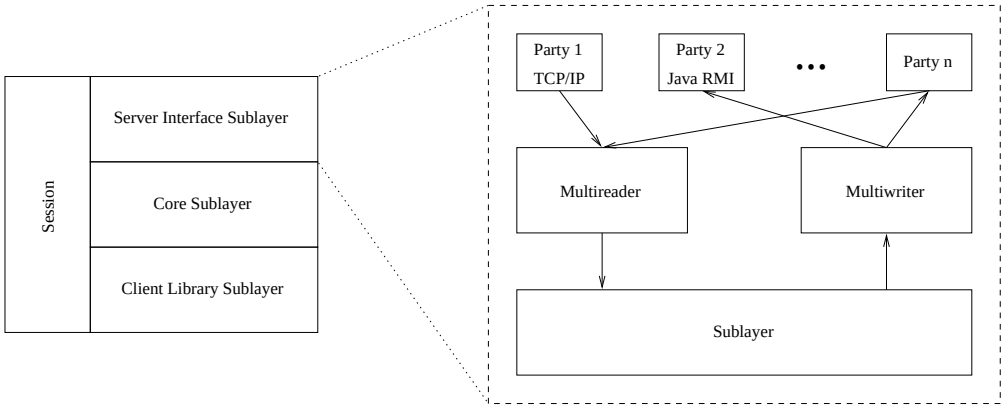

Figure 6.2: *Construct components architecture.*

tion between the components in the architecture, such as Java RMI[5], CORBA[6], TCP[7]/IP[8] or HTTP[9].

Sublayers

Each of the three sublayers making up a component is further divided into a multireader, a multiwriter and a set of parties (see Figure 6.2). A party represents a connection to another component. The multireader takes care of incoming requests or notifications, which are passed to the sublayer for further processing (typically forwarding them to the core). The multiwriter handles the opposite direction of the communication by distributing messages to several parties. The infrastructure facilities provided are used by the parties to send messages to other components. By encapsulating the parties into the multireader and multiwriter, the parties are viewed as one logical party pool to the sublayer. This makes the distribution of the system components transparent (including specific encoding facilities).

Hyperstore

As mentioned in Section 5.1, the hyperstore services are based on generic structure. In Construct, the generic structure element is called a *unit*. It is basically an arbitrary set of characteristics and a set of metadata (e.g., creation date, permissions, etc.). Characteristics are sets of values. Each set is either a set of attributes (strings), behaviours (identifiers of computations), or relationships (identifiers of other units).

As an additional feature, the Construct hyperstore allows assignment of types to units. This allows the hyperstore to provide some kind of common

[5]Remote Method Invocation
[6]Common Object Request Broker Architecture
[7]Transmission Control Protocol
[8]Internet Protocol
[9]Hypertext Transfer Protocol

tailored API according to the type of units. For example, a unit typed "link" could be accessed through the link-unit-API, allowing services such as "addEndpoint" that even could check for integrity. The provision of common tailored APIs offloads the structure server task of tailoring domain specific structural abstractions.

6.1.2 Our components

As mentioned earlier, we have implemented a spatial structure server and a spatial client. Both use the Construct component architecture (Figure 6.2). With respect to the communication mechanisms, we have decided to use TCP/IP, which is supported by Construct transferring XML representations of messages.

6.1.3 Summary

In this section, we have introduced the Construct CB-OHS, consisting of a hyperstore and a framework, complying with OHSWG standards, reference architectures and models. We have introduced the generic storage unit, the internal structure of components and how components communicate with each other.

The following sections will base their work on these given entities and allow us to implement a spatial structure server and a spatial client as part of the Construct CB-OHS.

6.2 Spatial model

The spatial model is implemented as a general class that defines a general space unit. This space unit class acts as a container for characteristics defined for space units. In accordance with the model presented in Section 5.2, it implements the following characteristics: `parent`, `children`, `content`, `kind`, `title`, `bounds`, `bg`, `border`, `shape` and `thickness`. Characteristics that refer to other parties are implemented as referring directly to other objects of the appropriate type. One exception to this is the `children` characteristic. It has the ability to refer to either another space unit directly or just its `id`. This allows space units to be instantiated without requiring all its children to be instantiated or, said in another way, it allows a space unit's children first to be fetched when actually used, resembling some kind of lazy evaluation. Appropriate accessor methods are also provided by the general space unit class. In addition, the class provides event subscription mechanisms. These are intended to be used by specialisations of the general space unit allowing the emission of events in case certain state changes of a space unit occur. The events are then sent to all its subscribers, acknowledging them of a state change on a specific space unit. Additional user defined characteristics are not supported by our prototype yet.

As mentioned above, this general class can be specialised for use in any context where processing of spatial structure is required (e.g., in our spatial server and spatial clients). The specialisations we use in the spatial server and the spatial editor are described in Sections 6.3.1 and 6.4.2.

6.3 Spatial server

This section describes our implementation of a prototypic spatial server, based on the conceptual design presented in Section 5.3. The task of a spatial structure server is to mediate between generic and spatial structure. The generic structure is used to store the spatial structure persistently and in a format common for all structure servers. The spatial structure is provided to spatial tools, allowing them to work with conceptually appropriate structural abstractions. Our spatial structure server adds a behaviour to all this, allowing implicit spatial structures to be recognised. Additionally, our server implements basic collaboration support, allowing multiple spatial applications to share a space.

6.3.1 Spatial server model

The spatial server model extends the general spatial model described above (Section 6.2). It incorporates server relevant information into the space unit such as access to the structure cache and the ability to trigger a parse of the (sub-)space in which the unit is contained.

The methods that are responsible for the triggering of the parser are obviously the ones that modify a space unit's position, extent or the relation to its subspace (i.e., `setBounds`, `addNewChild`, `addChild`, and `removeSpaceUnit`).

6.3.2 Spatial server architecture

As mentioned in Section 6.1.1, our spatial server implementation is based on the Construct component architecture (see Figure 6.2). This means, that a spatial server session is divided into three sublayers:

- the spatial service API;
- the spatial service core; and,
- the hyperstore client library.

The spatial service core encapsulates the functionality of the server, while the other sublayers mainly concern communication management aspects.

The spatial server core

To describe our spatial server, we consider one of Construct's process sessions that resembles some kind of generic session "building block", extensible to implement both communication and collaboration sessions. Such a session consists of the spatial server interface, Construct's hyperstore client library and the spatial server core described here. The server interface and the client library hide network communication specific tasks, which allows the server core to operate as if it had direct access to its clients.

In addition to Construct's division of the structure server into sublayers, our spatial core is further divided into three different sub-components: a structure processor, a structure-cache and the spatial parser. This was already addressed in the conceptual design of our spatial structure server (see Section 5.3.1).

With respect to the conceptual design of the internals of a spatial structure server (encapsulated by the core in Construct), our implementation is somewhat simplified. The core consists of one structure-cache, the spatial parser and a set of methods amounting to the functionality of the structure processor.

In fact, the current implementation of our server supports only one spatial document (in our architecture resembled by one process session). The reasons for this decision are: firstly, because the provided naming facilities of Construct have not been implemented yet; and, secondly, because this limitation does not restrict the possibility to verify most of our assumptions about collaborative spatial hypermedia. Users that work in their own space, can be simulated by letting them work in separate subspaces of the common spatial document root (called the root space). This is similar to the file-system paradigm.

Structure processor

As mentioned above, the structure processor is presented by a set of methods that ensure that all units in the core are of the right domain specific type (in our case, space units). Conversely, the methods are used to ensure the right object type when communicating with the hyperstore (in Construct's case, units). In other words, the structure processor translates (see Section 4.1.1) generic structure to space units and vice versa.

Structure-cache

In our implementation, the structure-cache represents the central entity providing existing space units. All entities requesting space units request them through the cache. If the cache does not contain the requested space unit, it automatically forwards the request to the hyperstore. On reception, the space unit is registered in the cache and delivered to the original requester. Future requests to that object are now handled locally in the core, without the involvement of the hyperstore. As mentioned in the conceptual design of our structure-cache, we have implemented a write-through cache, ensuring consistency between the hyperstore and the server cache. So far, the size of the cache is limited only by the operating system, possibly resulting in containing the whole spatial model connected to a spatial server.

Parser

Our parser is tightly coupled to our current spatial model. This is characterised by the fact that our server model knows about the existence of our parser (see Section 6.3.1). A detailed description of how the parser operates is presented below.

Chain of events

Because we have decided to store all structure persistently at all times, we have to take consistency issues into account. This means that our structure server must ensure consistency between its cache and the hyperstore. For this

reason, we have implemented the core in a way that applies changes first to the hyperstore. When accepted by the hyperstore, the changes are applied to the local model (residing in the cache). This procedure guarantees that the server model always is a reflection of the model stored in the hyperstore.

When introducing a parser that works directly on the model in the spatial server, we have to consider new consistency issues. For instance, the task of parsing a space includes removal and creation of temporary structure. The temporary structure will obviously *not* be stored persistently, while the opposite is true for the final structure with which the parser ends up. It is this final structure that is distributed to clients, presenting the parser's (and thereby hopefully the user's) view of the implicit structure of the parsed space. We have to implement the parsing process in a way that guarantees the consistency of the server cache and the hyperstore right before and right after the parsing. This is done in the following way:

- A hyperstore transaction is started.
- The parser starts by requesting the hyperstore to remove explicit structure units that have become invalid.
- The parser parses the space until no new explicit structures can be found.
- The parser requests the hyperstore to create the new explicit structures found; and, to update existing visual structure units affected by the new structure.
- The hyperstore transaction is committed.

There are now two possible situations: the hyperstore accepts the commit and thereby the new parsed structure; or, the hyperstore aborts the transaction, requiring the changes made to the structure cache to be undone.

In the former case, the new structure has to be distributed to the users, updating their view. To be able to do this, all requests sent to the hyperstore have to be recorded at parse-time so that it is possible to "playback" the changes performed on the spatial structure after the commit has been accepted.

In the latter case, the parser can undo its changes by clearing the server cache. This is a bit of a drastic reaction, but it is easy and effective. Following requests for space units for the cache will result in the reception of "old" (in the sense of "before the parser started") structure units from the hyperstore, since no units are residing in the servers cache. Thereby the cache is gradually updated, finally reflecting the persistent structure again.

This mechanism allows us to simulate a parse as an atomic operation as seen from the users. That is, there do not exist any structural states between the state right before the parser started and the state right after the parser stopped.

6.3.3 Spatial parser

We have decided to implement a spatial behaviour in form of an incremental spatial parser as part of our server. Different techniques upon which spatial parsing can be based were reviewed in the literature survey (see Section 3.2.3).

As described in the conceptual framework chapter, our spatial parser consists of a set of experts (see Section 5.3.2). Each expert is specialised to find specific spatial structure among a set of space units, according to some parameterised criteria. For each triggered incremental parse, a limited set of space units consisting of both structure units and ordinary space units are temporarily moved from the original space, in which an insert, move, resize or delete action of a space unit occurred, to a temporary separate space. These space units have to be reparsed because they either intersect or are part of structures that intersects with the critical area of the space unit that triggered the parse (see Section 5.3.2 for an introduction to this). The temporary space is now repeatedly parsed by the different experts, until no expert can derive any new structure from the space. The resulting structure is then merged into the original space again.

Our parser focuses on finding structure based on proximity and alignment of space units. This means that we have to clarify the concepts of proximity and alignment with which our experts work.

proximity Conceptually, implicit spatial structures based on relatively large objects allow a bigger inter-object distance than relatively small objects. On the other hand, this heuristic is difficult to formalise in an expert without running the risk of introducing strange side effects. An example of this is VIKI, where local proximity can depend on all objects extents. The side effect is that changing the extent of an object that spatially resides far away from some structured objects can influence the parser's decision of whether the structured objects shall remain in that structure or not. We have decided that the measure for proximity in our parser simply is the absolute distance between space units. The critical values for how far apart objects can be from each other in a two dimensional space to still be conceived as structure are parameterised with one parameter for each dimension. This represents an alternative to VIKI's measure of proximity.

alignment Alignment can be expressed in more than one way. A set of different sized rectangle objects can for example be aligned horizontally by their upper or lower edges or their centre. If the shapes of the objects are circles, triangles, diamonds, or other shapes, we might prefer to align them spatially by other criteria. We have decided that our measure of alignment originates from the upper left corner of a space unit's bounding box. This means that horizontal alignment is decided by defining a maximum acceptable variance in the vertical direction for the upper left corner of the space units in a horizontally aligned structure context. Vertical alignment is decided similarly by defining a maximum acceptable variance in the horizontal direction for the upper left corner of the space units in a vertical aligned structure context. The measures for alignment variance in the two dimensions are parameterised and used by the structure experts.

In the following, the different structure experts will be described.

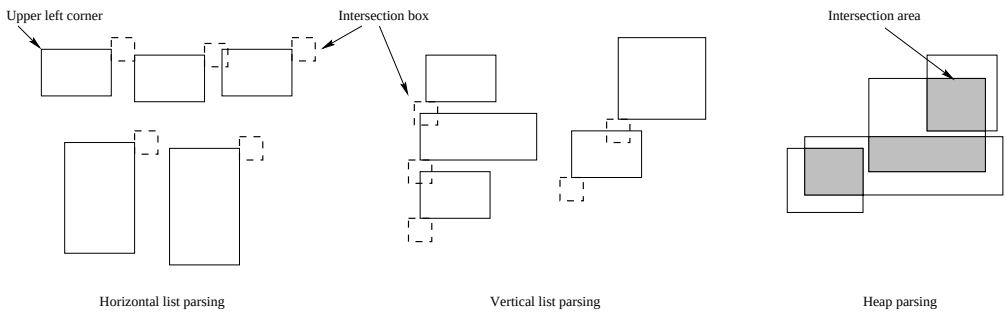

Figure 6.3: *Spatial parsing.*

List parsing

We have two list experts in our spatial parser. A vertical list expert and a horizontal list expert. They are essentially implemented the same way, except for small differences that make them parse for lists in different directions. We will explain how we implemented the horizontal list expert, and afterwards explain how the vertical list expert differs from the horizontal.

- The horizontal list expert starts by sorting all space units in a space according to the x coordinate of their upper left corner. This gives an order which is appropriate for iterating from left to right over the space units.
- The expert assigns an intersection box to all the space units (see Figure 6.3). A space unit's assigned intersection box is then checked for intersections with the upper left corner of other space units. If an intersection occurs, the two space units make up a list. The extent of the intersection box is based on the horizontal and vertical proximity parameters mentioned earlier.
- The expert iterates through the ordered space units, so that space units are taken left to right. If an alignment is detected between two space units by means of the intersection box, the expert builds a list starting with these two space units. Horizontal lists are built "breadth first", which means that the expert builds the list as long as possible, and marks space units found to be in the list, before returning to the original iteration over the ordered space units (skipping marked space units).

The vertical list expert functions do basically the same as the horizontal list expert, except that the vertical expert sorts space units by the y coordinate of the upper left corner of the space unit, instead of the x coordinate, also, intersection boxes are placed differently in the vertical expert (see Figure 6.3).

Heap parsing

The heap expert finds heap structures by checking if space units overlap with a certain percentage of their extent. The expert works as follows:

- The heap expert sorts space units according to their x coordinate, just like the horizontal list expert.

- The parser checks all combinations of any two space units to see if they overlap. If they do, a value for the area that overlaps, in percentage of the space units extents, is calculated for both space units. If one of the values exceeds a parameterised value (e.g., 50 %) the two space units comprise a heap (see Figure 6.3). The requirement for an extra space unit to join the heap is that one of the space units already in the heap has to overlap with the new space unit and at the same time satisfy the overlap percentage requirement just described. When a heap is found, the strategy is to make it as big as possible before the expert continues with the originally iteration to look for new heaps.

Matrix parsing

The matrix expert converts lists of lists into representations of matrices. This means that the matrix expert has to be called after the list experts have done their work. The matrix expert works as follows:

- Iterate through all space units, and look for potential matrices. A potential matrix is either represented as a vertical list of horizontal lists containing the same number of space units, or as a horizontal list of vertical lists containing the same amount of space units.

- Found potential matrices are reparsed to check whether all potential matrix elements contained in the list of lists structure are aligned correctly. The matrix expert does this by checking whether the elements that were parsed as a vertical list of horizontal lists can also be parsed as a horizontal list of vertical lists. This is done by applying the list experts appropriately on the potential matrix elements. This also applies to the symmetric case of a horizontal list of vertical lists.

- If potential matrices are found to align both vertically and horizontally, the list of lists structure is exchanged with a space unit that represent a matrix structure. The matrix space unit is then added to the original space.

Matrices build on lists. Lists contain at least two space units to be represented as explicit structure. Therefore, this method finds $n \times m$ matrices, where $n, m \geq 2$.

Incremental parsing

As described in the conceptual framework (see Section 5.3.2), the incremental parser is triggered by insert, move, resize or delete actions on a space unit. The incremental parser minimises the set of space units (structure units or ordinary space units) that are influenced by the triggering action, and therefore have to be reparsed.

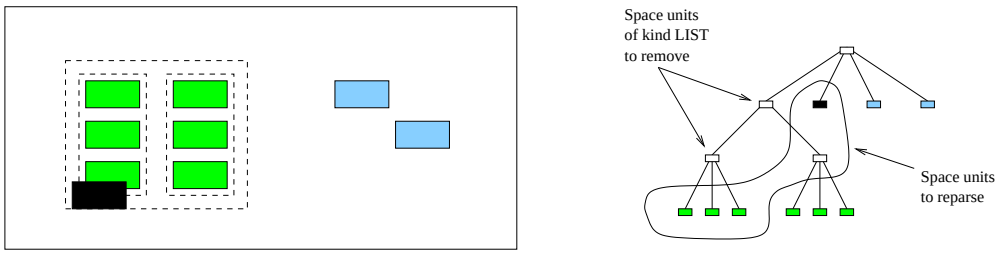

Figure 6.4: *Incremental spatial parsing.*

Keeping structure explicitly represented in the spatial model enables the parser to reparse only parts of the whole visual structure each time local modifications happen to the visual structure. Storing explicit structure in the hyperstore also enables us to avoid total reparses of the visual structure between sessions.

We have already explained how structure is removed, based on the critical area of a space unit (see Section 5.3.2). This section will elaborate on structure removal and creation and how structure modifications are communicated to the clients sharing the space.

THE INCREMENTAL PARSING ALGORITHM. One task of the incremental spatial parser algorithm is to limit the set of space units that has to be reparsed. This is done by traversing a spatial structure subtree, starting from the "nearest" visual space unit that is a (grand-)father of the parse triggering space unit. While traversing the subtree, each space unit (visual or explicit) is checked for intersection with the critical area. If a visual space unit intersects, it is added to the reparse-space. If an explicit structure unit intersects, it is removed and all its children are checked for intersection with the critical area. In the case of no intersection, the space unit (no matter if visual or explicit) is added to the reparse-space. Note that if we add an explicit structure unit, none of its children will be considered by the parser when reparsing, because the parser only takes the explicit structure into account. This is what saves us time.

The principles behind the algorithm are illustrated by Figure 6.4. To the left, the figure shows a space in which a black space unit is placed on top of structure indicators for an existing explicit structure. The internal representation of the space is shown in the right part of the figure. The triggering space unit is black. The nearest visual space unit of the black space unit is represented by the root of the tree representation of the depicted space. The figure has marked the explicit structure units that are going to be removed and the space units (both visual and explicit) that are going to be moved to the temporary reparse-space and reparsed. A parse of the temporary space is finally made, and the resulting structure is merged with the original space.

Based on the removed and the created structure, the parser generates an appropriate set of events that are forwarded to the clients (by the core) telling them how to update their structure.

TIME CONSIDERATIONS. The work of the incremental parser is to minimise the number of space units which have to be reparsed. Based on the description of our incremental spatial parser algorithm above, we can observe that the parser does not save any time, with respect to traditional parsers, as long as there are no high-order (explicit) structures or all high-order structures intersect with the critical area. If only one explicit structure unit does not intersect with the critical area, we save time with respect to traditional parsers that take account for all visual space units in the current space. This means that in the worst case will our parser be just as fast as traditional parsers (considering all space units in a space), but in the case of existing non-affected explicit structure units our parser will outperform traditionally parsers, because it takes fewer space units into account when reparsing,

6.3.4 Collaboration support

This section describes the collaboration support of our structure service. It is a continuation of the points made in Section 5.3.3, in that we discuss the implementation decisions in our prototype based on the conceptual design.

Session management

In our project, the version of the Construct framework that we used was not complete at the time of implementing our structure service—specifically, connection and session management were not available. To illustrate our ideas and the validity of our design, we decided to implement simple connection and session management facilities in the structure service.

PREDEFINED SESSIONS. For reasons of simplicity, our structure service provides three predefined sessions—this obviates the need for us to implement creation, deletion, and browsing of sessions. Clients may join and leave sessions at arbitrary times (i.e., sessions are dynamic).

In Section 3.3.1, a session was defined as a group of users/clients; the shared object(s) worked on; and, a specification of the mode of cooperativity. In our prototype, all sessions share the same object, namely, the root space unit of the structure service. They differ by the coupling modes they operate in—each session supports one of the three coupling modes listed in Section 5.3.3. Thus, we can talk about "the lonely coupled session", etc.

- *Lonely coupled session:* No notifications are sent to clients in this session, and clients are not required to send any notifications.

- *Loosely coupled session:* Events about spatial structure changes are sent to clients in this session. Clients are required to send events to support the group awareness functionality.

- *Cooperative session:* Clients in this session receive the same notifications as in the loosely coupled session. In addition, they receive events that allow clients to implement a group awareness mechanism, and they are required to send events to the server to support the group awareness functionality.

COUPLING MODES. Once a client joins a session, the associated coupling mode determines what events it will receive for the duration of the connection. We do not support transitions between coupling modes. Therefore, clients cannot switch coupling mode dynamically, nor can they switch to use another session without disconnecting from the one in use.

CONNECTION MANAGEMENT. Our connection management protocol becomes very simple due to the session management mechanism described above. When clients connect to the structure service, all they can do is to select which session to join from the predefined set above. If the request is granted, all further communication uses the spatial protocol; otherwise, the connection is terminated. Leaving a session is implicit on closing the connection.

Shared data management

This section describes how our spatial structure server handles concurrency issues and ensures consistency between its cache and its clients.

CONCURRENT UPDATES AND LOCKING. Since spatial services can be implemented by performing several sub-tasks, such as our parser, we have to consider how to handle concurrency issues. For example, how should the server act if requested for a space unit while the parser is in the middle of a parse? In that case, the structure-cache cannot be guaranteed to be consistent.

In Section 5.3.3 we described how the structure service uses the hyperstore to resolve concurrent update conflicts. Instead of using transaction management facilities of the hyperstore we have decided to solve this problem by requiring the thread requesting a service of the server to acquire a lock on the whole core. This includes the cache and the parser, guaranteeing that no two users can access the core simultaneously. This will solve the problem of a thread accessing the cache in an inconsistent state (while parsing), but it still leaves the question of what to do when the new consistent state resembles a state in which a waiting thread's request is invalid (e.g., a space unit that has been deleted is requested). In that case we require the server to send an error message, informing the requesting client of an invalid request. In other words, we have not implemented any explicit mechanisms to handle this, but rely on optimistic concurrency control.

Shared user interface management

As mentioned in the conceptual design, we focus on providing a simple form of group awareness that allows clients to track the presence of other clients.

The mechanism is based on clients sending *enter* and *leave* messages whenever appropriate to the spatial structure service. In our prototype, the notification is based on the concept of "the currently viewed space" that was introduced in Section 5.4.4. Whenever a space becomes the currently viewed space, it is "entered". Whenever the currently viewed space changes, the old one is left, and the new one entered. Thus, a user is able to see which other users are currently

viewing the same space as himself. Note that this is different from the conceptual design section, which stated that users were present in all the (sub-)spaces they could modify, while this implementation only considers users to be present in "the currently viewed space".

6.3.5 Summary

This section presented the implementation of our spatial server. It described how the spatial server core implements an incremental parser behaviour and a structure-cache. The incremental parser was examined in terms of its different expert components, and the incremental strategy was explained. Finally, the collaboration support of the server was described.

6.4 Example client: a spatial editor

This section describes the implementation of our sample spatial client—a spatial editor. It is designed to demonstrate the features of our spatial structure server in the following ways:

- It can visualise a spatial structure, using the `bounds`, `title` and `bg` visual properties available in the spatial model.
- It renders visual indicators that show users the explicit structure derived by the spatial parser.
- It can employ a simple session management protocol to select one of several sessions to join in the structure service. The session selected runs in some coupling mode, which determines the set of possible types of notification that can get sent to the client.
- It reacts to notifications received from the structure service that support collaboration. This includes notifications dealing with shared data management, to provide synchronous editing of shared spaces, and shared user interface management, to provide group interface support.
- It provides a sample implementation of how a group awareness mechanism can be implemented for spatial hypermedia.

Like the rest of the system, the spatial editor is implemented in Java[10]. The Java Foundation Classes (JFC) are used to provide and manage the GUI.

6.4.1 Process architecture

The editor is a Java application. Its overall architecture is shown in Figure 6.5. The application framework provides an environment within which the actual spatial editor lives and connections to spatial structure services operate. For

[10] Java Development Kit 1.2

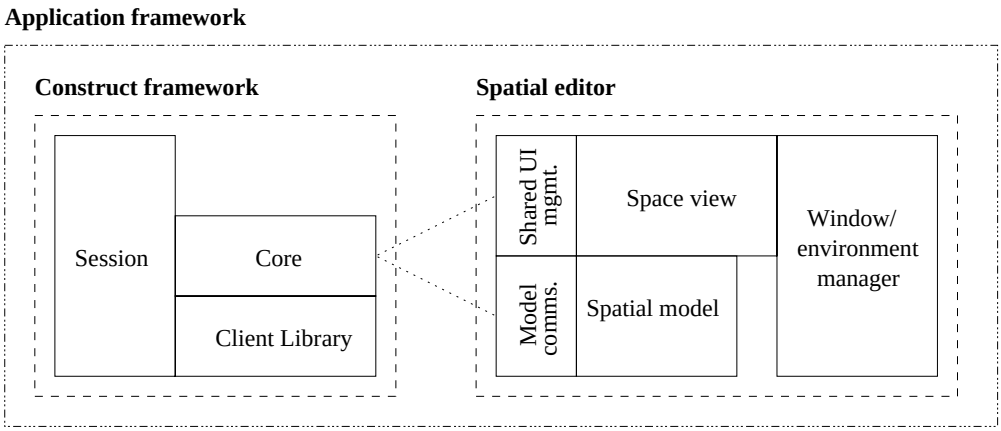

Figure 6.5: *The process architecture of the spatial editor.*

example, it is responsible for opening the main window on start-up. It also provides a menu bar, a status bar, etc., and it coordinates the interaction between these common GUI elements and the spatial editor.

The Construct communication framework (Figure 6.2) is used to perform communication with the spatial structure service. As can be seen, the editor does not implement the Server Interface layer, for obvious reasons. It supports a connection to a single session in a spatial structure service; thus, there is never more than one instance of the session object at a time.

The design of the spatial editor is based on the Model-Control-View design pattern. The spatial model is a cached copy of the spatial structure being visualised by the editor. On top of the model, the control and the view are unified in a common block. It handles rendering of the model, and the interaction between the user and the model.

6.4.2 Spatial model

This section describes the role of the local spatial model in Figure 6.5, which serves two purposes. Firstly, it is a cache of spatial structure received from the server. Secondly, it insulates the spatial editor from the server by passing all communication between the two through the local model. These two aspects are discussed in more detail below.

Local caching of spatial structure

The editor caches information about space units locally. The objects in the cache are descendants of the general space unit class described in Section 6.2. The local space unit class is an adaptor class that provides easy and fast programmatic access to fields in the space unit from the editor, which is useful when rendering and manipulating the visualisation of spatial structure.

Server communication flows

Communication about model changes is split into two decoupled flows of information. One flow is from the user requesting modifications to the model stored in the server. The other flow is from the server notifying its clients about state changes in the model. The two flows are described in the following.

SERVER NOTIFICATIONS. When a notification is received from the server telling the client that some part of the model has changed, it is given from the core to the model communication handler. It looks up the local representation of the affected structure unit, and applies the change to it. The change in the local model causes an event to be fired, using the property change event mechanism mentioned in Section 6.2. The spatial view component subscribes to such events, and the reception of one causes the view to update the visualisation of the spatial structure according to the event received.

USER MODIFICATIONS. When the user requests a modification of the visualised spatial structure, the request is forwarded to the model as a request to change one of its fields. The modification is forwarded by the model to the model communication component, which in turn forwards it to the core, and then on to the server.

Thus, we see that user modifications do not cause immediate changes to the local model. This happens when the server accepts the modification request and responds to it. The reception of the response triggers and updates both the model and the visualisation of the spatial structure, according to the description above.

6.4.3 Graphical user interface

Screen shot

Figure 6.6 shows a screen shot of our spatial editor. It illustrates the following features of our system:

- *Visualisation of spatial structure:* The editor visualises spatial structure according to the guidelines presented in Section 5.4.4, and according to Figure 5.6. Recursive visualisation of subspaces is also performed.
- *Unified space unit model:* The two space units called "Unit 12" and "Unit 3" show that the underlying spatial model does not distinguish between units and subspaces. In the example, the subspace of unit 12 is shown, whereas the content of unit 3 is shown.
- *Visual feedback of explicit structure:* Units 3 and 12 form a horizontal list, and have been marked as such by the editor, using a rectangle that surrounds the two units. In the subspace of unit 12, there is yet another horizontal list.

The menu bar above the space view, and the status bar below it, are provided by the application framework indicated in Figure 6.5.

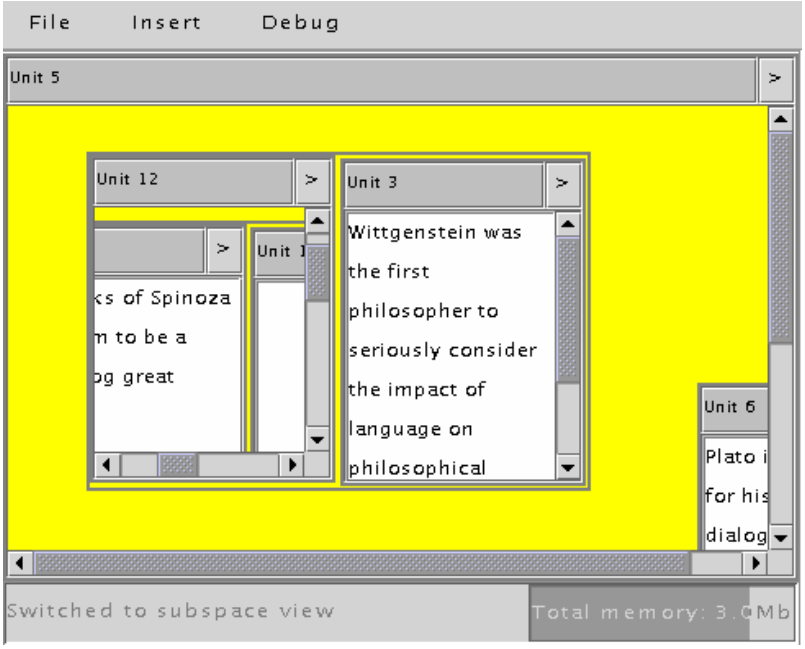

Figure 6.6: *A screen shot of the spatial editor.*

6.4.4 Collaboration support

Shared data management

Shared data management is handled by the concurrency control mechanisms in the spatial structure service (Section 6.3). The spatial editor never updates its model directly, as described in Section 6.4.2. It always sends requests for updates to the server, which responds with a change notification if the update succeeded. Thus, shared data consistency is guaranteed by giving the server control of the one valid copy of the spatial structure, to which all clients adhere.

The race condition between clients mentioned in Section 5.4.2 is dealt with in our editor by adopting the simplest solution, in terms of the number of entities involved. If an incoming notification makes some ongoing operation in the client invalid, the operation is cancelled and the notification is effected.

Shared user interface management

Our implementation of collaboration support includes shared user interface management primarily by providing synchronous editing of shared spaces. We do not focus on providing sophisticated shared user interface management in this prototype. Rather, we will implement a simple group awareness mechanism: As described in Section 6.3.4, the server broadcasts notifications about the whereabouts of other clients. This can be used by the editor to implement a basic group awareness mechanism. In our case, users can see other users that are present in the same space.

6.4.5 Summary

This section has described the implementation of a spatial editor for use with a spatial structure service component. It is a spatial client of significant size and complexity that demonstrates the utility and efficiency of providing open hypermedia facilities within the Construct framework. The editor shows the practical possibility of combining open spatial hypermedia with collaboration support.

6.5 Spatial interface

Based on the conceptual design of the spatial interface we have implemented our first proposal for a spatial protocol. Our implementation of the protocol matches the spatial server and the spatial client interface description of Section 5.5. Other implementation issues concerned with the spatial protocol are described below.

When implementing a prototype, we have to decide how to transfer the parameters of the messages. According to the interface described in the conceptual design we need to transfer identifiers, name-value pairs, content units and space units. Identifiers are of a basic type (in Construct) and do not require any special treatment to be transfered over the network. Name-value pairs, which we call characteristics, are transfered by compiling them to appropriate Construct types, which then can be transfered through provided Construct mechanisms. Finally, content units and space units are basically a set of characteristics and are transferred as such.

In our implementation, only defined characteristics of units are transmitted, allowing to transfer partially defined units. Each component of the system can further add new characteristics to the set, thereby stating its contribution to the units identity. For instance, it is the hyperstore that assigns an identifier to a space unit, thereby simultaneously acknowledging its existence to other components of the system. Actually, the existence of a unit begins when it is assigned its identifier which allows it to be referred to.

Turning to the type of messages sent back and forth between the clients and the server, the conceptual design chapter (Section 5.5) defines three different types: requests, responses, and notifications. For this, the Construct framework supports different mechanisms. How the server and the client use these mechanisms is described below.

6.5.1 Server interface

To enable the server to respond to a client's request, Construct provides a subscription mechanism. For instance, a request that requires a response, triggers a *one-time-only* subscription, ensuring that the server knows who to send the response to. Notifications are characterised by being sent to all clients of the current session.

Connection protocol

Our connection protocol is designed to be as simple as possible, while giving the system a flavour of session management. As discussed in Section 6.3.4, the only operation in the protocol is join, which enables clients to select a predefined collaborative session:

join A request from a newly connected client to select a collaboration session in which it wants to operate. The message parameters are:

- the session identifier; and,
- the name of the user of this client.

Following the exchange of this message, and presuming the request is granted, both the client and the server use the server and client interface, respectively, for all further communication. If the request fails, the spatial service will drop the connection to the client.

Leaving a session is implicit when the client closes the connection to the server.

6.5.2 Client interface

Clients behaviour after sending a message depends on the type of the message they send. In the case of a request that requires a response, the client blocks, waiting for the response before continuing its task. The remaining requests are non-blocking, characterised by the fact that the client progresses in its current task without waiting for a response from the server. Notification messages require no response and are therefore non-blocking, too.

The connection protocol defined above provides a single operation that is used by the client to join a predefined session in the server.

6.6 Summary

This chapter presented CAOS, our programming work—a spatial structure service component for use with the Construct CB-OHS framework, and an accompanying sample client, a spatial editor.

In the following chapter, we evaluate this work and identify critical issues that need further work in order to improve various aspects of the implementation. Chapter 8 suggests possible directions for future work, some with long-term implications.

Chapter 7

Evaluation

7.1 Evaluation of functionality

According to the introduction (Chapter 1), our main focus is the topics of spatiality, openness, interoperability, and collaboration support. This was supposed to result in a collaborative open spatial hypermedia system that supports a high degree of applicability. In the following, we will evaluate each of the concepts mentioned, and evaluate whether we attained our initial goals.

7.1.1 Spatiality

In this section we ask whether we have actually made an implementation that satisfies the characteristics of a spatial system. The literature survey (see Section 3.2.2) presents a section that characterises spatiality. According to the survey, spatiality can be characterised by the following keywords:

- Structure creation.
- Emerging structure.
- Flexibility.
- Common understanding.
- Handling large structures.
- Spatial parsing.

In the following we will discuss how our system satisfies these keywords.

Structure creation

In spatial hypermedia, focus is on active creation of structure rather than reading and traversing structure.

CAOS IS AN EDITOR. Our spatial application is an *editor*, not a browser. Although users can use our editor as a browser to navigate and inspect spatial structure, the focus is on structure creation. The mapping of space units to their

view (see Section 5.4 and Figure 5.6) is usable for visualising the generalised spatial structure model. Navigation of structure can be done by the user by diving in and out of subspaces (see Section 5.4.4). We adopted this convention from VIKI, and it seems to be the obvious way to navigate hierarchical structure in spatial hypermedia. Single space units or collections of space units can be moved around in a space or between different spaces. Space units can be resized and deleted from a space.

STORING STRUCTURE. The spatial parser in the server is triggered by modifications to spaces. Structure produced by a parse is shown visually as structure indicators. This functionality is basic for our system, and it shows that the incremental parser works by demonstrating this in a graphical user interface. All spatial structure, both visual and explicit, is stored explicitly. These aspects stress the focus on structure creation.

Emerging structure

Spatial hypermedia allows structure creation in a phase where users only have a fuzzy idea of what kind of structure to create. Spatial hypermedia allows explorative development of informal emergent structure. Stepwise refinement of structure can be done without changing the integrity of the whole structure.

FUZZY STRUCTURING. Users only need to worry about how to express implicit structure visually. The system automatically parses the implicit structure and creates an explicit representation of it. This can adjust the explicit structure without changing the integrity of the explicit structure. Fuzzy structure, that is, structure not recognised by the parser, is allowed to in a space. Fuzzy structure is not accepted as being explicit structure before it is modified so that it satisfies the heuristics specified by the parser experts.

NO NAMING. Space units need not have a name, and space units can be created as placeholders of unspecified content. This means that structure can be created without content and naming conventions being present. Explicit structure need no naming, either, it just appears visually as structure indicators.

CHANGING STRATEGIES. CAOS allows users to change structuring strategies during a structuring task. In CAOS, explicit structure is produced by reparsing visual structure whenever users move space units around. This means that users do not need to create or destroy explicit structure—it can emerge and disappear automatically, as controlled by the parser. This is useful when people change structuring strategy during a structuring task. The ease with which explicit structure is managed by the parser illustrates the support for emerging structure in CAOS.

Flexibility

Spatial hypermedia allows people to record various interpretations of bodies of material, by letting them arrange material in multiple ways. It allows users to visually structure space units by proximity and alignment. The ability to

express meta communication about material is also important. Along with spatial properties, high expressive flexibility should be supported by the ability to change visual characteristics of space units.

CHARACTERISTICS. We have made a spatial model capable of expressing mostly the same variety of space unit characteristics as seen in VIKI, where space units have the following visual characteristics: `bounds`, `shape`, `background colour`, `border colour`, `border thickness`, `text content`, `text font`, `type`, and the possibility to make extra fields definitions. Apart from `type`, our space unit has the same characteristics as seen in VIKI, plus the following characteristics: `title`, `kind`, `parent` and `children` (see Section 5.2.1).

The immediate difference between the two models, is that VIKI has the notion of a `type`, which resembles a set of characteristics in one abstraction. We have chosen not to introduce the notion of a typed space unit, since we want to keep our model simple.

The mentioned characteristics allow users to structure material in multiple ways. Visual characteristics allow users to specify meta communication about material (e.g., by using various colours, shapes, etc.). In the CAOS system, the spatial protocol supports communication about changes of the above mentioned characteristics. The spatial server supports at the moment changes to space unit fields, but support for specification of extra fields is not implemented yet. The spatial client supports the possibility to change colours of space units.

A comparison with VIKI reveals that VIKI offers a much broader support for different characteristics of space units. We have chosen to regard the ability to change colours as being sufficient to illustrate the point of supporting meta communication about space units. Our server can though support a much broader flexibility concerning the possibility to change characteristics.

UNIFYING SPACE UNIT AND SUBSPACE. Our spatial model unifies space units and collections, so that all space units have an associated collection, which we call a subspace. We think that unifying the collection and space unit reduces the number of structural abstractions with which the user has to cope without reducing the expressiveness of the model. Because all space units in our model can reference content objects containing text, all spaces (which are part of a space unit) thereby automatically have content. For example, this can be used to explain the purpose of the space textually).

VIRTUAL COPY. In our spatial model it is possible to make virtual copies, because content is external to space units. Virtual copies enable users to make different space units that refer to the same content. Virtual copies are important structural abstractions, because in combination with visual characteristics they can assign different structural meaning to the same content in different contexts (see Section 3.2.1).

EXPLICIT STRUCTURES. The spatial parser in CAOS finds structure based on proximity and alignment. The spatial parser in VIKI can find the structures shown in Figure 3.7. In comparison, our parser design can find structures such as heaps (called piles in VIKI), horizontal lists, vertical lists, and matrices, and

make them explicit. Our parser is not able to find stacks which is possible in VIKI. Users are free to visually structure space units in stacks, but the parser in CAOS will most possibly interpret stack structure as a heap. Our parser design is capable of finding matrices, which is not possible in VIKI.

SPATIAL RELATIONSHIPS. Figure 3.5 and 3.6 show different ways to express relationships in spatial hypermedia. In the following, we will argue that our spatial system enables users to express the whole family of spatial relationships:

- *Relations by spatial arrangement*: In CAOS it is possible to place space units using proximity and alignment to create visual structure. The spatial parser can find structure belonging to this family of structures.
- *Relations by object type*: In CAOS it is possible to change colours of space units. Thus, space units of the same colour can be interpreted as being related. We have not implemented a spatial parser that can reveal relationships belonging to this family of structures.
- *Relations by collection*: Every space unit in CAOS has an associated subspace that can contain space units. This implements the possibility of structuring space units by subspaces (collections). The resulting hierarchical structure is stored explicitly.
- *Relations by composite*: In CAOS it is possible to express composites visually, but users have to create them from scratch every time they wish to repeat a kind of structure. CAOS does not parse for composites.
- *Relations by activity related areas*: Spaces in CAOS are infinite. Therefore, spaces allow big areas to be populated with clusters of space units, thereby allowing activity related areas to emerge. The spatial parser in CAOS does not have a behaviour that can find activity related areas (like in Web Squirrel where neighbourhoods of clustered objects can be found).

From the arguments above we conclude that it is possible in CAOS to express the whole family of relationships mentioned in Figure 3.5 and 3.6. We have implemented a behaviour that can find some of the relations.

Common understanding

CAOS allows people to move objects around in spaces and suggest structure in group settings. A common understanding of structure can in this way be developed over time. The materials themselves are of limited utility without this shared understanding. It is the perception of the structure that spatial hypermedia wishes to address. Personal or common reflections, understanding and interpretation of collections of information can be communicated to others through spatial structure.

COLLABORATION FUNCTIONALITY. In CAOS we present an alternative to the work setting in which multiple users are placed in front of one instance of a spatial tool, talking about structure, while one user modifies spatial structure.

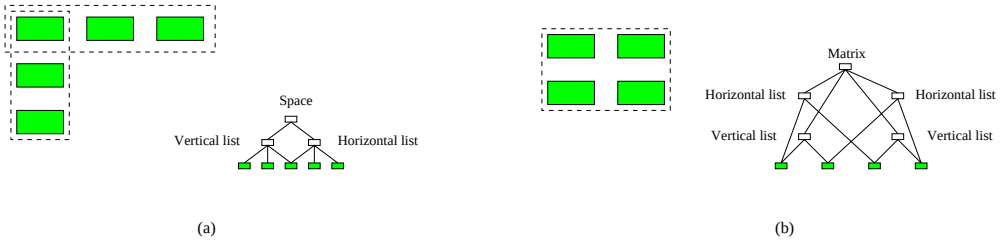

Figure 7.1: *Structures in which multiple parents would be useful.*

By supporting two coupling modes, and group awareness in CAOS, it is illustrated how users can cooperatively create structure in a setting where all users have hands on experiences with the task going on. This takes spatial hypermedia further than before, and enhances the common understanding that can be developed during the course of the task at hand.

Distributed editors. Distributed editors are a consequence of the CB-OHS framework. This means that users working collaboratively in a session can be distributed. Common understanding of the structuring task at hand is enhanced because they can share the hands on experience while structuring.

Visual feedback. Visual structure feedback makes possibly ambiguous perceivable structure unambiguous for collaborators because the parser aids in determining what is structure and what is not. The parser also aids in choosing what kind of structure to find. This means that collaborators to a higher degree can agree upon the structuring task at hand, because they are not in doubt of which structures have been made by other users, because the parser has taken that decision. Continuous visual feedback make users "structure aware" in the sense that they always knows what has been perceived as structure and what has not. This is important, since visual structure that has not been perceived by the spatial parser, has not been recorded and stored as explicit structure. By making visual feedback the user is not in doubt whether the system has perceived the intended structure.

Handling large structures

Handling large amounts of content addresses the problem of structure that grows large. The chance that users lose track of their position and context while they navigate or structure it is present. Spatial hypermedia allows people to structure large amounts of data in a comprehensible way.

Comprehensible structure. The spatial model in CAOS has only one parent. This is a consequence of unifying the space unit with a subspace. The model presents only two directions while navigating spatial structure, namely up to the containing space unit or down into the subspace of one of its contained space units. This single parent model makes our spatial structure more comprehensible, because choices of ways by which to navigate are limited, and

there is only one path leading to a particular space (see Section 3.2.2).

Introducing a spatial model with multiple parents would make the spatial structure more expressive, because more complicated structure could be expressed in such a model. For an example of this, look at Figure 7.1. If multiple parents are present, multiple paths can lead to the same space. This may cause users to lose the global overview of the structure because they no longer can deduct their position in the structure from how they navigated in to their current space. If cycles can occur in the structure this problem becomes even worse.

When dealing with large structures it is desirable to maintain an overview. We are aware that we limit the expressiveness of our model when restricting ourselves to a single parent spatial model. On the other hand, our model is easy to comprehend since it represents a hierarchy.

Infinite spaces. Our CAOS client supports an overview presentation of space units, which have been shown to be appropriate for information analysis in a spatial context. Systems like Aquanet, gIBIS, NoteCards, VIKI, Web Squirrel, Storyspace, etc., all present objects in an overview picture. Systems like VIKI and Web Squirrel offer the ability to view a space in different zoom modes. In VIKI a functionality called "view all elements" is present, which is useful to get an overview of an entire space. This overview can perhaps reveal activity related regions in that space which were not apparent to the user beforehand. Zoom functionality is not implemented in the CAOS application. Instead, scrollbars enable the users to view the created spaces.

Selecting space units. Hierarchical click selection, as seen in VIKI, is handy functionality for moving parts of structures, because it becomes easier to select (parts of) structures. Our CAOS application does not implement hierarchical click selection.

Spatial parsing

The spatial parser is a key component in a spatial hypermedia system. The spatial parser builds on heuristics for recognising general spatial patterns. This cannot satisfy all different personal perceptions of what is structure, but it can define a set of acceptable structures upon which most people agree that they represent common spatial structures.

Incremental parsing. The design of our parser presents an idea for how to make spatial parsing incremental. By designing an incremental spatial parser that finds structure based on proximity and alignment of space units, we have shown that it is in fact possible to localise structure changes applied to this kind of spatial structure. Based on these localised structure changes, an incremental parser can exclude parts of structure that are not effected by the localised changes. This makes the parser incremental. When many space units are present in a space, a considerable speed up can be noted with the incremental parser compared to spatial parsers where all structures are rebuild from scratch.

Continuous visual feedback. Our parser is triggered by user actions. Explicit structure is therefore always consistent with the visual structure present in the space. This presents the basis for provision of visual feedback, which gives the user a good idea of which structures are currently present explicitly and thereby persistently stored in the hyperstore.

The presentation of visual feedback in shape of frames surrounding the structured elements seem to give an acceptable encapsulation of the found structures. Lists and matrices that by nature have rectangular emergence are captured well by frames. Heaps though can leave larger areas of the surrounding rectangular frame unfilled by structure, but the frame can then be seen as a measure for the spatial extent of the heap structure, which is useful in contexts where alignment of heaps is intended.

The assignment of different colours to different structure indicators basically seems to enable users to differentiate between the found structures. The visual structure encapsulated in the visual indicator tells the user which kind of structure is found. Users do not seem to discern the found structure alone from the colour of the visual indicator.

We have not seen any systems that focus on automatic continuous structure feedback as our system does. VIKI can reveal explicit structure on demand, but our system seems to support the user with a better feeling for the structure at hand.

Because CAOS focuses on making explicit structure persistent and available for other applications to use, the continuous visual feedback is vital for our system. On the other hand too many and too complex structures can confuse the view of the space when many structural indicators are present. It could therefore be argued, that it should be possible to turn off visual feedback, thereby enabling the user to create structure solely based on implicit structure. It is not possible though in our client to turn off structure feedback.

Summary

This section evaluated to what extent our CAOS implementation can claim to be a spatial hypermedia system. Our CAOS implementation was compared to the characterisation presented in Section 3.2.2, and treated in detail. The evaluation revealed that our CAOS implementation succeeded in implementing a substantial amount of the functionality that characterise spatiality. On the server side we noted that we have a spatial model that is capable of expressing more characteristics than VIKI's spatial model. The spatial parser illustrates the system's ability to extract explicit structure from visual structure. The parser can be extended with extra experts defining new ways to perceive structure. On the client side, we have made a user interface that visualises the structural abstractions of the spatial model. Visual feedback can be shown, and the functionality required to change a subset of the characteristics of space units was implemented. All in all, the evaluation has shown that our system succeeds in providing enough spatial abstractions to be called a spatial hypermedia system.

7.1.2 Openness and interoperability

In this section we will ask whether we have succeeded in integrating an open system supporting interoperability. With respect to open hypermedia systems, the literature survey (see Chapter 3) presents two requirements

1. orthogonality of structure and data; and,
2. the specification of a well-defined protocol.

The interoperability issue is concerned with another two requirements:

3. ensuring the (re-)usability of the systems resulting product (in our case, spatial structure); and,
4. that other entities can use its services.

Items 2 and 4 concern the spatial protocol, while the remaining two concern the spatial model and the persistent format, respectively. These concerns will be evaluated in the following.

Spatial model

One of the requirements that openness puts on hypermedia systems is the orthogonality of data and structure. This has consequences for the model that the system operates on. In our case we have to ensure the separation of spatial structure and data. This is expressed by the separation of space units and content units described in Section 5.2. A space unit represents a visual object that can refer to a content unit storing data. The content unit is incapable of referring to other *spatial* objects.

This ensures the orthogonality of spatial structure and data, allowing spatial structure to be applied to an open set of data formats that can be referred to via the content unit.

Spatial protocol

A spatial protocol is a key issue for supporting both openness and interoperability. Openness requires the *provision* of a protocol so that other entities can be enabled and participate in CAOS's spatial hypermedia services. Interoperability requires the provision of a *general* spatial protocol so that applications enabled to use one spatial hypermedia service can use another spatial hypermedia service without the need for modifications.

Our spatial protocol is described in Section 5.5. This is an attempt to define a general spatial interface. Obviously it has been affected by our spatial model and the operations needed by our spatial editor client. The design reveals traces of our efforts to fit the protocol to our concept of spatiality. This resulted in an interface that provides operations such as, `add new child` and `add child` that build on the assumption that the spatial model is purely hierarchical. On the other hand, the existence of the `set characteristic` operation represents a very general operation, revealing our intention to provide a complete interface.

In other words, we may have been affected by the "ease of use" issue [NGS+98] when we designed the spatial interface, but we have not realised any flaws with respect to the completeness of it so far.

Another point worth mentioning is that the services provided by the server are implemented as several calls to the hyperstore, reflecting the need for several sub-tasks. Why did we not provide services reflecting each of the sub-tasks? One reason is that the encapsulation of the sub-tasks into one service allows us to omit the introduction of transactions to clients. For the clients all services appear to be atomic operations.

The provision of a well-defined spatial model satisfies the openness requirement. Our attempt to provide a well-defined *general* spatial protocol did not succeed with respect to its generality. Thus, the interoperability aspect of our protocol might not be satisfied. The supported operations are still far to specialised. Our protocol proposal can be seen as a first-generation solution, that obviously has to be iterated over.

Generic structure

The ability for other systems to make use of the spatial structure produced by our spatial hypermedia system is an issue of interoperability. Persistence of spatial structure is in this case essential, but this is not enough. Different systems also have to coincide on how to store structure.

In our case we have solved this problem by extending a CB-OHS called Construct. Construct provides us with a hyperstore, storing generic structure units, and an infrastructure including a hyperstore API that is used by our spatial structure server.

The adaptation of a generic structure unit and the use of a commonly accessible hyperstore stresses that our spatial hypermedia system supports interoperability at the storage layer. In the following we will give an evaluation of the extended CB-OHS.

CONSTRUCT. Construct is a public domain CB-OHS that is under development at the time of writing. This has had several consequences for our work. Firstly, some planned components of Construct were not available while we implemented our prototype. Secondly, changes made to the framework affected our implementation by requiring changes to our prototype. Thirdly, the provision of development tools [WN99] to support generation of communication-classes based on interface specifications would have been useful. These tools were not available, which forced us to implement trivial classes from scratch. Apart from these minor weaknesses, Construct has in fact provided us with essential parts to enable us to implement our current spatial prototype. The Construct framework has proven to be well designed and it already supports many of the facilities required by a CB-OHS. In particular, the integration of the hyperstore with our prototype has confirmed the applicability of the framework architecture. All in all, Construct has proven its worth with respect to integration of new structural domains, which in our case was the information analysis domain.

Summary

This section inspected our implemented spatial hypermedia system with respect to openness and interoperability support. Openness is supported by the provision of a spatial protocol and the separation of structure and content. Thereby, we have succeeded in providing an open spatial hypermedia system.

With respect to interoperability, an attempt to support storage layer interoperability has been made by using a hyperstore to maintain generic structure. The provision of a general spatial protocol, supporting so called presentation interoperability, failed. This leaves us with support for storage layer interoperability only.

7.1.3 Collaboration support

This section discusses our achievements with respect to incorporating CSCW functionality into our prototype. The problematic issues about the implementation are discussed.

Evaluation of collaboration support

The literature survey (see Section 3.3.1) presents a number of key concepts according to which CSCW functionality can be characterised: application architecture; session management; shared data management; shared user interface management; communication; and, coordination. In the following we will evaluate the support for collaboration in our prototype according to the concepts listed, except the last two, because they cover topics that are outside the scope of this thesis.

Application architecture. Our system is based on a centralised architecture consisting of a central structure service that broadcasts messages to distributed clients to keep them synchronised. The support for various aspects of cooperativity is provided through this centralised distribution of messages.

Session management. A rudimentary session management system coordinates the distribution of messages among clients. It uses coupling modes to filter the amount of messages sent to clients. Session management is not central to our thesis, but a prerequisite to providing collaboration, which is why we only implemented a basic mechanism to handle it.

Shared data management. The structure service offers shared data management by resolving concurrent write conflicts, allowing collaborative clients to modify structure in an ordered fashion. Structure changes are reported to clients through fine-grained notification messages that allow clients to maintain a synchronised view of shared structure. This is the aspect of collaboration support that is best developed—all four quadrants in the space-time taxonomy (Table 3.1) are supported with respect to shared data management

Shared user interface management. With respect to shared user interface management in combination with spatial hypermedia, we are entering a

field in which, to our knowledge, no previous work has been done. We take a first step by implementing a simple group awareness mechanism in the prototype. It is based on a protocol of voluntary information sharing between clients, and it illustrates an approach to adapting a generic concept of CSCW functionality to spatial hypermedia. It is successful in the sense that is works, and in that the adaptation makes sense.

Problematic issues

We have identified scalability as one major issue in our approach to combining collaboration with spatial hypermedia. There is a substantial issue regarding scalability in our current prototype. The normal operation of clients is very heavy on communication with the server. This is aggravated by the automatic spatial parser, since a simple modification request can result in a flood of updates that has to be distributed to all interested clients. We conjecture that the bottleneck that limits performance of such a spatial structure service will be communication overload. To reduce these problems, changes to the architecture and to protocols should be proposed that change the communication patterns in the system towards having clients communicate directly with each other instead of through the server. This goes for both shared data and shared user interface management.

Summary

To summarise, our implementation of collaboration support can be characterised as a proof-of-concept experiment to assess the viability of adapting collaboration to spatial hypermedia. The results are positive, showing that sharing of spatial structure is certainly possible, by employing standard mechanisms for sharing hypermedia structure. It also demonstrates that the adaptation of the concept of group awareness is sensible and doable. We conclude that collaboration support in spatial hypermedia is a viable goal, and our prototypic implementation of a spatial system shows the usefulness of such functionality. We also discussed that careful design of the communication patterns between structure services and clients (and possibly between clients) are required to prevent overloading the server.

7.1.4 Summary

This section evaluated the three main areas treated in this thesis. The functionality of our CAOS implementation was compared with general functionality described in the literature survey chapter. Generally, we can conclude that our implementation satisfies the demands for being an open hypermedia system. It enables interoperability which is the prerequisite for authoring. The demands for being a spatial hypermedia system is clearly satisfied. The last issue of collaboration is a bit more doubtful, but we call CAOS collaborative, because CAOS implements shared data management, and simple shared user interface management. This justifies calling CAOS is a cooperative and open spatial hypermedia system.

7.2 Evaluation against scenario

In this section we evaluate our prototypic implementation against the scenario and assess to what degree it is able to support the work process described therein. We proceed by identifying the required functionality in the scenario; for each item, we describe to what degree CAOS is able to support it.

7.2.1 Supported functionality

In the following, we will go through the operations mentioned in the scenario that our current prototype supports.

Single-user scenario

- "*He uses a spatial hypermedia system ... to organise his material, and to perform brainstorming.*" This is supported by the spatial functionality in CAOS.
- "*Allan activates his favourite editor application ...*" Because it is an open system, CAOS allows any application enabled for spatial hypermedia to access spatial structure.
- "*He then opens an information space ...*" Opening an information space is equivalent to connecting to the spatial structure service in CAOS, since it serves only one information space at a time.
- "*Documents are represented to Allan as small paper notes on a big table*" This is the de-facto standard way to represent information units in spatial hypermedia tools, including in CAOS. The big table with paper notes on it is a metaphor that is modelled by spatiality.
- "*He adds some icons that contain references ...; others that contain the documents themselves ...*" Text is the content type allowable in content units that can be viewed through space units in CAOS, thus permitting both documents and references to external documents to be entered.
- "*A web browser extended for use with spatial hypermedia ...*" A suitably extended web browser can act as a spatial client to the CAOS structure service due to it being an open system.
- "*... Allan makes patterns with the icons, in the form of piles and lists ... Some documents are marked ...*" CAOS allows users to create spatial arrangements of space units, and to change colours of the space units.
- "*The system helps ... by providing various visual structure indicators ...*" The CAOS spatial editor provides visual feedback of explicit structure found by the spatial parser.
- "*He never asks the system to look for the structure—it happens automatically...*" Parsing in CAOS is triggered by user updates to spatial

structure, thus keeping the explicit structure synchronised with the visual structure.

Multi-user scenario

- "*Brian starts up a spatial client . . . [and] . . . opens the space Allan is working on*" Sharing of spaces is supported in CAOS, provided that the two clients connect to the same structure service.

- "*Brian adds his references to the common space, and his additions are immediately shown on Allan's terminal*" Because the structure service keeps collaborating clients synchronised at all times, changes made by one user are automatically reflected to all collaborators.

- "*. . . the common understanding of the subject . . .*" The property of being able to enable a common understanding of the subject is a consequence of the fact that spatial hypermedia and collaboration have been merged into our system.

Interoperability scenario

- "*. . . they both close their spatial editors. The structure they have made is stored persistently in a hyperstore.*" All structure, both visual and explicit, is always stored persistently in a hyperstore.

- "*The published parts . . . are still available . . .*" Because spatial structure is stored persistently in a hyperstore, this enables it to be used by other structure services.

- "*. . . changes to the published parts being reflected immediately to people browsing the material.*" Because the hyperstore provides notifications whenever (spatial) structure is modified, all structure services can maintain an up-to-date view of (spatial) structure, no matter how it is modified.

7.2.2 Unsupported functionality

Single-user scenario

- "*. . . he marks it, so that a reference to the page is automatically added to his information space.*" A browser application that uses the spatial protocol to create space units via the spatial service has not been developed, but the system support needed by such a client is in place.

- "*. . . whenever Allan needs a closer look at a reference . . . the spatial system can display the document using the browser . . .*" There is no support for using external viewers in CAOS, so making it spawn a browser to view a document referenced by a URL in a space unit is not implemented.

Multi-user scenario

- "*Brian's view of the space is shown according to his own preferences ...*" The topic of preferences was discarded in Section 4.1.2.
- "*Brian ... has recorded some observations ... in a space of his own.*" It is not possible to work with the spatial application off-line.
- "*Brian ... opens the space Allan is working on.*" Sharing of structure can happen at two levels in a CB-OHS: either, clients use the same structure service, or multiple structure services share a hyperstore. CAOS only supports sharing of a structure service so far. This implies that Brian, if he was using CAOS, would have to connect to Allan's structure service.
- "*Brian adds his references to the common space ...*" The CAOS spatial client does not support moving spatial structure between structure services (using, e.g., cut and paste). It is only allowed between spaces served by the same structure service.
- "*So they start a collaborative work session*" CAOS supports collaboration, but not transition between collaboration (i.e., coupling) modes.
- "*While ... pointing with telepointers ...*" CAOS does not support telepointers since it is related to tightly coupled clients, and CAOS only supports loosely coupled mode.

Interoperability scenario

- "*Eventually, Allan and Brian ... decide to publish some of their findings on the World Wide Web.*" Interoperability, to the extent of performing structure processing to convert spatial structure to navigational structure, is outside the scope of our project, and has not been implemented. However, we have taken the first step in the right direction by making explicit structure persistent. A separate navigational structure service can be written that processes the explicit structure to produce HTML from it.

7.2.3 Summary

This section evaluated our CAOS implementation against the scenario. A thorough look through the scenario functionality and a comparison with functionality in CAOS was presented.

From this walk-through we conclude that the functionality presented in the single-user scenario can almost be satisfied by our present implementation. However, the multi-user scenario requires some functionality that our system cannot currently provide wrt. CSCW. The interoperability scenario presented functionality that includes aspects of interoperability. Our CAOS system has provided the needed functionality for storing spatial structures as generic structure. This structure can then be used by associative domain structure servers to present navigational structure. We have though no practical experiences with storage layer interoperability.

7.3 Summary

The result of the two sections in this chapter leaves us with the result, that we have implemented a system that combines the aspects of spatiality, architecture openness and collaboration support. We have argued that our system satisfies each of these aspects in turn, which means that we can now start to get experiences with a collaborative open spatial hypermedia system. Our implementation was compared to the original intention in the scenario, and we can now see that we were too optimistic about how much we'd be able to implement.

We have made an open spatial hypermedia service supporting collaboration. This service constitutes the middleware of a CB-OHS, with advantages already known. We have implemented a sample client to demonstrate spatial functionality at the application layer. The CAOS application only implements a subset of the offered functionality from the spatial service. This means that our evaluation leaves us with the impression that our CAOS prototype is fairly vertical, and lacks some horizontal work. In other words, the usability of our system at the moment is not outstanding because we have only implemented a limited sample application.

Chapter 8

Future Work

This chapter presents directions for future work on our spatial structure service that we have identified, and of spatial hypermedia in general in the context of CB-OHSs.

8.1 Future work on CAOS

This section discusses possible solutions to, or makes points that can alleviate, problems addressed in the evaluation of our work (see Chapter 7).

8.1.1 Spatial service

Based on the evaluation of the spatial server, we present several directions in which future work can proceed. We have divided this section into two parts: Firstly, we consider functionality improvements based on the current state of our prototype. Secondly, we discuss architectural improvements of our prototype.

Functionality

In its current state, our prototypic implementation provides most required services with respect to spatial functionality. Collaboration support is provided through shared data management and a group awareness mechanism. This is sufficient to start gathering practical experiences with respect to collaborative spatial hypermedia. Before implementing sophisticated collaboration support it would be useful to investigate how people actually work with CAOS.

The integration of further collaboration support is a major area of future work. As mentioned throughout the thesis, there are a multitude of CSCW issues to be considered. Some depend on the level of general collaboration support in the architecture or the framework. A clarification of these issues with respect to spatial hypermedia would help towards our goal of making a collaborative spatial system. We consider our research and prototype a good base for future work in this direction.

Designing new behaviours for the spatial structure service is also an interesting direction of work. A system that provides pluggable behaviours would be adequate for this field of research. Thereby, it would be easy to extend, or even

customise, the functionality of a spatial service according to the preferences of users. This could be done, by letting users decide which behaviours they want to use. For example, users working with such a system could be interested in finding composites (see Section 3.2.1) in their visual structure. For this purpose, the system could at run-time load a behaviour that provides the desired functionality, thereby extending its applicability.

Some aspects that we have not considered at all are issues like crash recovery and scalability. To some degree scalability will be inherited from the CB-OHS framework. Experimentation with the current prototype could be performed to identify bottlenecks in the architecture of the structure server that affect scalability.

Architecture

The architecture of the spatial server can be improved in several ways. One major improvement would be the introduction of a new spatial model that can improve expressiveness. This would have major impact on all parts of our system, and would cause extensive modifications, not least in the spatial parser. A more general spatial model is presented in Section 8.2.1.

Any structure service should support arbitrary behaviour components to be plugged in to extend it. We consider our spatial parser to consist of such behaviours. At the moment the parser is integrated in the spatial structure service. We would like make the structure service extensible with respect to behaviour components. This would allow different parser components to be present in the service. Parser components could be implemented as schedulers for expert components, that specialise on finding one kind of specific structure. Support for pluggable behaviours should be present in the framework before seriously undertaking work in this direction. Currently, it is still a field of research.

Generally, the architecture of the core of the structure service is still an open issue. In our case, we decided to implement a single central structure-cache that is modifiable by the structure service *and* all behaviours. This resulted in some undesirable properties when failures occured (e.g., the occurrence of a hyperstore abort while parsing). Maybe a better suggestion would be to require each behaviour to maintain its own cache, presenting its private view of the visual structure. Again, practical experiences are crucial.

Something worth adjusting is the spatial protocol, to make it comply with the proposed format described in [NGS+98]. This would require either a rewrite of the spatial editor's communication layer, or constructing a proper spatial server client library, as suggested by the Construct framework.

8.1.2 Spatial editor

This section addresses some aspects of the prototypical spatial editor that need more work.

Collaboration support

Significant effort is required to improve the current shared user interface management to the standards of modern CSCW implementations. The hardest task is adaptation of collaboration concepts to spatial hypermedia, to make them meaningful and useful, and to ensure that they are feasible to implement.

Disabling visual feedback

It should be possible to switch off visual feedback of explicit structure, if desired. In some cases, it can be disturbing to be presented with such indicators (e.g., when one has no intent of storing it or re-using it in some other context). This allows free play with spatial structure without worrying about the parser and what it does. This is an easy feature to add.

Spatial expressiveness

There should be better support for spatial expressiveness. For example, being able to specify border thickness, fonts, extra characteristics, etc., is required to reach the level of expressiveness in VIKI. This is fairly easy to add, since almost all the required backend functionality is already there.

Overview zooming

This feature allows the view of a space to be zoomed out, so that the entire space can be rendered with all its contents visible in one glance. This functionality could be implemented in a general way by allow zooming of views of spaces, which is not trivial to implement.

Advanced content

One may imagine a spatial editor that is capable of handling a large range of content, such as text and pictures (both in various formats). For example, if HTML is viewed, it should be rendered properly in the spatial view, and pictures should be shown directly.

Hierarchical click selection

Hierarchical click selection is handy when moving parts of structures, because this selection makes it easier to select parts of structures. It can be added to our spatial editor relatively easily, because it bases its functionality on the provided explicit structure.

8.2 Future work on spatial hypermedia

8.2.1 Generalised spatial model

As hinted throughout the thesis, our spatial model is rather limited in its expressiveness. It is a hierarchy, in which all space units have one parent. This

section presents an alternative model that copes with the problems we have identified in the evaluation.

Requirements

The spatial model we presented in Section 5.2 is limited in many respects. It does not support the following properties of our experimental general structural model of spatial hypermedia:

1. The subspace of a space unit should be considered a kind of content, not an inherent property of all space units. This opens up for what one might call "virtual spaces", along the lines of reasoning behind the term "virtual copies".

2. Creating a view into a space from multiple visual units should be possible. This changes the nature of spatial structure to general graphs, introducing the possibility of cycles in spatial structure. Thus, behaviours that traverse spatial structure must use graph algorithms to do so correctly.

3. A space unit should be able to refer to an arbitrary set of content (e.g., a space unit with two subspaces, an image, and a text as content should be possible).

4. Space units should be able to have multiple structural parents in structures for which it makes sense.

5. The structure of space units should be independent of the space units themselves, and a space unit should be able to be part of multiple structures. This enables separate storage of structure over space units (e.g., subspace containment, multiple parsed structures, foreign domain structure, etc.).

The model

This section presents our suggestion for a generalised model of spatial hypermedia. It is designed for use with CB-OHS's in that it is based on the use of general structure units that we presume are provided by the hyperstore in that architecture. The new model is based on the observation that our present space units consist of three orthogonal kinds of information (see Figure 8.1 *A*), namely: the structure(s) the unit is a part of; its visual appearance; and, its content. As indicated in Figure 8.1 *B*, the three kinds of information are split into separate "layers" in the new model, each stored in separate structure units and with cross-references between them.

In the generalised model, the concept of a "space unit" is changed to refer to the visuals layer only. This is so because, firstly, the structure in which a space unit is organised is not part of the space unit itself; and, secondly, the content is not included in space units—rather, they refer to content.

Model layers. The structure units in each model layer have different properties, which will be described in the following.

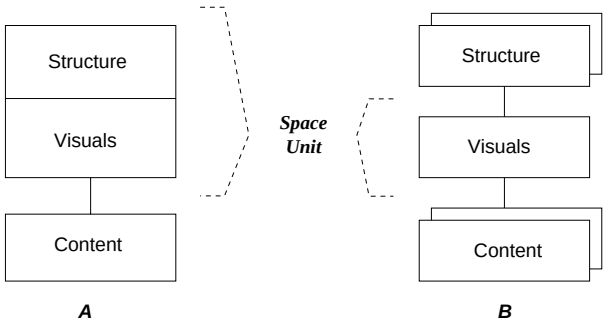

Figure 8.1: *The evolution of our spatial model. Boxes represent structure units, lines represent references between structure units. Model* A *is the present model that uses a single structure unit to represent a space unit. Model* B *is the generalised space model.*

STRUCTURE LAYER. The structure layer represents relationships between spaces and between space units. Because the structure is separate from the space units, it is possible to represent (store) multiple structures over the same spatial objects. It is not a requirement to use separate structure units to achieve this, but our approach avoids problems that might arise from trying to reuse structure units for multiple purposes (e.g., unit field name clashes, or synchronising multiple entities that manipulate the same structure units).

Examples of use are storing explicit structure produced by a spatial parser; storing the visual structure (subspace containment) produced by a spatial editor; or, having structure generated in another context (e.g., in a foreign domain) that makes use of spatial structure.

VISUALS LAYER. The visuals layer stores the characteristics that are related to the visual aspects of spatiality. That is, it stores the visual characteristics that people manipulate in a spatial editor, such as position, size, colour, etc.

CONTENT LAYER. The content layer refers to what is "contained" in a space unit. One may refer to an arbitrary number of other structure units that actually store content (e.g., "text units", spaces, etc.).

SPACES. A space is a structure unit referring to a set of visual units. It may be used as the contents of a unit. Spaces are interconnected in the structure layer to indicate subspace containment relationships.

Spatial structure

SPACES. Figure 8.2 shows a sample spatial structure modelled in the generalised model. There are two spaces $S1$ and $S2$. $S1$ has three visual items in it ($V1$, $V2$, $V3$), and $S2$ has $V4$. The content of $V1$ is represented by $C1$, and it is the text in $T1$. In a similar fashion, the content of $V2$ is represented by $C2$,

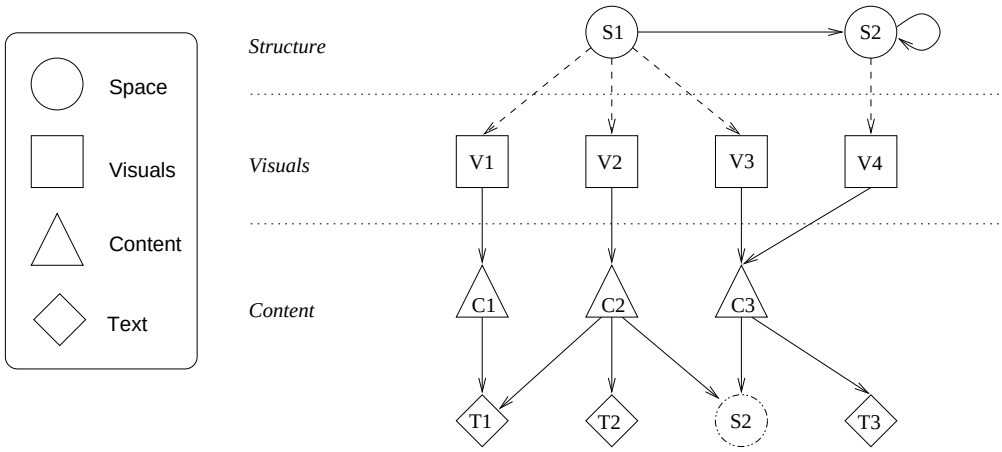

Figure 8.2: *An example visual structure represented in the generalised spatial model. The possibilities for sharing data across levels are indicated.*

and it is $T1$, which it shares with $C1$; and $S2$[1], which then becomes a subspace to $V2$. The content of $V3$ is the title in $C3$; the subspace $S2$; and, the text $T3$.

Because $S2$ is a subspace to $S1$ through $V2$ and $V3$, a parent/child relationship is set up between them in the structural layer. Similarly, because $S2$ is a subspace of itself, there is a parent/child relationship to itself. Using the structure layer like this to represent subspace relationships between spaces can be useful for discovering the relationships between spaces without having to go through all the visual structure, which can become very large. It also exemplifies the idea that structure may exist in the structure layer independently—one does not have to cross layer boundaries to discover the structure.

The example illustrates that the general model can satisfy the properties described in items 1, 2, and 3 in the list of requirements. Items 1 (subspaces as content) and 3 (content is an arbitrary set) are illustrated by $C3$. Item 2 (multiple views into a (sub)space) is illustrated by $C2$ and $C3$ sharing $S2$.

Structure over space units. Figure 8.3 illustrates item 5 (multiple structures over a space unit). It shows the structure and the visuals layers of a space S, and two structures X and Y that organise the visual items $V1$ to $V4$ in S. Item 4 (multiple structural parents to a space unit) is illustrated by $V2$, which has two parents in the X structure.

The X and Y structures could have been produced by, for example, spatial parsers. Each of them has a root node (S' and S'' for X and Y, correspondingly) that represents S in the original visual structure. On the paths from S' or S'' to the visual items, each structure has specialised structure units that group the visuals in substructures. For example, the Y structure could mean "The space S'' (i.e., S) is structured into two lists".

Structure over space units does not necessarily have to belong to the informa-

[1]Remember that spaces can be content to visual units. This is represented by rendering a "shade" of $S2$ in the content layer, even if spaces belong to the structure layer

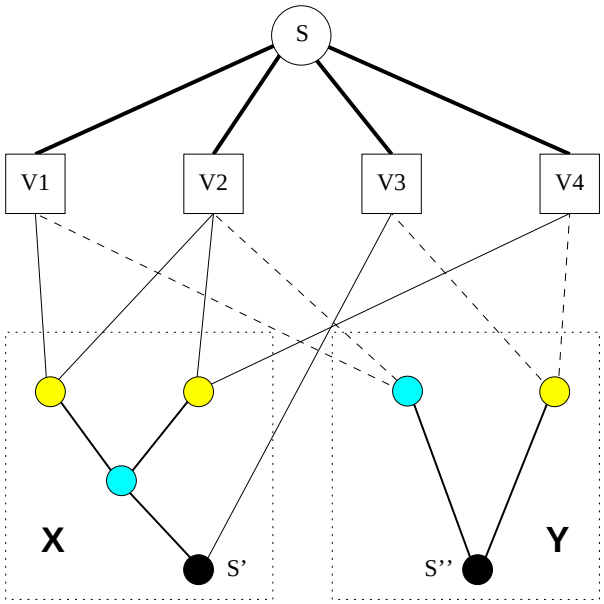

Figure 8.3: *An example of two structures X and Y over the same visual units in space S.*

tion analysis domain. For example, one could make navigational links between space units (as in Storyspace [Joy91]) that is stored as separate structure outside the spatial structure, but with direct references to space units.

Analysis. We have demonstrated that the generalised spatial model presented in this section possesses significant expressive potential, and that it can model structure that satisfies the requirements stated. It is also evident that it is both more expressive, and more complicated than the model presented in Section 5.2. Whether the benefits gained justify the onus of dealing with a significantly more complicated model remains to be seen. In this section, we describe some benefits that have been identified.

Model expressiveness. The expressiveness of the generalised model solves the following problems that we encountered at various stages of the development of our system.

- Consider the five space units arranged so that they form a straight angle (Figure 7.1 a). Such a structure might be parsed in a way that produces a vertical and a horizontal list, each with three elements. Such a structure also require multiple parents, since the corner unit is shared between the two lists.

- The way we store a matrix is not optimal, in the sense that structural information about rows and columns is not stored explicitly—all units in the matrix are just put in a single container. Storing a matrix as columns and rows requires the use of multiple structural parents for visual units,

since they need to refer to both the row and the column they are in (Figure 7.1 b).

In the design of a parser that can generate structures that share visual units, one may use a strategy similar to the one used in our matrix parser: Construct two different parses of the same visuals, then coalesce them to a single structure. In practice, this can be done by cascading parsers, each of which works on structure made by predecessors in the chain.

A feature such as hierarchical click selection can make use of a richer parsed structure, offering better functionality.

PARALLEL STRUCTURES. There ought to be a major positive benefit when it comes to supporting behaviours that store structure. Because logically distinct structure produced by an arbitrary behaviour is stored separately, interfaces between structures become cleaner and the chance of interference is reduced.

8.3 Summary

This chapter presented the directions along which we would like to continue work in the field of spatial hypermedia. We would like to introduce a more general model of spatiality into the system. This would allow us to explore a host of interesting new research directions.

The introduction of a model for general behaviours in a spatial structure service using the general model is interesting. It would provide insight into the nature of behaviours, and it would enable us to experiment with streamlining the interface between the spatial parser and the structure service. Also, experimenting with multiple parsers, and other useful behaviour, could be interesting.

The spatial client poses challenges of its own. Visualising the generalised spatial model is not trivial.

Chapter 9

Conclusion

This chapter presents the conclusions of our work in the fields of: spatial hypermedia; open hypermedia architectures; and, collaboration support. Firstly, we give a brief walk-through of our thesis, to mention the important contributions provided in each chapter. Secondly, we present our conclusions with respect to both our goals and the process by which they were reached. Finally, we audit the title of our thesis.

9.1 Summary of research

The work presented in this thesis was initially motivated by the observation that tools for organising information spatially are rather sparse, and that spatial tools supporting collaboration are nonexisting. Our idea was to implement this functionality in an open hypermedia architecture, and support the possibility for other systems to (re-)use structure created through spatial abstractions. We therefore came up with a scenario that described the functionality required by a collaborative open spatial hypermedia system supporting interoperability issues. We acknowledged three pivotal topics on which we had to focus, namely: spatial hypermedia; open hypermedia architectures; and collaboration support.

Next, we presented a literature survey on the topics acknowledged from the previous chapter. The literature revealed that CB-OHS architectures provide support for openness and interoperability; that spatial hypermedia in general is highly influenced by the work presented in VIKI; and, that collaboration support is primarily concerned with provision of shared data management and shared user interface management.

With the literature survey in mind, we were now able to discuss the topics more throughly, resulting in rough design decisions limiting the issues originally addressed by us. This included the decision of using a CB-OHS framework as the base for our collaborative spatial hypermedia system. Additionally, we noted that faster parsing could be achieved using an incremental spatial parser, which is particularly important when providing collaboration the way our system does it. In the thesis chapter, these throughts evolved into a more concise specification of what our further fields of interest should be.

During the conceptual design and the prototypic implementation we pre-

sented a spatial model capable of meeting the demands stated in the literature survey. The Construct framework was extended with a spatial structure server and a spatial client. Also, an interface between these two components was designed. The prototypic implementation provided us with a system, offering open spatial hypermedia and basic collaboration facilities.

The evaluation compared the outcome of our design and implementation with the intended goals presented in the introduction. Additionally, functionality of an (imaginary) ideal collaborative open spatial hypermedia system described in the scenario was compared with our CAOS prototype.

Based on the evaluation and experiences gained throughout the thesis, the future work chapter treated issues we consider important to the field of cooperative open spatial hypermedia. This included the suggestion of a new spatial model allowing greater flexibility.

9.2 Research conclusions

This thesis has suggested a new tool to help alleviate the problems of organising and analysing (large amounts) of information. We implemented a cooperative open spatial hypermedia system that presented innovative combinations of research fields. For example, allowing users to collaborate in the task of organising information spatially, and to open up for the possibility to further (re-)use structure produced by the organising task.

The evaluation (see Chapter 7) demonstrated that our prototypic implementation provides the required functionality regarding spatiality and openness. The collaboration aspect was much more demanding than anticipated, resulting in a prototype that has only a proof-of-concept implementation of collaboration support. Thus, the resulting prototype lacks practical applicability to real-world-scenarios.

We have realised that we underestimated the work required to provide a system that proves our main goals, the practical applicability of a collaborative open spatial hypermedia system. This caused us to narrow down the expanding set of issues that emerged while researching (see Chapter 4). This resulted in a revision of our goal so that our focus was primarily on proving the conceptual soundness of our proposed system. Secondarily, on its completeness with respect to the functionality making it applicable for real-world-scenarios.

We consider this thesis to be a verification of our primary goal, while lacking with respect to the secondary goal. Nevertheless, the current state of CAOS allows first experiences with a collaborative open spatial hypermedia system to be gathered. These experiences will provide valuable information for further research in the field of spatial hypermedia. Therefore, we estimate our research in general to be of considerable value and believe it provides a good base for future work in this field.

The CAOS implementation has been donated to the public domain under the auspices of the Construct group. This will benefit both CAOS as well as Construct. CAOS will be more or less guaranteed to participate in future research, while Construct can provide software that supports the information

analysis domain. Hopefully, this combination will carry forward the ideas of both collaborative spatial hypermedia and CB-OHSs.

9.3 CAOS

Finally, this section asks whether the name of our prototype is fitting—does CAOS live up to its name?

cooperative We have demonstrated that our system does in fact support cooperative work on spatial hypermedia structure. The spatial structure service enforces shared data management, ensuring structure consistency between clients and server. It also provides common collaboration mechanisms, such as session and connection management.

The structure service provides facilities for clients to implement a shared user interface on the shared data. The spatial protocol includes support for a group awareness mechanism that allows clients to track which spaces are shared with other clients, and who those clients are.

authoring This is indirectly supported, due to the use of a component-based open hypermedia framework that provides hyperstores maintaining generic structure. Thereby, spatial structure is "published" in a publicly accessible location and in a format that is universally understandable.

open Our structure server provides a well-defined spatial protocol and it ensures the orthogonality of structure and data. These are the key issues of openness.

spatial hypermedia Our structure server serves domain-specific spatial structure that can be visualised by our spatial editor. The editor allows users to modify spatial structure, by offering expressive capabilities on par with leading systems mentioned in the literature.

Furthermore, the structure service incorporates a spatial parser (a behaviour), that finds certain types of implicit structure created by the user, and creates an explicit representation of it. Explicit structure is used to provide visual feedback to users about the implicit structure found by the parser. Also, it is stored permanently, along with the visual structure, and kept up to date at all times. These properties of the explicit structure makes it suitable for (re-)use.

Bibliography

[AMY88] Robert M. Akscyn, Donald L. McCracken, and Elise A. Yoder. Kms: a distributed hypermedia system for managing knowledge in organizations. *Communications of the ACM*, 31(7):820–835, July 1988.

[ATW94] Kenneth M. Anderson, Richard N. Taylor, and James E. Whitehead, Jr. Chimera: hypertext for heterogeneous software environments. In [ECH94].

[B+97] R. Bentley et al. Basic support for cooperative work on the World Wide Web. *International Journal of Human-Computer Studies*, 46(6):827–846, June 1997.

[BBF+91] Mark Bernstein, Peter J. Brown, Mark Frisse, Robert Glushko, Polle Zellweger, and George Landow. Structure, navigation, and hypertext: the status of the navigation problem. In [HT91], pages 363–366.

[Bus45] Vannevar Bush. As we may think. *Atlantic Monthly*, July 1945.

[CB88] Jeff Conklin and Michael L. Begeman. gIBIS: a hypertext tool for exploratory policy discussion. In [CSC88], pages 140–152.

[CHI87] ACM. *CHI/GI '87. Proceedings on Human Factors in Computing Systems and Graphics Interfaces*, May 5–9 1987.

[CHI91] ACM. *CHI '91. Human Factors in Computing Systems. Proceedings on Reaching through Technology*, April 27–May 2 1991.

[CM+90] Terrence Crowley, Paul Milazzo, et al. MMConf: An infrastructure for building shared multimedia applications. In [CSC90], pages 329–342.

[CSC88] ACM. *CSCW '88. Proceedings on Computer Supported Cooperative Work*, September 26–28 1988.

[CSC90] ACM. *CSCW '90. Proceedings on Computer Supported Cooperative Work*, October 7–10 1990.

[CSC92] ACM. *CSCW '92. Proceedings on Computer Supported Cooperative Work*, October 31–November 4 1992.

[CSC94] ACM. *CSCW '94. Proceedings on Computer Supported Cooperative Work*, October 22-26 1994.

[CSC96] ACM. *CSCW '96. Proceedings on Computer Supported Cooperative Work*, November 16–20 1996.

[dA86] A. diSessa and H. Abelson. Boxer: A reconstructible computational medium (review). *Communications of the ACM*, 29(2):859–868, September 1986.

[DC91] Prasun Dewan and Rajiv Choudhard. Flexible user interface coupling in a collaborative system. In [CHI91], pages 41–48.

[DHH+92] Hugh C. Davis, Wendy Hall, Ian Heath, Gary Hill, and Rob Wilkins. Towards an integrated information environment with open hypermedia systems. In [ECH92], pages 181–190.

[DKH94] Hugh C. Davis, Simon Knight, and Wendy Hall. Light hypermedia link services: a study of third party application integration. In [ECH94].

[ECD98] *ECDL '98. Proceedings of the Second European Conference on Digital Libraries*, number 1513 in Lecture notes in Computer Science. Springer Verlag, September 21–23 1998.

[ECH90] A. Rizk, N. Streitz, and J. André, editors. *ECHT '90. Hypertext: Concepts, Systems and Applications. Proceedings of the European Conference on Hypertext. INRIA, France*, The Cambridge Series On Electronic Publishing. Cambridge University Press, November 1990.

[ECH92] ACM. *ECHT 92. Proceedings of the ACM Conference on Hypertext*, November 30–December 4 1992.

[ECH94] ACM. *ECHT '94. European Conference on Hypermedia Technology Proceedings*, September 18–23 1994.

[Edw94] W. Keith Edwards. Session management for collaborative applications. In [CSC94], pages 323–330.

[EGR91] C. A. Ellis, S. J. Gibbs, and G. L. Rein. Groupware: some issues and experiences. *Communications of the ACM*, 34(1):38–58, January 1991.

[ES95] Eastgate Systems. WebSquirrel. http://www.eastgate.com/-squirrel/, 1995.

[EWN73] Douglas C. Engelbart, R. W. Watson, and J. C. Norton. The augmented knowledge workshop. In *Proceedings of the National Computer Conference*, 1973.

[Fei88] Steven Feiner. Seeing the forest for the trees: Hierarchical display of hypertext structure. In *Conference Sponsored by ACM SIGOIS and IEEECS TC-OA on Office information systems*, pages 205–212, 1988.

[GHMS94] Kaj Grønbæk, Jens A. Hem, Ole L. Madsen, and Lennert Sloth. Cooperative hypermedia systems: a Dexter-based architecture. *Communications of the ACM*, 37(2):64–74, February 1994.

[GRO97] ACM. *GROUP '97. Proceedings of the International Conference on Supporting Group Work: the Integration Challenge*, 1997.

[GT94] Kaj Grønbæk and Randall H. Trigg. Design issues for a Dexter-based hypermedia system. *Communications of the ACM*, 37(2):40–49, February 1994.

[Hal88] Frank G. Halasz. Reflections on NoteCards: Seven issues for the next generation of hypermedia systems. *Communications of the ACM*, 31(7):836–852, July 1988.

[HMT87] Frank G. Halasz, Thomas P. Moran, and Randall H. Trigg. NoteCards in a nutshell. In [CHI87], pages 45–52.

[HS94] Frank Halasz and Mayer Schwartz. The Dexter hypertext reference model (edited). *Communications of the ACM*, 37(2):30–39, February 1994.

[HS97] R. V. Guha. Hot Sauce. http://www.xspace.net/, 1997.

[HT91] ACM. *HT '91. Third ACM Conference on Hypertext*, December 15–18 1991.

[HT93] ACM. *HT '93. Proceedings of the Fifth ACM Conference on Hypertext*, November 14–18 1993.

[HT96] ACM. *HT '96. Proceedings of the the Seventh ACM conference on Hypertext*, March 16–20 1996.

[HT97] Mark Bernstein, Leslie Carr, and Kasper Østerbye, editors. *HT'97. Eighth ACM Conference on Hypertext*. ACM, April 6–11 1997.

[HT98] ACM. *HT '98. Ninth ACM Conference on Hypertext and Hypermedia: Links, Objects, Time and Space—Structure in Hypermedia Systems*, June 20–24 1998.

[HW92] Jörg M. Haake and Brian Wilson. Supporting collaborative writing of hyperdocuments in SEPIA. In [CSC92], pages 138–146.

[HW97] Jörg M. Haake and Weigang Wang. Flexible support for business processes: Extending cooperative hypermedia with process support. In [GRO97], pages 341–350.

[HW98] Jörg M. Haake and Weigang Wang. Collaboration support in open hypermedia environments. In Wiil [OHS98], pages 39–44.

[IMM95] Takeo Igarashi, Satoshi Matsuoka, and Toshiyuki Masui. Adaptive recognition of implicit structures in human-organized layouts. In *Proceedings 11th International IEEE Symposium on Visual Languages*, http://www.computer.org/conferen/vl95/GENHP.html, September 1995. IEEE.

[Joy91] Michael Joyce. Storyspace as a hypertext system for writers and readers of varying ability. In [HT91], pages 381–387.

[KL91] Charles J. Kacmar and John J. Leggett. PROXHY: a process-oriented extensible hypertext architecture. *ACM Transactions on Information Systems*, 9(4):399–419, October 1991.

[LS94] John J. Leggett and John L. Schnase. Viewing Dexter with open eyes. *Communications of the ACM*, 37(2):76–86, February 1994.

[Mar92] Catherine C. Marshall. Two years before the mist: Experiences with Aquanet. In [ECH92], pages 53–62.

[MB+90] Raymond J. McCall, Patrick R. Bennet, et al. PHIDIAS: Integrating CAD graphics into dynamic hypertext. In Rizk et al. [ECH90], pages 152–165.

[Mey86] Norman Meyrowitz. The missing link: Why we're all doing hypertext wrong. *The Society of Text*, pages 107–114, 1986.

[MH+91] Catherine C. Marshall, Frank G. Halasz, et al. Aquanet: a hypertext tool to hold your knowledge in place. In [HT91], pages 261–275.

[MS93] Catherine C. Marshall and Frank M. Shipman, III. Searching for the missing link: Discovering implicit structure in spatial hypertext. In [HT93], pages 217–230.

[MS95] Catherine C. Marshall and Frank M. Shipman, III. Spatial hypertext: designing for change. *Communications of the ACM*, 38(8):88–97, August 1995.

[MS97] Catherine C. Marshall and Frank M. Shipman, III. Spatial hypertext and the practice of information triage. In Bernstein et al. [HT97], pages 124–133.

[MSC94] Catherine C. Marshall, Frank M. Shipman, III, and James H. Coombs. VIKI: Spatial hypertext supporting emergent structure. In [ECH94], pages 13–23.

[NGS+98] Peter J. Nürnberg, Kaj Grønbæk, Lennert Sloth, Niels-Olof Bouvin, and Uffe K. Wiil. Notes on modification of the OHSWG navigation standard and related interfaces. Technical report, Department of Computer Science, Aarhus University, Denmark and Department of Computer Science, Aalborg University Esbjerg, Denmark, 1998.

[NLS97] Peter J. Nürnberg, John J. Leggett, and Erich R. Schneider. As we should have thought. In Bernstein et al. [HT97], pages 96–101.

[NLSS96] Peter J. Nürnberg, John J. Leggett, Erich R. Schneider, and John L. Schnase. Hypermedia operating systems: a new paradigm for computing. In [HT96].

[NLW98] Peter J. Nürnberg, John J. Leggett, and Uffe K. Wiil. An agenda for open hypermedia research. In [HT98].

[Nür97] Peter J. Nürnberg. *HOSS: An Environment to Support Structural Computing*. PhD thesis, Texas A&M University, August 1997.

[NWL98] Peter J. Nürnberg, Uffe K. Wiil, and John J. Leggett. Structuring facilities in digital libraries. In [ECD98].

[OHS98] Uffe K. Wiil, editor. *OHS4. Proceedings of the Fourth Workshop on Open Hypermedia Systems (HT '98)*. ACM, June 20–24 1998.

[OOP95] ACM. *OOPSLA '95. Proceedings of the tenth annual conference on Object-oriented programming systems, languages, and applications*, October 15–19 1995.

[Par91] H. Van Dyke Parunak. Don't link me in: set based hypermedia for taxonomic reasoning. In [HT91], pages 233–242.

[PH+90] John F. Patterson, Ralph D. Hill, et al. Rendezvous: An architecture for synchronous multi-user applications. In [CSC90], pages 317–328.

[RG92] Mark Roseman and Saul Greenberg. GROUPKIT: A groupware toolkit for building real-time conferencing applications. In [CSC92], pages 43–50.

[Ros96] Jim Rosenberg. The structure of hypertext activity. In [HT96], pages 22–30.

[SFB+98] Frank M. Shipman, III, Richard Furuta, Donald Brenner, Chung-Chi Chung, and Hao wei Hsieh. Using paths in the classroom: experiences and adaptations. In [HT98], pages 267–276.

[SGHH94] Norbert A. Streitz, Jörg Geißler, Jörg M. Haake, and Jeroen Hol. DOLPHIN: Integrated meeting support across local and remote desktop environments and liveboards. In [CSC94], pages 345–358.

[SH+92] Norbert A. Streitz, Jörg M. Haake, et al. SEPIA: A cooperative hypermedia authoring environment. In [ECH92], pages 11–22.

[SK+96] Christian Schuckmann, Lutz Kirchner, et al. Designing object-oriented synchronous groupware with COAST. In [CSC96], pages 30–38.

[SM93] Frank M. Shipman, III and Catherine C. Marshall. Formality considered harmful: Experiences, emerging themes, and directions. Technical Report CU-CS-648-93, Dept. CS, University of Colorado, Boulder, 1993.

[SMU95] Randall B. Smith, John Maloney, and David Ungar. The Self-4.0 user interface: Manifesting a system-wide vision of concreteness, uniformity, and flexibility. In [OOP95], pages 47–60.

[SS90] W. Schuler and J. B. Smith. Author's argumentation assistant (aaa): A hypertext-based authoring tool for argumentative texts. In Rizk et al. [ECH90], pages 137–151.

[Str94] Norbert A. Streitz. Putting objects to work: Hypermedia as the subject matter and the medium for computer-supported cooperative work. In M. Tokoro and R. Pareschi, editors, *ECOOP '94: Object-Oriented Programming.*, Lecture Notes in Computer Science, Berlin, July 4–8 1994. Springer Verlag.

[SU91] Randall B. Smith and David Ungar. SELF: The power of simplicity. *Lisp and symbolic computation*, 1991.

[WH97] Weigang Wang and Jörg M. Haake. Supporting user-defined activity spaces. In Bernstein et al. [HT97], pages 112–123.

[WH98] Weigang Wang and Jörg M. Haake. Flexible coordination with cooperative hypermedia. In [HT98], pages 245–255.

[Wiia] Uffe K. Wiil. Infrastructure. http://www.ohswg.org/infra/index.html.

[Wiib] Uffe K. Wiil. Message from the OHSWG chair. http://www.-ohswg.org/intro/chair.html.

[Wii93] Uffe K. Wiil. Experiences with HyperBase: A hypertext database supporting collaborative work. *SIGMOD*, 22(4), December 1993.

[WL92] Uffe K. Wiil and John J. Leggett. HyperForm: Using extensibility to develop dynamic, open and distributed hypertext systems. In [ECH92], pages 251–261.

[WL93] Uffe K. Wiil and John J. Leggett. Concurrency control in collaborative hypertext systems. In [HT93], pages 14–24.

[WL96] Uffe K. Wiil and John J. Leggett. The HyperDisco approach to open hypermedia systems. In [HT96].

[WL97] Uffe K. Wiil and John J. Leggett. Workspaces: the HyperDisco approach to Internet distribution. In Bernstein et al. [HT97].

[WN98] Uffe K. Wiil and Peter J. Nürnberg. Collaboration in open hypermedia environments. In Wiil [OHS98].

[WN99] Uffe K. Wiil and Peter J. Nürnberg. Evolving hypermedia middleware services: lessons and observations. In *Proceedings on Applied Computing, to appear*. ACM, 1999.